

尚硅谷大模型技术之 LangGraph

（作者：尚硅谷研究院）

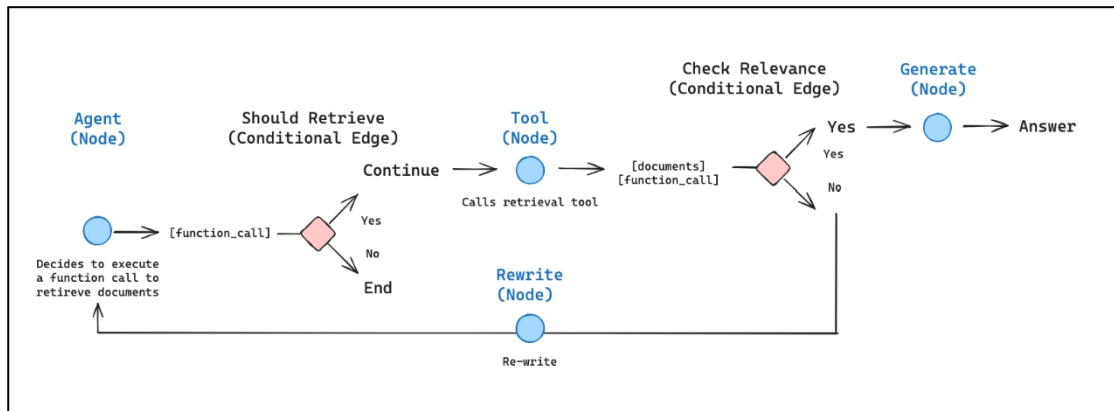
版本：V1.0.1

第 1 章 LangGraph 概述

1.1 什么是 LangGraph

LangGraph 是一个低级编排框架(可以直接掌控系统行为的原子级组件)和运行时环境，用于构建、管理和部署长期运行的有状态智能体（agents）。核心理念是将 Agent 工作流建模为图（Graph），其中：

- 节点（Nodes）：代表计算单元，可以是 LLM 调用、工具执行或任何自定义逻辑
- 边（Edges）：定义节点之间的转换逻辑，决定执行流程
- 状态（State）：在整个图执行过程中共享和传递的数据



LangGraph 提供了构建生产级智能体应用的核心能力：

- 持久化执行：构建能够从故障中恢复并长时间运行的智能体
- 人机协作：在任何时刻检查和修改智能体状态
- 记忆管理：支持短期工作记忆和跨会话的长期记忆
- 流式处理：专为流式工作流设计
- 生产级部署：为有状态、长期运行的工作流提供可扩展的基础设施

LangGraph 可独立使用，也可与 LangChain、LangSmith 无缝集成。

LangGraph 与 LangChain 的高级抽象不同，它提供了更细粒度的控制，让开发者能够精确控制智能体的执行流程，适合需要定制化编排的复杂应用场景。

- GitHub 地址: <https://github.com/langchain-ai/langgraph>
- 官方文档: <https://docs.langchain.com/oss/python/langgraph/overview>

1.2 为什么需要 LangGraph

随着应用复杂度的提升，传统的 Agent 框架往往面临着状态管理混乱、执行流程不可控、错误恢复困难等挑战。大语言模型的使用不仅仅是作为执行工具，而更多作为推理引擎的需求在日益增长。这种转变带来的是**更多的重复（循环）**和**复杂条件**的交互需求，这就导致基于 LCEL(LangChain Expression Language)的线性序列构建方式在构建更复杂、更智能的系统时显示出了明显的局限性。

LangGraph 作为 LangChain 团队推出的新一代 Agent 框架，通过引入图计算模型和状态机理念，为构建生产级 AI Agent 提供了全新的解决方案。

1.3 LangGraph 使用场景

- 复杂的多智能体系统
- 需要长期记忆的应用
- 需要人工审核的工作流
- 后台处理任务和实时交互
- 需要精细控制的定制化智能体编排

1.4 与 LangChain 区别

特性	LangGraph	LangChain
抽象级别	低级，提供细粒度控制	高级，提供开箱即用的链
状态管理	内置状态机和检查点	需要自行管理状态
执行模型	基于图的并行执行	线性链式执行
持久化	原生支持	需要额外实现
适用场景	复杂、有状态的工作流	简单的链式调用

1.5 LangGraph 架构设计

1) Pregel 架构

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

LangGraph 的运行基于 Google 的 Pregel 算法，这是一种用于大规模并行图计算的模型。执行过程分为三个阶段：

- (1) Plan（规划）：确定本轮要执行的节点
- (2) Execution（执行）：并行执行所有选中的节点
- (3) Update（更新）：将节点输出更新到通道（channels）

每个执行轮次称为一个“超步（super-step）”，系统会持续迭代直到没有节点需要执行。

2) Actors (PregelNode)

订阅通道、读取和写入数据的节点，实现 LangChain 的 Runnable 接口。

3) Channels（通道）

用于 actors 之间通信，包括：

- LastValue: 存储最后发送的值
- Topic: 可配置的发布-订阅主题
- BinaryOperatorAggregate: 用于聚合操作

第 2 章 快速入门

2.1 环境安装

1) 创建环境

```
conda create -n langgraph python==3.12
conda activate langgraph
```

2) 执行安装

```
pip install -U langgraph
```

3) 确认成功

```
pip show langgraph
```

2.2 简单示例

```
from langgraph.graph import StateGraph, MessagesState, START, END
def this_is_a_method(params1:int,param2:int):
    print("this is a print")
# 1、定义节点函数
def mock_llm(state: MessagesState):
    """ 模拟调用 LLM """
    return {"messages": [{"role": "ai", "content": "hello world"}]}
```

2、定义图

```
graph = StateGraph(MessagesState)
```

3、添加节点和边

```
graph.add_node(mock_llm)
graph.add_edge(START, "mock_llm")
graph.add_edge("mock_llm", END)
```

4、编译图

```
graph = graph.compile()
```

5、调用图

```
response = graph.invoke({"messages": [{"role": "user", "content": "hi!"}]}))
print(response)
```

第 3 章 Graph API

从核心来看，LangGraph 将智能体 workflow 建模为图。你可以使用三个关键组件来定义智能体的行为：

- (1) **State**: 一种共享数据结构，用于表示应用程序的当前快照。它可以是任何数据类型，但通常使用共享状态模式来定义。
- (2) **Nodes**: 对智能体逻辑进行编码的函数。它们接收当前状态作为输入，执行一些计算或副作用，并返回更新后的状态。
- (3) **Edges**: 根据当前状态确定下一个要执行的 Node 的函数。它们可以是条件分支或固定转换。

节点执行工作，边指示下一步该做什么。通过组合 Nodes 和 Edges，可以创建复杂的、循环的工作流，这些工作流会随着时间推移不断演化状态。不过，真正的强大之处在于 LangGraph 对该状态的管理方式。Nodes 和 Edges 只不过是函数——它们可以包含大语言模型，或者仅仅是一些传统代码。

StateGraph 类是要使用的主要图类。它由用户定义的 State 对象进行参数化。

要构建图，首先需要定义状态，然后添加节点和边，接着进行编译。

3.1 State

定义图时，首先要做的是定义图的 State。State 由图的 schema 以及 reducer 函数组成，其中 reducer 函数指定了如何对状态进行更新。State 的 schema 将作为图中所有 Nodes 和 Edges

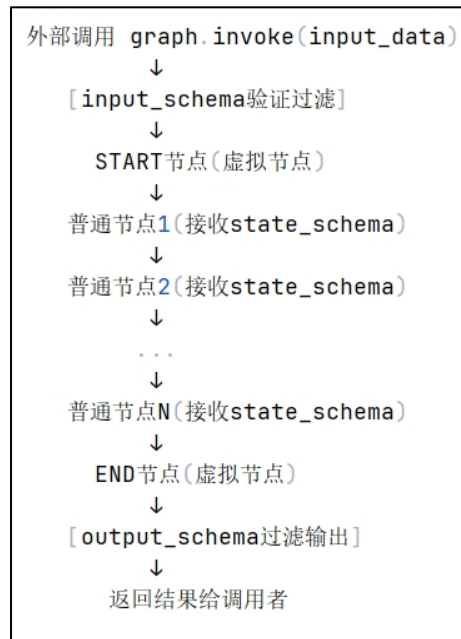
更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

的输入模式，它可以是 TypedDict 或 Pydantic 模型。所有 Nodes 都会发出对 State 的更新，然后使用指定的 reducer 函数应用这些更新。

3.1.1 Schema

指定图的架构的主要方式是使用 TypedDict。如果想在状态中提供默认值，请使用 dataclass。

LangGraph 中 state_schema、input_schema 和 output_schema 这三个概念是用于管理图状态的不同方面：



1) state_schema

这是图的完整内部状态，包含了所有节点可能读写的字段，必须指定，不能为空。

特点：

- 是图的"全局状态空间"
- 所有节点都可以访问和写入这个 schema 中的任何字段

2) input_schema

定义图接受什么输入，是 state_schema 的子集

特点：

- 可选参数，如果不指定，默认等于 state_schema
- 限制图的输入接口，只能传入这些字段
- 是 state_schema 的子集或相等

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

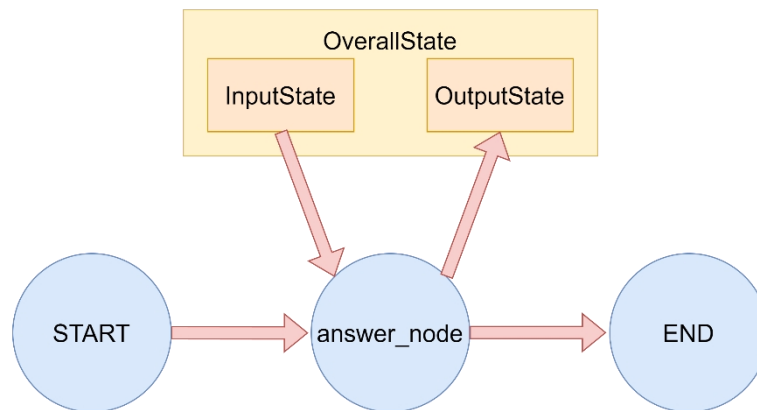
3) output_schema

定义图返回什么输出，是 `state_schema` 的子集

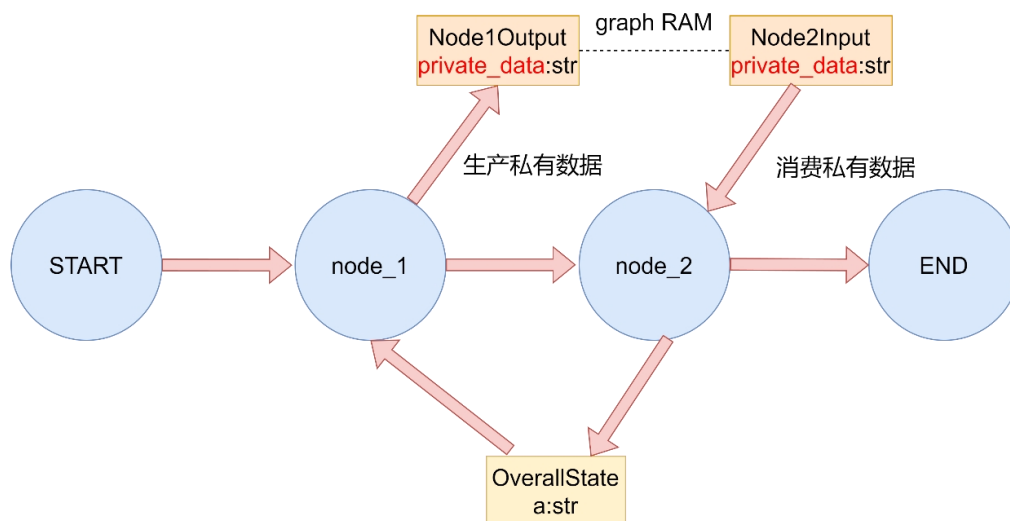
特点：

- 可选参数，如果不指定，默认等于 `state_schema`
- 限制图的输出接口，只返回这些字段
- 是 `state_schema` 的子集或相等

下图中展示了一个**输入输出状态分离**的图结构示例，其中 `OverallState` 为该图的主状态，其中包含了 `InputState` 和 `OutputState` 作为子输入和输出状态，用于被读取和被写入。`OverallState` 继承了 `InputState` 和 `OutputState` 中所有的状态信息。



下图中展示了一个**私有状态传递**的图结构，`OverallState` 为图中的主状态，其中 `node_1` 接收 `OverallState`，将输出写入到私有状态 `Node1Output` 中的 `private_data` 字段中。但此时图中的主状态并没有该字段，于是将 `private_data` 字段与对应的数据构成的 kv 对暂存到 RAM 中。在进入下一个节点 `node_2` 的时候，接收参数为 `Node2Input` 中的同名字段 `private_data`。图的管理行为会提取出该字段，并在暂存区寻找有无对应的字段，发现存在，对 `private_data` 进行消费，并将 `node_1` 赋的值传递到 `node_2` 中，在进行了消费之后销毁内存中的该字段，实现私有字段的传递。



"""

LangGraph 图输入输出模式和私有状态传递演示

该演示展示了：

1. 如何定义图的输入和输出模式
2. 如何在节点间传递私有状态

"""

```
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict
```

定义输入状态模式

```
class InputState(TypedDict):
    question: str
```

定义输出状态模式

```
class OutputState(TypedDict):
    answer: str
```

定义整体状态模式，结合输入和输出

```
class OverallState(InputState, OutputState):
    pass
```

定义处理节点

```
def answer_node(state: InputState):
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
"""
    处理输入并生成答案的节点

    Args:
        state: 输入状态

    Returns:
        dict: 包含答案的字典
    """
    print(f"执行 answer_node 节点:")
    print(f"  输入: {state}")

    # 示例答案
    answer = "再见" if "bye" in state["question"].lower() else "你好"
    result = {"answer": answer, "question": state["question"]}

    print(f"  输出: {result}")
    return result


# 定义整体状态（这是在节点间共享的公共状态）
class OverallStatePrivate(TypedDict):
    a: str


# node_1 的输出包含不属于整体状态的私有数据
class Node1Output(TypedDict):
    private_data: str


# 私有数据仅在 node_1 和 node_2 之间共享
def node_1(state: OverallStatePrivate) -> Node1Output:
    """
        第一个节点，生成私有数据

        Args:
            state: 整体状态

        Returns:
            dict: 包含私有数据的字典
    """
```



```
output = {"private_data": "由 node_1 设置"}
print(f"进入 node_1 节点:")
print(f"  输入: {state}")
print(f"  返回: {output}")
return output

# 节点 2 的输入只请求 node_1 之后可用的私有数据
class Node2Input(TypedDict):
    private_data: str

def node_2(state: Node2Input) -> OverallStatePrivate:
    """
    第二个节点，可以访问 node_1 的私有数据

    Args:
        state: 包含私有数据的输入状态

    Returns:
        dict: 更新的整体状态
    """
    output = {"a": "由 node_2 设置"}
    print(f"进入 node_2 节点:")
    print(f"  输入: {state}")
    print(f"  返回: {output}")
    return output

# 节点 3 只能访问整体状态（无法访问 node_1 的私有数据）
def node_3(state: OverallStatePrivate) -> OverallStatePrivate:
    """
    第三个节点，只能访问整体状态

    Args:
        state: 整体状态

    Returns:
        dict: 更新的整体状态
    """
    output = {"a": "由 node_3 设置"}
```

```
print(f"进入 node_3 节点:")
print(f"  输入: {state}")
print(f"  返回: {output}")
return output

def demo_input_output_schema():
    """演示输入输出模式"""
    print("=== 演示输入输出模式 ===")

    # 使用指定的输入和输出模式构建图
    builder = StateGraph(OverallState, input_schema=InputState,
output_schema=OutputState)
    builder.add_node("answer_node", answer_node) # 添加答案节点
    builder.add_edge(START, "answer_node") # 定义起始边
    builder.add_edge("answer_node", END) # 定义结束边
    graph = builder.compile() # 编译图

    # 使用输入调用图并打印结果
    result = graph.invoke({"question": "你好"})
    print(f"图调用结果: {result}")

    print()

def demo_private_state():
    """演示私有状态传递"""
    print("=== 演示私有状态传递 ===")

    # 连接节点序列
    # node_2 接受来自 node_1 的私有数据, 而
    # node_3 看不到来自 node_1 的私有数据
    builder = StateGraph(OverallStatePrivate).add_sequence([node_1,
node_2, node_3])
    builder.add_edge(START, "node_1")
    graph = builder.compile()

    # 使用初始状态调用图
    response = graph.invoke({
        "a": "在开始时设置",
    })
```

```
print()
print(f"图调用的输出: {response}")
print()

def main():
    """主函数"""
    print("=== LangGraph 图输入输出模式和私有状态传递演示 ===\n")

    # 演示输入输出模式
    demo_input_output_schema()

    # 演示私有状态传递
    demo_private_state()

    print("=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

3.1.2 Reducer

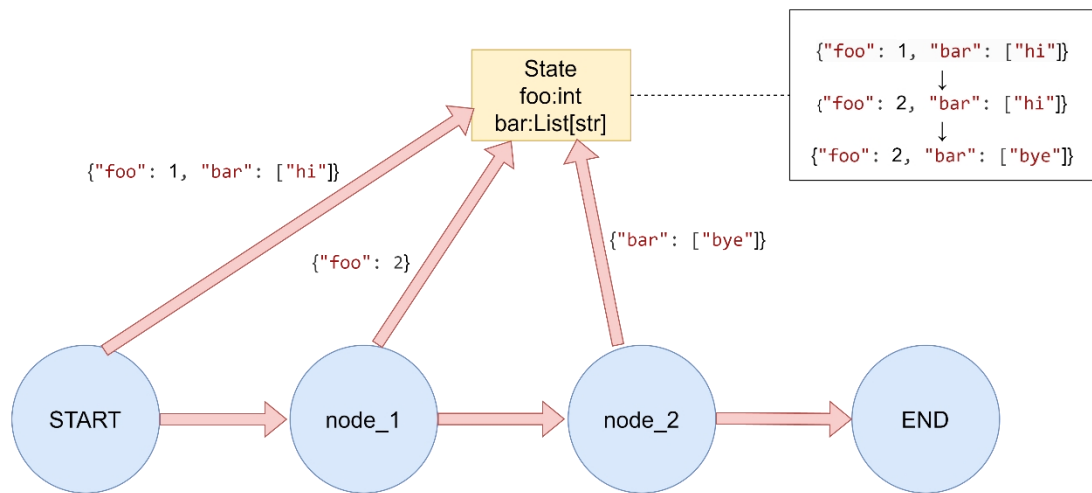
reducer 是理解节点更新如何应用于 State 的关键, State 中的每个键都有其独立的 reducer 函数。每个 node 的返回值中的每个 key 与全局 state_schema 中对应的 key 进行合并更新, 具体更新逻辑取决于每个 key 指定的 reducer 函数。

Reducer 常用函数有以下几种:

- 默认行为: 未指定 Reducer 时使用覆盖更新
- add_messages: 用于消息列表追加
- operator.add: 用于列表追加或数值累加
- operator.mul: 用于数值相乘
- 自定义 Reducer: 支持用户自定义合并逻辑

3.1.2.1 默认覆盖

如果未明确指定 reducer 函数, 则默认对该键的更新是覆盖行为。



```

"""
LangGraph Reducer 函数演示 - 默认 Reducer（覆盖更新）
"""

from typing import List
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 1. 默认 Reducer（覆盖更新）
class DefaultReducerState(TypedDict):
    foo: int
    bar: List[str]

def node_default_1(state: DefaultReducerState) -> dict:
    return {"foo": 2}

def node_default_2(state: DefaultReducerState) -> dict:
    return {"bar": ["bye"]}

def run_demo():
    print("1. 默认 Reducer（覆盖更新）演示:")
    builder = StateGraph(DefaultReducerState)
    builder.add_node("node1", node_default_1)
    builder.add_node("node2", node_default_2)
    builder.add_edge(START, "node1")
    builder.add_edge("node1", "node2")
    builder.add_edge("node2", END)
    graph = builder.compile()

```

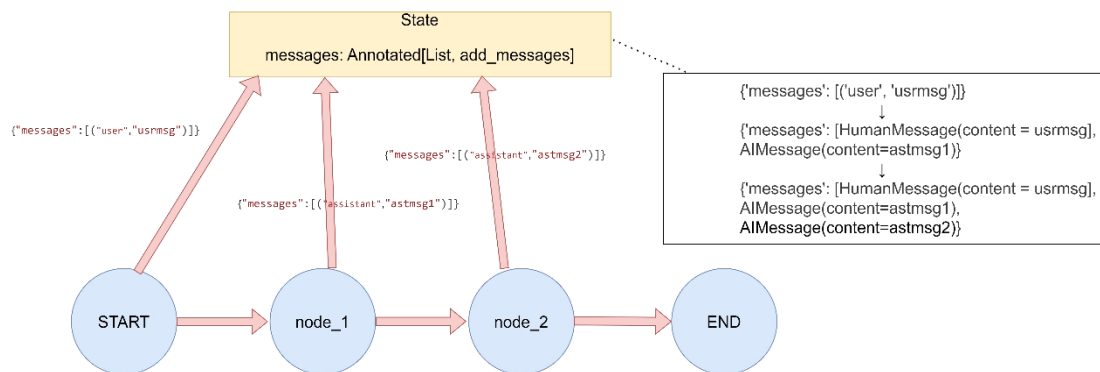
更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
result = graph.invoke({"foo": 1, "bar": ["hi"]})
print(f"初始状态: {{'foo': 1, 'bar': ['hi']}}")
print(f"执行结果: {result}\n")

if __name__ == "__main__":
    run_demo()
```

3.1.2.2 add_messages

专门用于处理消息列表，当使用 `add_messages` 时，消息会被转换为 `LangChain` 的消息对象（如 `HumanMessage`、`AIMessage` 等）。



```
"""
LangGraph Reducer 函数演示 - add_messages Reducer（消息列表专用）
"""
```

```
from typing import Annotated, List
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

# 2. add_messages Reducer（消息列表专用）
class AddMessagesState(TypedDict):
    messages: Annotated[List, add_messages]

def chat_node_1(state: AddMessagesState) -> dict:
    return {"messages": [("assistant", "Hello from node 1")]}

def chat_node_2(state: AddMessagesState) -> dict:
    return {"messages": [("assistant", "Hello from node 2")]}
```

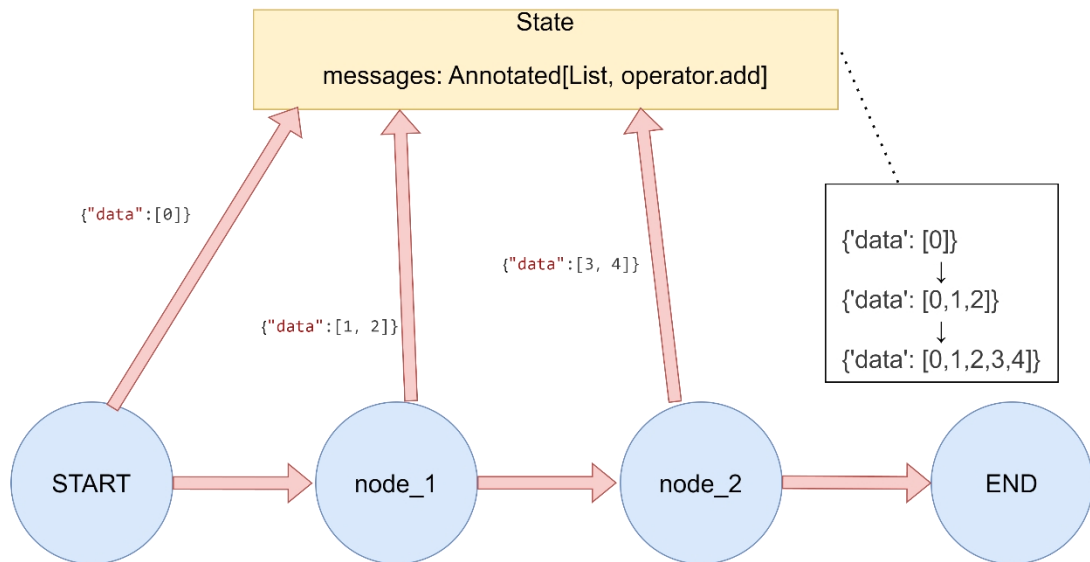
```
def run_demo():
    print("2. add_messages Reducer（消息列表专用）演示:")
    builder = StateGraph(AddMessagesState)
    builder.add_node("chat1", chat_node_1)
    builder.add_node("chat2", chat_node_2)
    builder.add_edge(START, "chat1")
    builder.add_edge(START, "chat2") # 并行执行
    builder.add_edge("chat1", END)
    builder.add_edge("chat2", END)
    graph = builder.compile()

    result = graph.invoke({"messages": [("user", "Hi there!")]})
    print(f"初始状态: {'messages': [('user', 'Hi there!')]}")
    print(f"执行结果: {result}\n")

if __name__ == "__main__":
    run_demo()
```

3.1.2.3 operator.add

将元素追加到现有元素中，支持列表、字符串、数值类型的追加。



1) 列表追加

```
"""
LangGraph Reducer 函数演示 - operator.add Reducer（列表追加）
"""

import operator
```

```
from typing import Annotated, List
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 3. operator.add Reducer (列表追加)
class ListAddState(TypedDict):
    data: Annotated[List[int], operator.add]

def producer_1(state: ListAddState) -> dict:
    return {"data": [1, 2]}

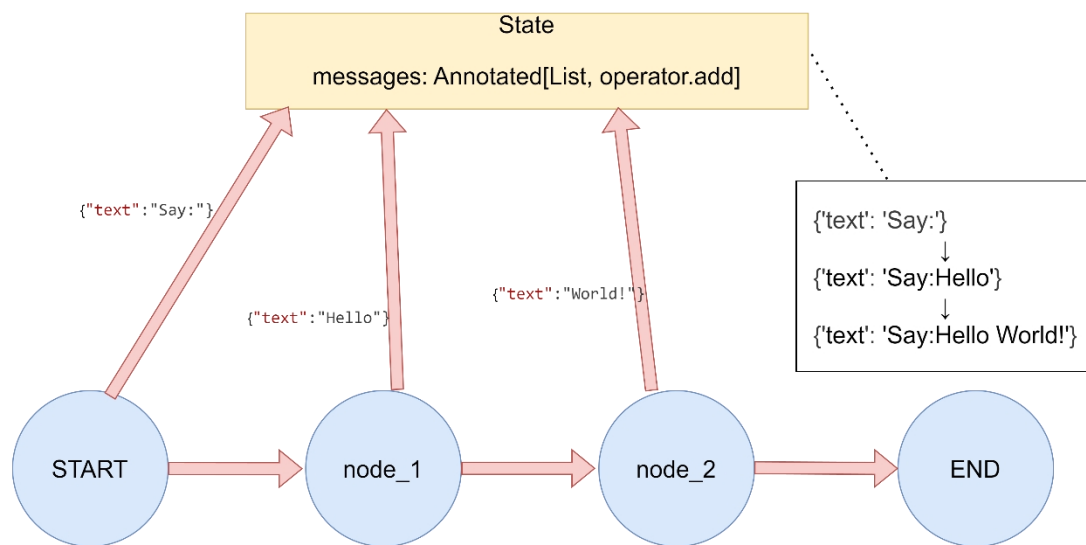
def producer_2(state: ListAddState) -> dict:
    return {"data": [3, 4]}

def run_demo():
    print("3.1 operator.add Reducer (列表追加) 演示:")
    builder = StateGraph(ListAddState)
    builder.add_node("producer1", producer_1)
    builder.add_node("producer2", producer_2)
    builder.add_edge(START, "producer1")
    builder.add_edge(START, "producer2") # 并行执行
    builder.add_edge("producer1", END)
    builder.add_edge("producer2", END)
    graph = builder.compile()

    result = graph.invoke({"data": [0]})
    print(f"初始状态: {'data': [0]}")
    print(f"执行结果: {result}\n")

if __name__ == "__main__":
    run_demo()
```

2) 字符串连接



```

"""
LangGraph Reducer 函数演示 - 字符串连接 Reducer
"""

import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 6. 字符串连接 Reducer
class StringConcatState(TypedDict):
    text: Annotated[str, operator.add]

def add_text_1(state: StringConcatState) -> dict:
    return {"text": "Hello "}

def add_text_2(state: StringConcatState) -> dict:
    return {"text": "World!"}

def run_demo():
    print("3.2 字符串连接 Reducer 演示:")
    builder = StateGraph(StringConcatState)
    builder.add_node("add_text_1", add_text_1)
    builder.add_node("add_text_2", add_text_2)
    builder.add_edge(START, "add_text_1")
    builder.add_edge(START, "add_text_2") # 并行执行
    builder.add_edge("add_text_1", END)
    
```

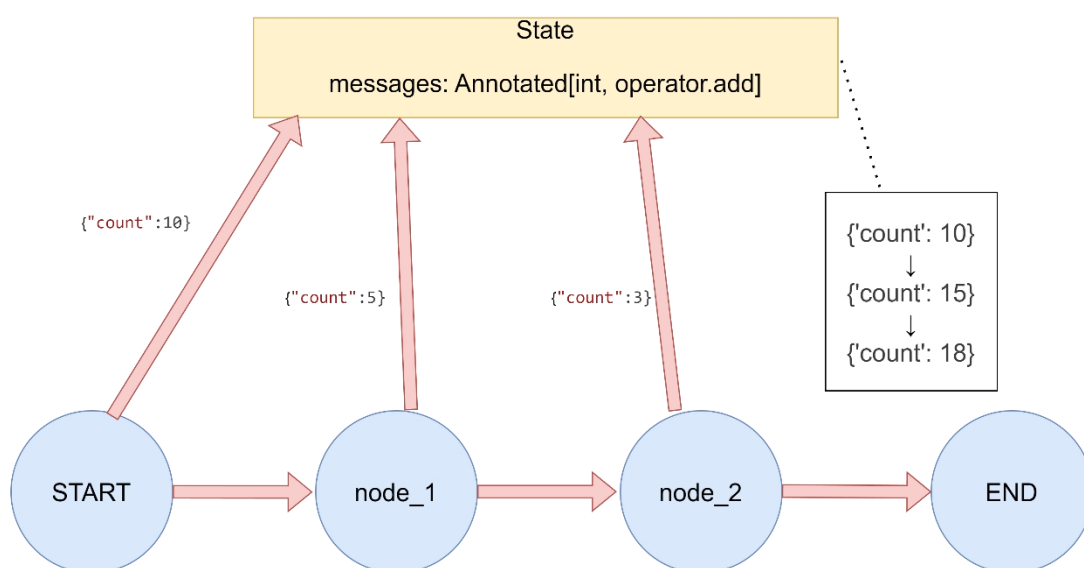
更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网


```
builder.add_edge("add_text_2", END)
graph = builder.compile()

result = graph.invoke({"text": "Say: "})
print(f"初始状态: {{'text': 'Say: '}}")
print(f"执行结果: {result}\n")

if __name__ == "__main__":
    run_demo()
```

3) 数值累加



```
"""
LangGraph Reducer 函数演示 - 数值累加 Reducer
"""

import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 7. 数值累加 Reducer
class NumberAddState(TypedDict):
    count: Annotated[int, operator.add]

def increment_1(state: NumberAddState) -> dict:
    return {"count": 5}
```

```
def increment_2(state: NumberAddState) -> dict:
    return {"count": 3}

def run_demo():
    print("3.3 数值累加 Reducer 演示:")
    builder = StateGraph(NumberAddState)
    builder.add_node("increment_1", increment_1)
    builder.add_node("increment_2", increment_2)
    builder.add_edge(START, "increment_1")
    builder.add_edge(START, "increment_2") # 并行执行
    builder.add_edge("increment_1", END)
    builder.add_edge("increment_2", END)
    graph = builder.compile()

    result = graph.invoke({"count": 10})
    print(f"初始状态: {'count': 10}")
    print(f"执行结果: {result}\n")

if __name__ == "__main__":
    run_demo()
```

3.1.2.4 operator.mul

用于数值字段的相乘操作，官方设计上存在缺陷。

```
"""
LangGraph Reducer 函数演示 - operator.mul Reducer（数值相乘）
"""

import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 4. operator.mul Reducer（数值相乘）
class MultiplyState(TypedDict):
    factor: Annotated[float, operator.mul]

def multiplier(state: MultiplyState) -> dict:
    return {"factor": 2.0}

def run_demo():
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```

"""
!!! operator.mul 实际使用上，官方设计存在 bug: !!!
在执行初始阶段（我们定义的第一个 node 前），会默认调用一次 reducer（后面自定义 reducer 案例中进行了打印验证），用默认值与 invoke 传递的值进行计算：
此案例中，invoke 中传递了一个默认值 5.0，由于会默认调用一次 reducer，执行的计算是：0.0（float 默认值） * 5.0(invoke 传递的初始值) = 0.0
导致后续乘法结果一直都是 0

解决方案： 使用自定义 reducer
"""

print("4. operator.mul Reducer（数值相乘）演示:")
builder = StateGraph(MultiplyState)
builder.add_node("multiplier", multiplier)
builder.add_edge(START, "multiplier")
builder.add_edge("multiplier", END)
graph = builder.compile()

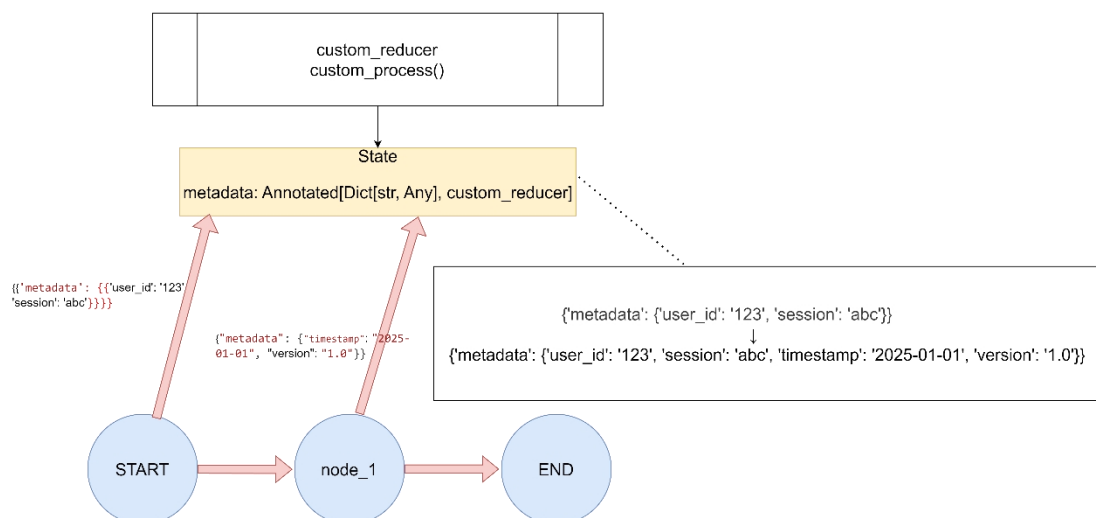
result = graph.invoke({"factor": 5.0})
print(f"初始状态: {{'factor': 5.0}}")
print(f"执行结果: {result}\n")

if __name__ == "__main__":
    run_demo()

```

3.1.2.5 自定义 Reducer 函数

1) 案例一



```

"""
LangGraph Reducer 函数演示 - 自定义 Reducer 函数
更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网

```

```
"""

from typing import Annotated, Dict, Any
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 5. 自定义 Reducer 函数
def custom_reducer(current_value: Dict[str, Any], new_value: Dict[str, Any])
-> Dict[str, Any]:
    """合并两个字典，新值会覆盖旧值，但保留旧值中不存在的键"""
    result = current_value.copy()
    result.update(new_value)
    return result

class CustomReducerState(TypedDict):
    metadata: Annotated[Dict[str, Any], custom_reducer]

def update_metadata(state: CustomReducerState) -> dict:
    return {"metadata": {"timestamp": "2025-01-01", "version": "1.0"}}

def run_demo():
    print("5. 自定义 Reducer 演示:")
    builder = StateGraph(CustomReducerState)
    builder.add_node("update_metadata", update_metadata)
    builder.add_edge(START, "update_metadata")
    builder.add_edge("update_metadata", END)
    graph = builder.compile()

    result = graph.invoke({"metadata": {"user_id": "123", "session":
"abc"}})
    print(f"初始状态: {{'metadata': {{'user_id': '123', 'session':
'abc'}}}}")
    print(f"执行结果: {result}\n")

if __name__ == "__main__":
    run_demo()
```

2) 案例二：自定义 reducer 实现数值乘法

```
"""

LangGraph Reducer 函数演示 - 自定义 mul reducer 实现数值相乘
使用全局变量区分初始化调用和正常调用
更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网
```

```
"""

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 使用全局变量来跟踪是否是第一次调用（初始化阶段）
_is_initial_call = True

def my_mul_reducer(current_value: float, new_value: float) -> float:
    """
    自定义乘法 reducer，使用全局变量区分初始化调用和正常调用

    Args:
        current_value: 当前状态值
        new_value: 新值

    Returns:
        计算后的结果
    """
    global _is_initial_call

    print(f"Reducer 被调用: current_value={current_value},\nnew_value={new_value}, is_initial_call={_is_initial_call}")

    # 如果是初始化调用，直接返回 new_value，避免默认值 0 的影响
    if _is_initial_call:
        _is_initial_call = False # 重置标志
        return new_value
    else:
        # 正常的乘法操作，包括乘以 0 的情况
        return current_value * new_value

class MultiplyState(TypedDict):
    factor: Annotated[float, my_mul_reducer]

def multiplier_by_two(state: MultiplyState) -> dict:
    """将 factor 乘以 2"""
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
        return {"factor": 2.0}

def multiplier_by_zero(state: MultiplyState) -> dict:
    """将 factor 乘以 0"""
    return {"factor": 0.0}

def run_demo():
    """
    演示增强版乘法 reducer 的使用
    """
    global _is_initial_call

    print("=== operator.mul 增强版解决方案演示 ===\n")

    # 演示 1: 正常乘法操作
    print("1. 正常乘法操作演示:")
    _is_initial_call = True # 重置初始化标志
    builder = StateGraph(MultiplyState)
    builder.add_node("multiplier_by_two", multiplier_by_two)
    builder.add_edge(START, "multiplier_by_two")
    builder.add_edge("multiplier_by_two", END)
    graph = builder.compile()

    result = graph.invoke({"factor": 5.0})
    print(f"初始状态: {'factor': 5.0}")
    print(f"执行结果: {result}")
    print(f"预期结果: 10.0 (5.0 * 2.0)\n")

    # 演示 2: 乘以 0 的操作
    print("2. 乘以 0 的操作演示:")
    _is_initial_call = True # 重置初始化标志
    builder2 = StateGraph(MultiplyState)
    builder2.add_node("multiplier_by_zero", multiplier_by_zero)
    builder2.add_edge(START, "multiplier_by_zero")
    builder2.add_edge("multiplier_by_zero", END)
    graph2 = builder2.compile()

    result2 = graph2.invoke({"factor": 5.0})
    print(f"初始状态: {'factor': 5.0}")
```

```
print(f"执行结果: {result2}")
print(f"预期结果: 0.0 (5.0 * 0.0)\n")

# 演示 3: 连续乘法操作
print("3. 连续乘法操作演示:")
_is_initial_call = True # 重置初始化标志
builder3 = StateGraph(MultiplyState)
builder3.add_node("multiplier_by_two_1", multiplier_by_two)
builder3.add_node("multiplier_by_zero", multiplier_by_zero)
builder3.add_node("multiplier_by_two_2", multiplier_by_two)

builder3.add_edge(START, "multiplier_by_two_1")
builder3.add_edge("multiplier_by_two_1", "multiplier_by_zero")
builder3.add_edge("multiplier_by_zero", "multiplier_by_two_2")
builder3.add_edge("multiplier_by_two_2", END)

graph3 = builder3.compile()

result3 = graph3.invoke({"factor": 3.0})
print(f"初始状态: {'factor': 3.0}")
print(f"执行结果: {result3}")
print(f"预期过程: 3.0 -> 6.0 -> 0.0 -> 0.0")
print(f"预期结果: 0.0\n")

if __name__ == "__main__":
    run_demo()
```

3.1.2.6 实际应用示例

一个完整的聊天机器人示例。

```
from typing import Annotated, List
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
import operator

class ChatState(TypedDict):
    messages: Annotated[List, add_messages] # 消息历史
    tags: Annotated[List[str], operator.add] # 标签列表
    score: Annotated[float, operator.add] # 累计分数
```

```
def process_user_message(state: ChatState) -> dict:
    user_message = state["messages"][-1] # 获取最新消息
    # 修复: 正确访问消息内容
    return {
        "messages": [("assistant", f"Echo: {user_message.content}")],
        "tags": ["processed"],
        "score": 1.0
    }

def add_sentiment_tag(state: ChatState) -> dict:
    return {
        "tags": ["positive"],
        "score": 0.5
    }

# 构建图
builder = StateGraph(ChatState)
builder.add_node("process", process_user_message)
builder.add_node("sentiment", add_sentiment_tag)

builder.add_edge(START, "process")
builder.add_edge(START, "sentiment")
builder.add_edge("process", END)
builder.add_edge("sentiment", END)

graph = builder.compile()

# 使用示例 -使用正确的消息格式
result = graph.invoke({
    "messages": [{"role": "user", "content": "Hello, how are you?"}],
    "tags": ["greeting"],
    "score": 0.0
})

print(result)
```

3.1.2.7 总结

Reducer 函数在 LangGraph 中的作用:

- 控制状态更新方式: 决定新值如何与现有值合并。
- 处理并行更新: 当多个节点同时更新同一字段时, 确保数据一致性。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

- 提供灵活性：支持不同的合并策略，如覆盖、追加、相加等。
- 增强表达力：允许开发者根据业务需求自定义合并逻辑。

通过合理使用 **Reducer** 函数，可以构建更强大和灵活的状态管理机制，特别是在处理复杂工作流和并行执行场景时。

3.2 Nodes

在 LangGraph 中，节点是 Python 函数（可以是同步的，也可以是异步的），它们接受以下参数：

- **state**：图的状态
- **config**：一个 **RunnableConfig** 对象，包含诸如 **thread_id** 之类的配置信息以及诸如 **tags** 之类的跟踪信息
- **runtime**：一个 **Runtime** 对象，包含运行时 **context** 以及其他信息，如 **store** 和 **stream_writer**

定义好 **node** 函数后，使用 **add_node** 方法将这些节点添加到图中。如果在向图中添加节点时未指定名称，系统会为其分配一个与函数名相同的默认名称。

3.2.1 START Node

START 节点是一个特殊节点，它代表着将用户输入发送到图中的节点。引用此节点的主要目的是确定应首先调用哪些节点。

3.2.2 END Node

END 节点是一个特殊节点，代表终端节点。当你想表示哪些边在完成后没有动作时，会引用这个节点。

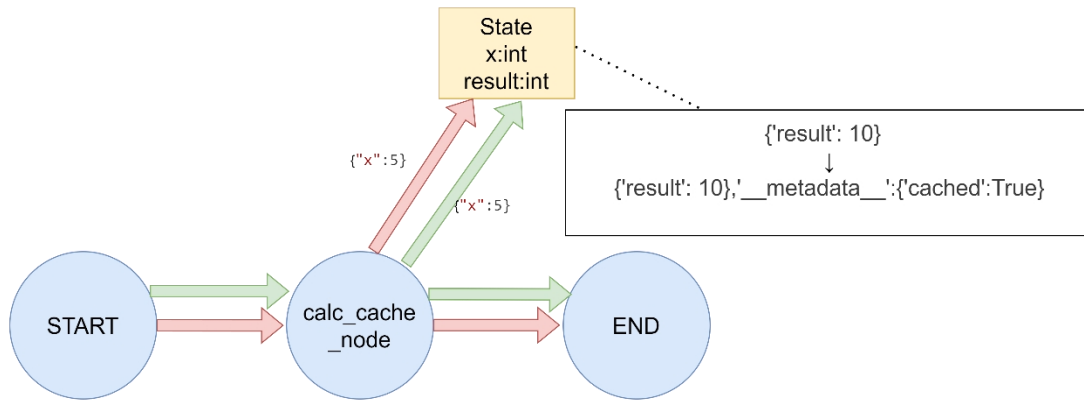
3.2.3 Node Caching

LangGraph 支持基于节点输入对任务/节点进行缓存。使用缓存的方法如下：

编译图（或指定入口点）时指定缓存。

为节点指定缓存策略。每个缓存策略支持：

- **key_func** 用于根据节点的输入生成缓存键，默认情况下是使用 **pickle** 对输入进行 **hash** 运算的结果。
- **ttl**，即缓存的生存时间（以秒为单位）。如果未指定，缓存将永不过期。



```
import time
from typing_extensions import TypedDict
from langgraph.graph import StateGraph
from langgraph.cache.memory import InMemoryCache
from langgraph.types import CachePolicy

# 定义状态
class State(TypedDict):
    x: int
    result: int

# 创建图
builder = StateGraph(State)

# 定义节点
def expensive_node(state: State) -> dict[str, int]:
    # expensive computation
    time.sleep(2)
    return {"result": state["x"] * 2}

# 添加节点，并设置缓存策略
builder.add_node("expensive_node", expensive_node,
    cache_policy=CachePolicy(ttl=3))

# 设置入口和出口
builder.set_entry_point("expensive_node")
builder.set_finish_point("expensive_node")

# 编译图
graph = builder.compile(cache=InMemoryCache())
```

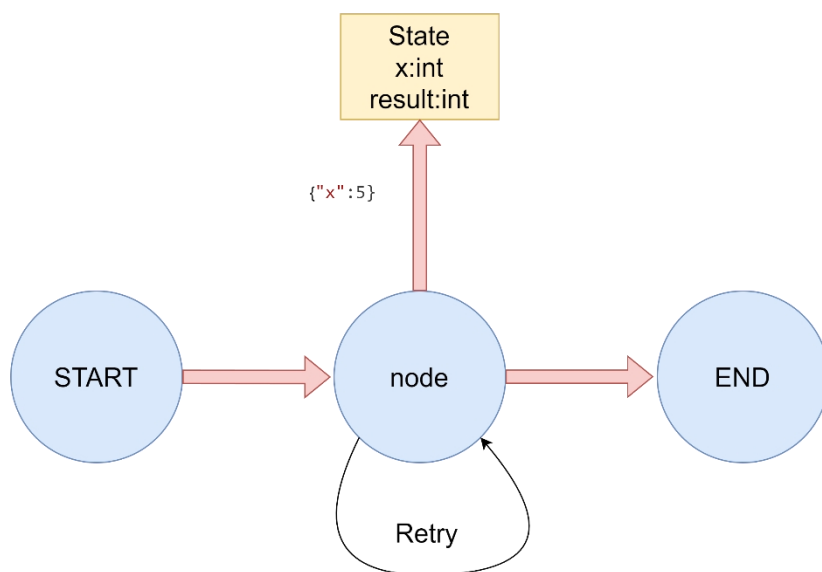
```
# 执行图
print(graph.invoke({"x": 5}, stream_mode='updates'))
# [{'expensive_node': {'result': 10}}]
# 第二次运行利用缓存并快速返回
print(graph.invoke({"x": 5}, stream_mode='updates'))
# [{'expensive_node': {'result': 10}, '__metadata__': {'cached': True}}]
```

3.2.4 添加重试策略

在很多使用场景中，你可能希望节点拥有自定义的重试策略，例如在调用 API、查询数据库或调用大语言模型（LLM）等情况下。

为节点添加重试策略，需要在 `add_node` 中设置 `retry_policy` 参数。`retry_policy` 参数接受一个 `RetryPolicy` 命名元组对象。默认情况下，`retry_on` 参数使用 `default_retry_on` 函数，该函数会在遇到任何异常时重试，除了以下情况：

- `ValueError`（值错误）
- `TypeError`（类型错误）
- `ArithmeticError`（算术错误）
- `ImportError`（导入错误）
- `LookupError`（查找错误）
- `NameError`（名称错误）
- `SyntaxError`（语法错误）
- `RuntimeError`（运行时错误）
- `ReferenceError`（引用错误）
- `StopIteration`（停止迭代）
- `StopAsyncIteration`（停止迭代）
- `OSError`（操作系统错误）



```

"""
LangGraph 节点重试策略演示
"""

import random
from typing import Dict, Any
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import RetryPolicy

# 定义状态
class State(TypedDict):
    result: str

# 模拟不稳定的 API 调用，使用全局变量跟踪尝试次数
attempt_counter = 0

def unstable_api_call(state: State) -> Dict[str, Any]:
    """
    模拟一个不稳定的 API 调用，有一定概率失败
    """
    global attempt_counter
    attempt_counter += 1
    print(f"尝试调用 API，这是第 {attempt_counter} 次尝试")
    
```

```
# 模拟前几次尝试失败，最后一次成功
if attempt_counter < 3:
    raise Exception(f"模拟 API 调用失败 (尝试 {attempt_counter})")
else:
    # 第三次尝试成功
    return {
        "result": f"API 调用成功，经过 {attempt_counter} 次尝试"
    }

# 自定义重试策略
def custom_retry_on(exception: Exception) -> bool:
    """
    自定义重试条件：只对特定错误进行重试
    """
    # 只对包含"模拟 API 调用失败"的异常进行重试
    if "模拟 API 调用失败" in str(exception):
        print(f"捕获到可重试异常: {exception}")
        return True

    # 对其他异常不重试
    print(f"捕获到不可重试异常: {exception}")
    return False

# 模拟抛出 ValueError 的节点
def value_error_call(state: State) -> Dict[str, Any]:
    """
    模拟抛出 ValueError 的节点（不会被默认重试策略重试）
    """
    print("调用会抛出 ValueError 的节点")
    raise ValueError("模拟 ValueError 异常")

def run_demo():
    print("=== LangGraph 节点重试策略演示 ===\n")

    # 重置全局计数器
    global attempt_counter
    attempt_counter = 0
```

```
# 演示 1: 使用默认重试策略
print("1. 使用默认重试策略:")
print("    默认策略会对除特定异常外的所有异常进行重试")
print("    不会重试的异常包括: ValueError, TypeError, ArithmeticError,
ImportError,")
print("                                LookupError, NameError, SyntaxError,
RuntimeError,")
print("                                ReferenceError, StopIteration,
StopAsyncIteration, OSError\n")

builder1 = StateGraph(State)

# 添加节点, 使用默认重试策略
builder1.add_node(
    "unstable_call",
    unstable_api_call,
    retry_policy=RetryPolicy(max_attempts=5) # 允许最多 5 次尝试
)

builder1.add_edge(START, "unstable_call")
builder1.add_edge("unstable_call", END)

graph1 = builder1.compile()

print("测试默认重试策略:")
try:
    result = graph1.invoke({"result": ""})
    print(f"最终结果: {result}\n")
except Exception as e:
    print(f"最终失败: {type(e).__name__}: {e}\n")

# 演示 2: 使用自定义重试策略
print("2. 使用自定义重试策略:")
print("    自定义策略只对特定错误进行重试\n")

# 重置全局计数器
attempt_counter = 0

builder2 = StateGraph(State)

# 添加节点, 使用自定义重试策略
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
builder2.add_node(  
    "custom_retry_call",  
    unstable_api_call,  
    retry_policy=RetryPolicy(max_attempts=5,  
retry_on=custom_retry_on)  
)  
  
builder2.add_edge(START, "custom_retry_call")  
builder2.add_edge("custom_retry_call", END)  
  
graph2 = builder2.compile()  
  
print("测试自定义重试策略:")  
try:  
    result = graph2.invoke({"result": ""})  
    print(f"最终结果: {result}\n")  
except Exception as e:  
    print(f"最终失败: {type(e).__name__}: {e}\n")  
  
# 演示 3: 不会重试的异常类型  
print("3. 测试不会重试的异常类型:")  
  
builder3 = StateGraph(State)  
  
# 添加节点, 使用默认重试策略  
builder3.add_node(  
    "value_error_call",  
    value_error_call,  
    retry_policy=RetryPolicy(max_attempts=3)  
)  
  
builder3.add_edge(START, "value_error_call")  
builder3.add_edge("value_error_call", END)  
  
graph3 = builder3.compile()  
  
print("测试 ValueError (默认策略不会重试):")  
try:  
    result = graph3.invoke({"result": ""})  
    print(f"最终结果: {result}\n")  
except Exception as e:
```

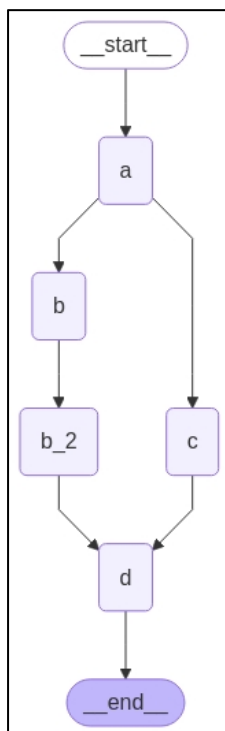
更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
print(f"最终失败: {type(e).__name__}: {e}\n")

if __name__ == "__main__":
    run_demo()
```

3.2.5 延迟节点执行

延迟节点执行就是将某个节点的执行推迟到所有其他待处理任务完成后,这在分支长度不同的情况下尤其适用。



```
"""
```

LangGraph 延迟节点执行演示

本示例展示了如何使用 `defer=True` 来实现节点延迟执行,确保该节点等待所有其他并行分支任务完成后才执行。

```
"""
```

```
import operator
from typing import Annotated, Any
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```



```
class State(TypedDict):
    """
    状态类型定义

    aggregate: 使用 operator.add reducer 使这个列表为追加模式，确保每个节点的结果都能被正确合并
    """
    # The operator.add reducer fn makes this append-only
    aggregate: Annotated[list, operator.add]

def a(state: State):
    """
    节点 a: 启动分支

    此节点是工作流的起点，负责初始化流程并分发到不同的分支。

    Args:
        state: 当前状态

    Returns:
        dict: 包含新结果的状态更新
    """
    print(f'Adding "A" to {state["aggregate"]}')
    return {"aggregate": ["A"]}

def b(state: State):
    """
    节点 b: 第一个分支

    此节点处理第一个分支的任务，与节点 c 并行执行。

    Args:
        state: 当前状态

    Returns:
        dict: 包含新结果的状态更新
    """
    print(f'Adding "B" to {state["aggregate"]}')
    return {"aggregate": ["B"]}

def b_2(state: State):
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
"""
节点 b2: 第二个分支

此节点处理第二个分支的任务，在节点 b 完成后执行。

Args:
    state: 当前状态

Returns:
    dict: 包含新结果的状态更新
"""
print(f'Adding "B_2" to {state["aggregate"]}')
return {"aggregate": ["B_2"]}

def c(state: State):
    """
    节点 c: 另一个分支

    此节点处理另一个分支的任务，与节点 b 并行执行。

    Args:
        state: 当前状态

    Returns:
        dict: 包含新结果的状态更新
    """
    print(f'Adding "C" to {state["aggregate"]}')
    return {"aggregate": ["C"]}

def d(state: State):
    """
    节点 d: 延迟执行的汇总节点

    此节点设置了 defer=True，因此会等待所有其他任务完成后才执行。
    它负责汇总所有分支的结果。

    Args:
        state: 当前状态

    Returns:
        dict: 包含新结果的状态更新
    """
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
"""
print(f'Adding "D" to {state["aggregate"]}')
return {"aggregate": ["D"]}

# 创建图
builder = StateGraph(State)

# 添加节点
builder.add_node("a", a)
builder.add_node("b", b)
builder.add_node("b_2", b_2)
builder.add_node("c", c)
builder.add_node("d", d, defer=True) # 设置 defer=True 延迟执行

# 添加边
builder.add_edge(START, "a")
builder.add_edge("a", "b")
builder.add_edge("a", "c")
builder.add_edge("b", "b_2")
builder.add_edge("b_2", "d")
builder.add_edge("c", "d")
builder.add_edge("d", END)

# 编译图
graph = builder.compile()

# 执行图
print("=== 开始执行工作流 ===")
result = graph.invoke({})
print("=== 执行结果 ===")
print(result)
```

3.3 Edges

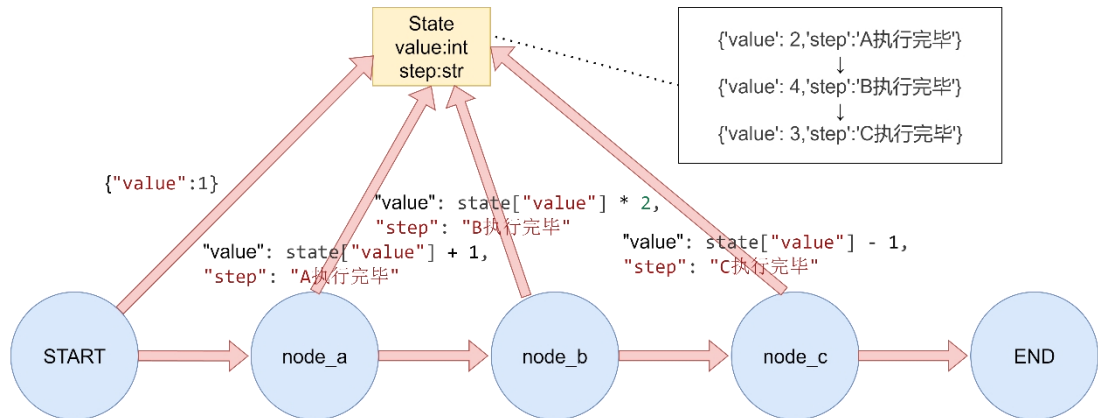
边定义了逻辑的路由方式以及图如何决定停止。这是智能体工作方式以及不同节点之间通信方式的重要组成部分。边有几种关键类型：

- Normal Edges: 普通边。直接从一个节点连接到下一个节点。
- Conditional Edges: 条件边。调用函数以确定接下来要前往哪个（哪些）节点。
- Entry Point: 入口点。用户输入到达时首先调用哪个节点。

- **Conditional Entry Point:** 条件入口点。调用一个函数来确定当用户输入到达时，首先调用哪个（些）节点。

一个节点可以有多个出边。如果一个节点有多个出边，那么所有这些目标节点都将作为下一个超级步骤的一部分并行执行。

3.3.1 Normal Edges



```

"""
LangGraph 普通边演示

普通边是直接连接两个节点的边，表示无条件地从一個节点跳转到另一个节点。
"""

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 定义状态
class GraphState(TypedDict):
    value: int
    step: str

# 定义节点函数
def node_a(state: GraphState) -> dict:
    """节点 A"""
    print("执行节点 A")
    return {"value": state["value"] + 1, "step": "A 执行完毕"}

def node_b(state: GraphState) -> dict:
    """节点 B"""
    print("执行节点 B")
    return {"value": state["value"] * 2, "step": "B 执行完毕"}

def node_c(state: GraphState) -> dict:
    """节点 C"""
    print("执行节点 C")
    return {"value": state["value"] - 1, "step": "C 执行完毕"}

# 构建图
graph = StateGraph(GraphState)
graph.add_node(node_a)
graph.add_node(node_b)
graph.add_node(node_c)
graph.add_edge(START, node_a)
graph.add_edge(node_a, node_b)
graph.add_edge(node_b, node_c)
graph.add_edge(node_c, END)
graph.set_entry_point(START)
graph.compile()

# 运行图
state = GraphState(value=1, step='')
graph.invoke(state)
  
```

```
    """节点 B"""
    print("执行节点 B")
    return {"value": state["value"] * 2, "step": "B 执行完毕"}

def node_c(state: GraphState) -> dict:
    """节点 C"""
    print("执行节点 C")
    return {"value": state["value"] - 1, "step": "C 执行完毕"}

def main():
    """演示普通边"""
    print("=== 普通边演示 ===")

    # 创建图
    builder = StateGraph(GraphState)

    # 添加节点
    builder.add_node("node_a", node_a)
    builder.add_node("node_b", node_b)
    builder.add_node("node_c", node_c)

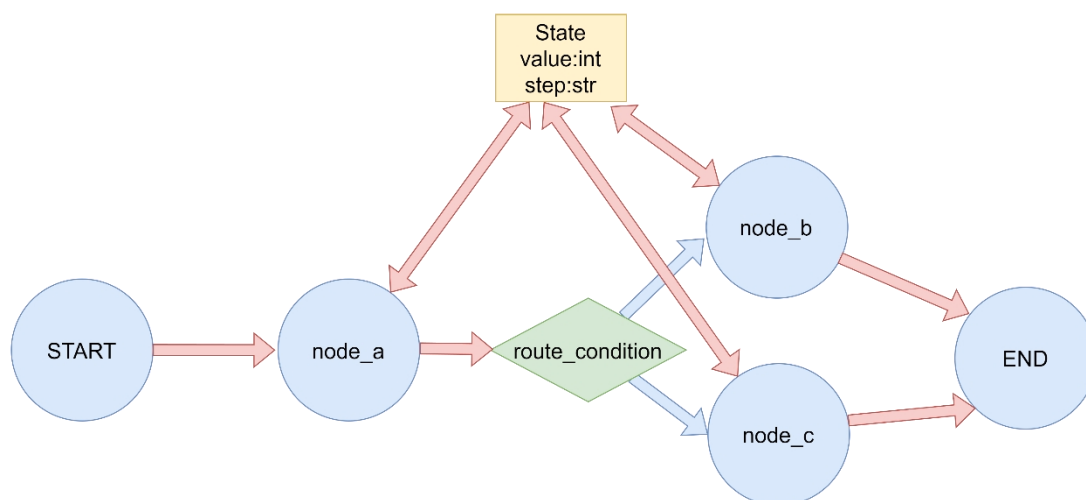
    # 添加普通边
    builder.add_edge(START, "node_a") # 从开始到 A
    builder.add_edge("node_a", "node_b") # 从 A 到 B
    builder.add_edge("node_b", "node_c") # 从 B 到 C
    builder.add_edge("node_c", END) # 从 C 到结束

    # 编译图
    graph = builder.compile()

    # 执行图
    result = graph.invoke({"value": 1})
    print(f"执行结果: {result}\n")

if __name__ == "__main__":
    main()
```

3.3.2 Conditional Edges



"""

LangGraph 条件边演示

条件边根据当前状态动态决定下一个要执行的节点。

"""

```

from typing import Literal
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 定义状态
class GraphState(TypedDict):
    value: int
    step: str

# 定义节点函数
def node_a(state: GraphState) -> dict:
    """节点 A"""
    print("执行节点 A")
    return {"value": state["value"] + 1, "step": "A 执行完毕"}

def node_b(state: GraphState) -> dict:
    """节点 B"""
    print("执行节点 B")
    return {"value": state["value"] * 2, "step": "B 执行完毕"}
    
```

```
def node_c(state: GraphState) -> dict:
    """节点 C"""
    print("执行节点 C")
    return {"value": state["value"] - 1, "step": "C 执行完毕"}

# 条件边的路由函数
def route_condition(state: GraphState) -> Literal["node_b", "node_c"]:
    """根据 value 值决定路由到哪个节点"""
    if state["value"] % 2 == 0:
        return "node_b_alias" # 偶数路由到节点 B
    else:
        return "node_c_alias" # 奇数路由到节点 C

def main():
    """演示条件边"""
    print("=== 条件边演示 ===")

    # 创建图
    builder = StateGraph(GraphState)

    # 添加节点
    builder.add_node("node_a", node_a)
    builder.add_node("node_b", node_b)
    builder.add_node("node_c", node_c)

    # 添加边
    builder.add_edge(START, "node_a") # 入口点

    # 添加条件边
    builder.add_conditional_edges(
        "node_a", # 源节点
        route_condition, # 路由函数
        { # 路由映射
            "node_b_alias": "node_b",
            "node_c_alias": "node_c"
        }
    )

    # 从 B 和 C 到结束
```

```

builder.add_edge("node_b", END)
builder.add_edge("node_c", END)

# 编译图
graph = builder.compile()

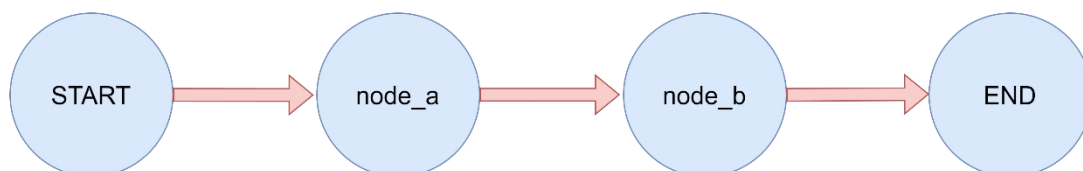
# 执行图 - 偶数情况
print("输入值为偶数:")
result = graph.invoke({"value": 2})
print(f"执行结果: {result}")

# 执行图 - 奇数情况
print("\n输入值为奇数:")
result = graph.invoke({"value": 1})
print(f"执行结果: {result}\n")

if __name__ == "__main__":
    main()

```

3.3.3 Entry Point



```

"""
LangGraph 入口点演示

入口点定义了图开始执行的第一个节点。
"""

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 定义状态
class GraphState(TypedDict):
    value: int
    step: str

```



```
# 定义节点函数
def node_a(state: GraphState) -> dict:
    """节点 A"""
    print("执行节点 A")
    return {"value": state["value"] + 1, "step": "A 执行完毕"}

def node_b(state: GraphState) -> dict:
    """节点 B"""
    print("执行节点 B")
    return {"value": state["value"] * 2, "step": "B 执行完毕"}

def main():
    """演示入口点"""
    print("=== 入口点演示 ===")

    # 创建图
    builder = StateGraph(GraphState)

    # 添加节点
    builder.add_node("node_a", node_a)
    builder.add_node("node_b", node_b)

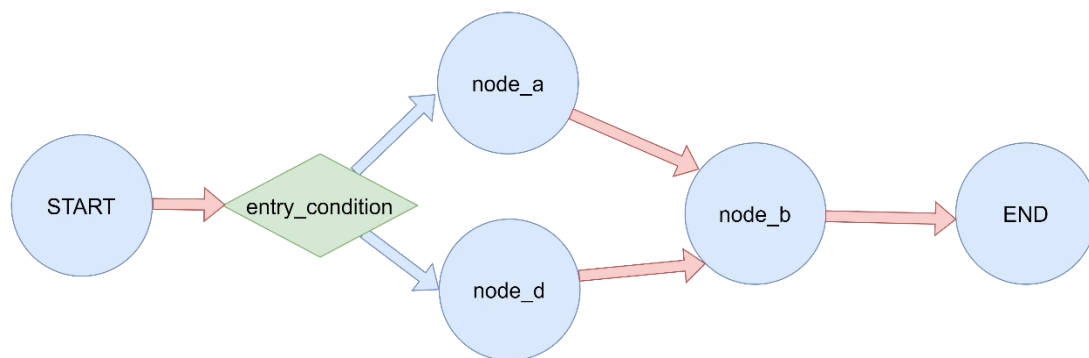
    # 设置入口点和出口点
    builder.add_edge(START, "node_a")
    builder.add_edge("node_a", "node_b")
    builder.add_edge("node_b", END)

    # 编译图
    graph = builder.compile()

    # 执行图
    result = graph.invoke({"value": 0})
    print(f"执行结果: {result}\n")

if __name__ == "__main__":
    main()
```

3.3.4 Conditional Entry Point



```

"""
LangGraph 条件入口点演示

条件入口点允许根据输入状态动态决定从哪个节点开始执行。
"""

from typing import Literal
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 定义状态
class GraphState(TypedDict):
    value: int
    step: str

# 定义节点函数
def node_a(state: GraphState) -> dict:
    """节点 A"""
    print("执行节点 A")
    return {"value": state["value"] + 1, "step": "A 执行完毕"}

def node_b(state: GraphState) -> dict:
    """节点 B"""
    print("执行节点 B")
    return {"value": state["value"] * 2, "step": "B 执行完毕"}

def node_d(state: GraphState) -> dict:
    """节点 D"""
    print("执行节点 D")
  
```

```
    return {"value": state["value"] + 10, "step": "D 执行完毕"}

# 条件入口点的路由函数
def entry_condition(state: GraphState) -> Literal["node_a", "node_d"]:
    """根据输入值决定从哪个节点开始"""
    if state.get("value", 0) > 5:
        return "node_d" # 大于 5 从节点 D 开始
    else:
        return "node_a" # 否则从节点 A 开始

def main():
    """演示条件入口点"""
    print("=== 条件入口点演示 ===")

    # 创建图
    builder = StateGraph(GraphState)

    # 添加节点
    builder.add_node("node_a", node_a)
    builder.add_node("node_d", node_d)
    builder.add_node("node_b", node_b)

    # 添加条件入口点
    builder.add_conditional_edges(
        START, # 起始点
        entry_condition, # 路由函数
        { # 路由映射
            "node_a": "node_a",
            "node_d": "node_d"
        }
    )

    # 添加普通边
    builder.add_edge("node_a", "node_b")
    builder.add_edge("node_d", "node_b")
    builder.add_edge("node_b", END)

    # 编译图
    graph = builder.compile()
```

```
# 执行图 - 小于等于 5 的情况
print("输入值小于等于 5:")
result = graph.invoke({"value": 3})
print(f"执行结果: {result}")

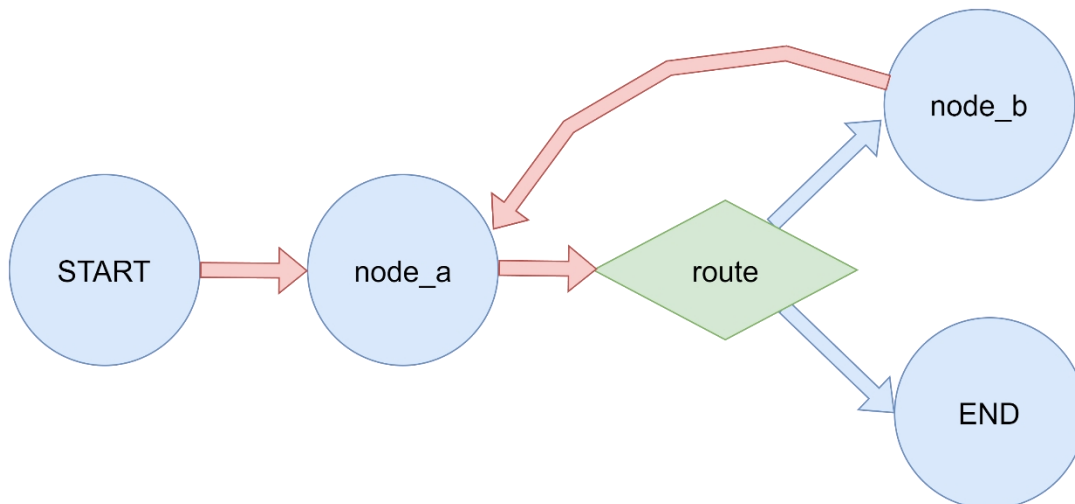
# 执行图 - 大于 5 的情况
print("\n输入值大于 5:")
result = graph.invoke({"value": 10})
print(f"执行结果: {result}\n")

if __name__ == "__main__":
    main()
```

3.3.5 创建和控制循环

在创建带有循环的图时，需要一种终止执行的机制。最常见的做法是添加一条条件边，当达到某个终止条件时，该边会路由到 END 节点。

递归限制设定了图在抛出错误之前允许执行的超级步骤数量，默认值 25，在 `graph.invoke` 的 `config` 参数中指定。在某些应用中，我们无法保证会达到给定的终止条件。在这种情况下，我们可以设置图的递归限制。这将在经过指定数量的超级步骤后引发 `GraphRecursionError`。然后我们可以捕获并处理这个异常。



```
from typing import Annotated, Dict, Literal
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.errors import GraphRecursionError
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
class LoopState(TypedDict):
    count: int
    result: str
    max_count: int

def node_a(state: LoopState) -> dict:
    """节点 a: 处理逻辑并更新计数"""
    print(f"执行节点 a, 当前计数: {state['count']}")
    return {
        'count': state['count'] + 1,
        'result': f"已处理{state['count']}次"
    }

def node_b(state: LoopState) -> dict:
    """节点 b: 辅助处理"""
    print(f"执行节点 b, 当前计数: {state['count']}")
    return {
        'result': f"已处理{state['count']}次 - 辅助处理"
    }

def route(state: LoopState) -> Literal["b", END]:
    """条件路由函数: 决定是继续循环还是终止"""
    # 终止条件: 当计数达到最大值时终止
    if state['count'] >= state['max_count']:
        print(f"满足终止条件, 计数 {state['count']} >= {state['max_count']},
返回 END")
        return END
    else:
        print(f"未满足终止条件, 计数 {state['count']} < {state['max_count']},
返回 b")
        return "b"

# 创建图
builder = StateGraph(LoopState)

# 添加节点
builder.add_node("a", node_a)
builder.add_node("b", node_b)

# 添加边
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
builder.add_edge(START, "a")
builder.add_conditional_edges("a", route)
builder.add_edge("b", "a")

# 编译图
graph = builder.compile()

# 执行图
print("=== 开始执行工作流 ===")
try:
    result = graph.invoke(input={
        'count': 0,
        'result': '',
        'max_count': 3
    }, config={
        'recursion_limit': 6 # 设置递归限制
    })
    print("=== 执行结果 ===")
    print(result)
except GraphRecursionError as e:
    print(f"递归错误: {e}")
```

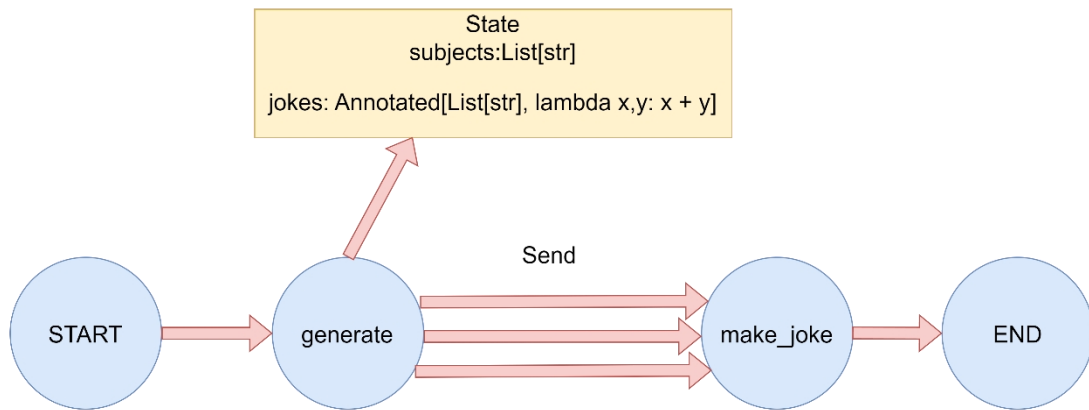
3.4 Send

在传统的图结构中，节点和边都是预先定义好的，但在某些场景下，我们需要动态地根据运行时状态来决定执行哪些节点。Map-Reduce 模式就是这样一个典型场景：

- 一个节点生成一个动态数量的对象列表。
- 另一个节点需要对列表中的每个对象进行处理。
- 最终将所有处理结果合并。

为了支持这种设计模式，LangGraph 支持从条件边返回 Send 对象。Send 接受两个参数：

- 第一个是节点的名称。
- 第二个是要传递给该节点的状态。



"""

LangGraph Map-Reduce 模式演示

演示如何使用 **Send** 对象实现 **map-reduce** 设计模式。

在这种模式中，第一个节点生成一个对象列表，

然后将其他节点应用于所有这些对象。

"""

```

from typing import Annotated, List, Sequence
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send
    
```

定义状态

```

class OverallState(TypedDict):
    subjects: List[str]
    jokes: Annotated[List[str], lambda x, y: x + y] # 使用列表合并的方式
    
```

第一个节点：生成需要处理的主体列表

```

def generate_subjects(state: OverallState) -> dict:
    """生成需要处理的主体列表"""
    print("执行节点: generate_subjects")
    subjects = ["猫", "狗", "程序员"]
    print(f"生成主体列表: {subjects}")
    return {"subjects": subjects}
    
```

Map 节点：为每个主题生成笑话

```

def make_joke(state: OverallState) -> dict:
    
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
"""为单个主题生成笑话"""
subject = state.get("subject", "未知")
print(f"执行节点: make_joke, 处理主题: {subject}")

# 根据主题生成相应笑话
jokes_map = {
    "猫": "为什么猫不喜欢在线购物? 因为它们更喜欢实体店!",
    "狗": "为什么狗不喜欢计算机? 因为它们害怕被鼠标咬!",
    "程序员": "为什么程序员喜欢洗衣服? 因为他们在寻找 bugs!",
    "未知": "这是一个关于未知主题的神秘笑话。"
}

joke = jokes_map.get(subject, f"这是一个关于{subject}的即兴笑话。")
print(f"生成笑话: {joke}")
return {"jokes": [joke]}

# 条件边函数: 根据主题列表生成 Send 对象列表
def map_subjects_to_jokes(state: OverallState) -> Sequence[Send]:
    """将主题列表映射到 joke 生成任务"""
    print("执行条件边函数: map_subjects_to_jokes")
    subjects = state["subjects"]
    print(f"映射主题到 joke 任务: {subjects}")

    # 为每个主题创建一个 Send 对象, 指向 make_joke 节点
    # 每个 Send 对象包含节点名称和传递给该节点的状态
    send_list = [Send("make_joke", {"subject": subject}) for subject in
subjects]
    print(f"生成 Send 对象列表: {send_list}")
    return send_list

def main():
    """演示 Map-Reduce 模式"""
    print("=== Map-Reduce 模式演示 ===\n")

    # 创建图
    builder = StateGraph(OverallState)

    # 添加节点
    builder.add_node("generate_subjects", generate_subjects)
```



```
builder.add_node("make_joke", make_joke)

# 添加边
builder.add_edge(START, "generate_subjects")

# 添加条件边, 使用 Send 对象实现 map-reduce
builder.add_conditional_edges(
    "generate_subjects", # 源节点
    map_subjects_to_jokes # 路由函数, 返回 Send 对象列表
)

# 从 make_joke 到结束
builder.add_edge("make_joke", END)

# 编译图
graph = builder.compile()

# 执行图
initial_state = {"subjects": [], "jokes": []}
print("初始状态:", initial_state)
print("\n开始执行图...")

result = graph.invoke(initial_state)
print(f"\n最终结果: {result}")

print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

从运行输出可以看到:

- (1) 首先执行 `generate_subjects` 节点, 生成主题列表: ['猫', '狗', '程序员']
- (2) 然后通过条件边函数 `map_subjects_to_jokes` 为每个主题创建一个 `Send` 对象
- (3) `make_joke` 节点被并行执行 3 次, 每次处理一个主题
- (4) 最终将所有生成的笑话合并到一个列表中

这种模式非常适合处理动态数量的任务, 比如:

- (1) 批量处理用户请求
- (2) 并行处理文档集合

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

(3) 分布式任务调度

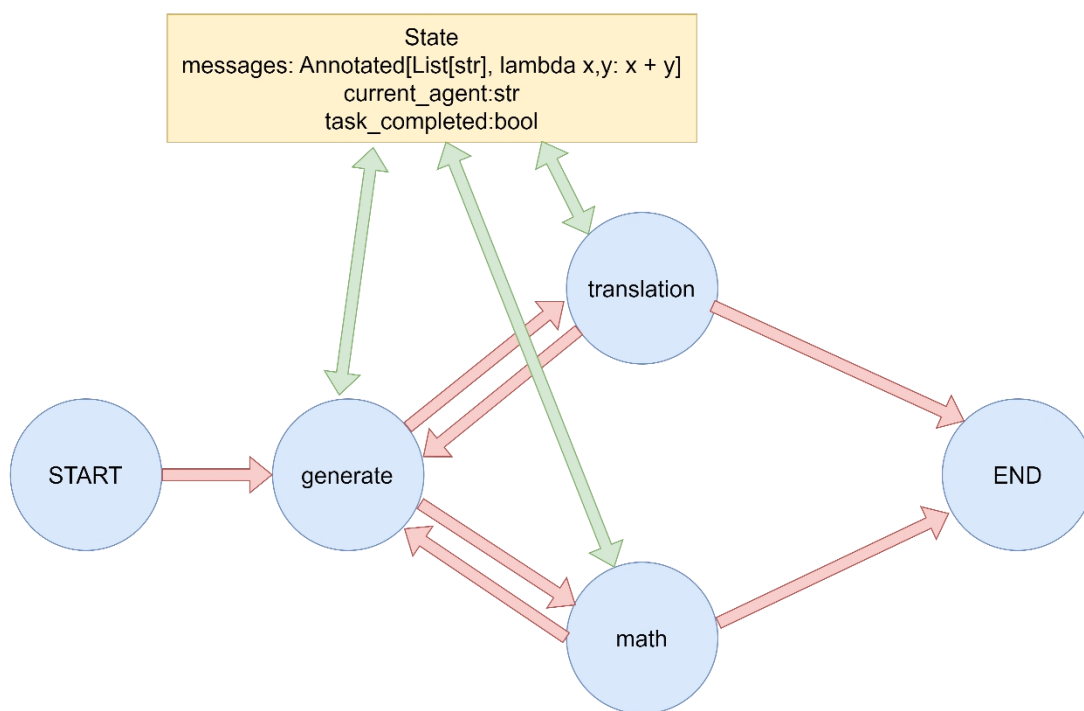
通过使用 `Send` 对象, `LangGraph` 提供了一种优雅的方式来实现这种动态图结构, 使得我们可以根据运行时状态来决定执行路径。

3.5 Command

将控制流(边)和状态更新(节点)结合起来可能会很有用。例如, 你可能希望在同一个节点中既执行状态更新, 又决定下一步前往哪个节点。`LangGraph` 提供了一种实现方式, 即从节点函数返回一个 `Command` 对象。

3.5.1 Command 基本用法

借助 `Command`, 可以实现动态控制流行为(与条件边相同): 根据消息内容动态决定执行路径, 并在节点中同时更新状态和控制流程。



```
"""
```

LangGraph Command 基础演示

演示如何在节点中使用 `Command` 对象同时更新状态和控制流程。

```
"""
```

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
from langgraph.types import Command

# 定义状态
class AgentState(TypedDict):
    messages: Annotated[list, lambda x, y: x + y]
    current_agent: str
    task_completed: bool

# 节点函数：决策代理
def decision_agent(state: AgentState) -> Command[AgentState]:
    """决策代理节点，根据消息内容决定下一步操作"""
    print("执行节点：decision_agent")

    # 检查最新的消息
    last_message = state["messages"][-1] if state["messages"] else ""
    print(f"最新消息：{last_message}")

    # 根据消息内容决定下一步
    if "数学" in last_message:
        # 更新状态并跳转到数学代理
        return Command(
            update={
                "messages": [("system", "路由到数学代理")],
                "current_agent": "math_agent"
            },
            goto="math_agent"
        )
    elif "翻译" in last_message:
        # 更新状态并跳转到翻译代理
        return Command(
            update={
                "messages": [("system", "路由到翻译代理")],
                "current_agent": "translation_agent"
            },
            goto="translation_agent"
        )
    else:
        # 任务完成
        return Command(
```

```
        update={
            "messages": [("system", "任务完成")],
            "task_completed": True
        },
        goto=END
    )

# 节点函数：数学代理
def math_agent(state: AgentState) -> Command[AgentState]:
    """数学代理节点"""
    print("执行节点：math_agent")

    # 执行数学计算任务
    result = "2 + 2 = 4"
    print(f"计算结果：{result}")

    # 更新状态并返回决策代理
    return Command(
        update={
            "messages": [("assistant", f"数学计算结果：{result}")],
            "current_agent": "decision_agent"
        },
        goto="decision_agent"
    )

# 节点函数：翻译代理
def translation_agent(state: AgentState) -> Command[AgentState]:
    """翻译代理节点"""
    print("执行节点：translation_agent")

    # 执行翻译任务
    translation = "Hello -> 你好"
    print(f"翻译结果：{translation}")

    # 更新状态并返回决策代理
    return Command(
        update={
            "messages": [("assistant", f"翻译结果：{translation}")],
            "current_agent": "decision_agent"
```

```
    },
    goto="decision_agent"
)

def main():
    """演示 Command 基础用法"""
    print("=== Command 基础演示 ===\n")

    # 创建图
    builder = StateGraph(AgentState)

    # 添加节点
    builder.add_node("decision_agent", decision_agent)
    builder.add_node("math_agent", math_agent)
    builder.add_node("translation_agent", translation_agent)

    # 设置入口点
    builder.add_edge(START, "decision_agent")

    # 编译图
    graph = builder.compile()

    # 执行图 - 测试数学任务
    print("测试 1: 数学任务")
    initial_state = {
        "messages": [("user", "我需要计算数学题")],
        "current_agent": "user",
        "task_completed": False
    }
    print("初始状态:", initial_state)
    result = graph.invoke(initial_state)
    print("最终状态:", result)
    print("\n" + "="*50 + "\n")

    # 执行图 - 测试翻译任务
    print("测试 2: 翻译任务")
    initial_state = {
        "messages": [("user", "我需要翻译文本")],
        "current_agent": "user",
        "task_completed": False
    }
```

```
}
print("初始状态:", initial_state)
result = graph.invoke(initial_state)
print("最终状态:", result)
print("\n" + "="*50 + "\n")

# 执行图 - 测试完成任务
print("测试 3: 完成任务")
initial_state = {
    "messages": [("user", "你好")],
    "current_agent": "user",
    "task_completed": False
}
print("初始状态:", initial_state)
result = graph.invoke(initial_state)
print("最终状态:", result)

if __name__ == "__main__":
    main()
```

案例中使用 Command 对象同时更新状态和控制流程。主要特点包括：

- 决策代理节点根据消息内容决定下一步操作。
- 数学代理和翻译代理分别处理特定任务。
- 每个节点都使用 Command 对象更新状态并指定下一步要执行的节点。

3.5.2 Command vs 条件边

应该在什么时候使用 Command 而不是条件边？

- 使用 Command：当你需要在更新状态的同时决定下一步执行哪个节点时。例如，在多智能体系统中，需要在切换智能体的同时传递信息。
- 使用条件边：当你只需要在节点之间有条件地路由而不需要更新状态时。

3.5.3 父图导航

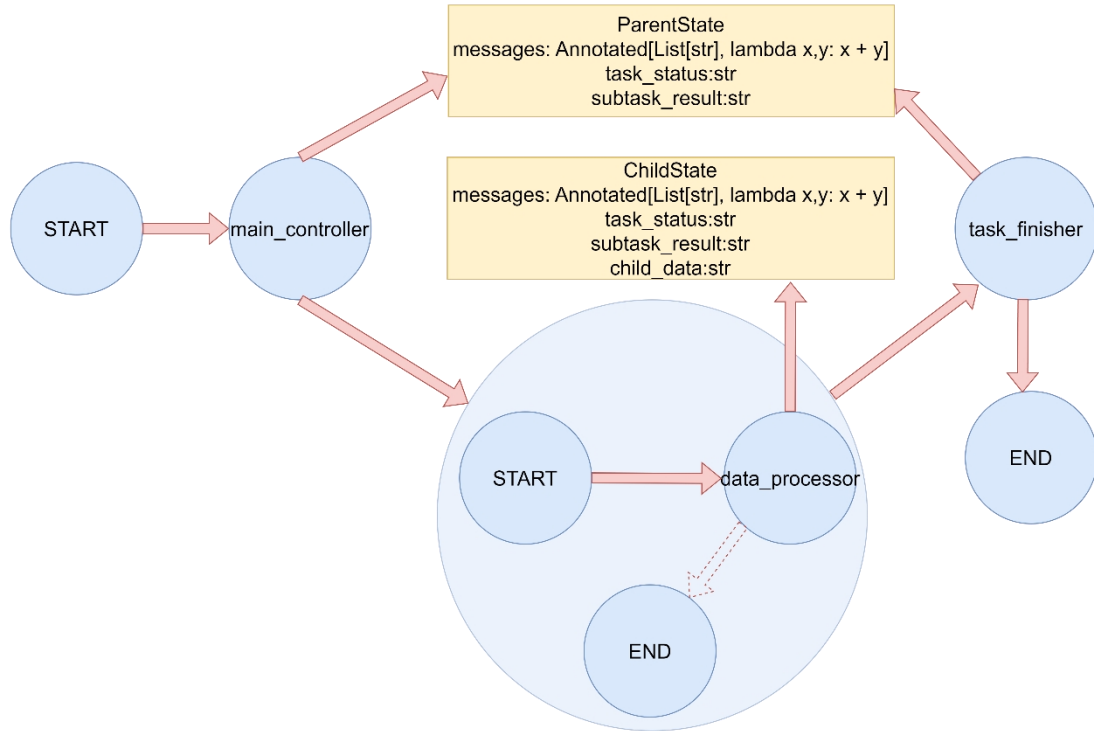
使用子图时，可能希望从子图内的某个节点导航到另一个子图（即父图中的另一个节点）。这在实现多智能体交接时特别有用。

要实现这一点，可以在 Command 中指定 `graph=Command.PARENT`：

- 将 graph 设置为 Command.PARENT 将导航到最近的父图。

- 当从子图节点向父图节点发送更新，且更新的键是父图和子图的共享的状态模式时，必须为父图状态中要更新的键定义一个 reducer。

子图节点如何通过 Command 对象导航回父图，并更新父图的状态：



```

"""
LangGraph Command 父图导航演示

演示如何使用 Command 对象从子图导航到父图节点。
"""

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

# 定义父图状态
class ParentState(TypedDict):
    messages: Annotated[list, lambda x, y: x + y]
    task_status: str
    subtask_result: str

# 定义子图状态（继承父图状态）

```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
class ChildState(TypedDict):
    messages: Annotated[list, lambda x, y: x + y]
    task_status: str
    subtask_result: str
    child_data: str

# 父图节点：主控制器
def main_controller(state: ParentState) -> Command[ParentState]:
    """主控制器节点"""
    print("执行节点：main_controller (父图)")

    # 启动子任务
    return Command(
        update={
            "messages": [("system", "启动子任务")],
            "task_status": "subtask_started"
        },
        goto="subgraph_node"
    )

# 父图节点：任务结束
def task_finisher(state: ParentState) -> dict:
    """任务结束节点"""
    print("执行节点：task_finisher (父图)")

    return {
        "messages": [("system", "任务完成")],
        "task_status": "completed"
    }

# 子图节点：数据处理
def data_processor(state: ChildState) -> Command[ParentState]:
    """数据处理节点（在子图中）"""
    print("执行节点：data_processor (子图)")

    # 处理数据
    processed_data = "处理后的数据"
    print(f"处理结果：{processed_data}")
```



```
# 导航回父图的 task_finisher 节点
return Command(
    update={
        "messages": [("subtask", f"子任务完成: {processed_data}")],
        "subtask_result": processed_data,
        "task_status": "subtask_completed"
    },
    goto="task_finisher",
    graph=Command.PARENT # 指定导航到父图
)

def create_subgraph() -> StateGraph:
    """创建子图"""
    subgraph_builder = StateGraph(ChildState)
    subgraph_builder.add_node("data_processor", data_processor)
    subgraph_builder.add_edge(START, "data_processor")
    subgraph_builder.add_edge("data_processor", END)
    return subgraph_builder.compile() # 编译子图

def main():
    """演示 Command 父图导航"""
    print("=== Command 父图导航演示 ===\n")

    # 创建父图
    parent_builder = StateGraph(ParentState)

    # 添加节点
    parent_builder.add_node("main_controller", main_controller)
    parent_builder.add_node("task_finisher", task_finisher)
    parent_builder.add_node("subgraph_node", create_subgraph()) # 添加子图作为节点

    # 添加边
    parent_builder.add_edge(START, "main_controller")
    parent_builder.add_edge("main_controller", "subgraph_node")

    # 编译图
    graph = parent_builder.compile()
```

```
# 执行图
initial_state = {
    "messages": [("user", "开始任务")],
    "task_status": "init",
    "subtask_result": ""
}
print("初始状态:", initial_state)

result = graph.invoke(initial_state)
print("最终状态:", result)

if __name__ == "__main__":
    main()
```

案例中使用 Command 对象从子图导航到父图节点。主要特点包括：

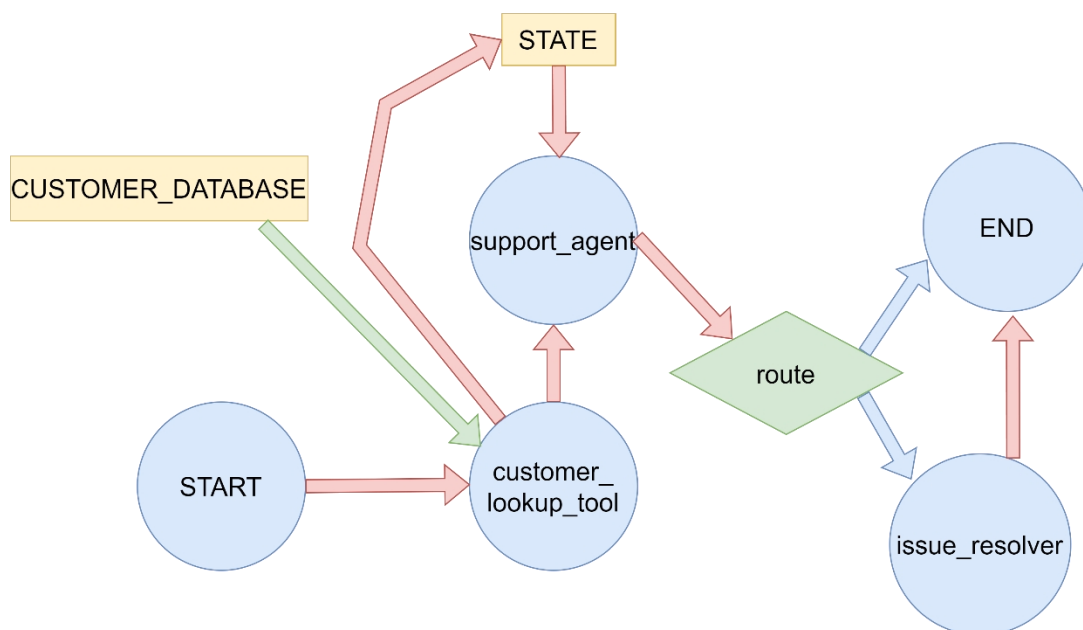
- 创建了一个包含主控制器和任务结束节点的父图。
- 创建了一个包含数据处理器节点的子图。
- 数据处理器节点使用 `graph=Command.PARENT` 参数导航回父图的特定节点。
- 展示了父子图之间的状态传递和控制流转移。

3.5.4 案例：工具中更新图状态

在客户支持应用程序中，你可能希望在对话开始时根据客户的账号或 ID 查询客户信息。

以客户支持应用程序为例，演示了如何在工具内部使用 Command 对象更新图状态。主要特点包括：

- 场景设定：客户支持应用，根据客户 ID 查询客户信息。
- 工具函数：`lookup_customer_info` 模拟查询客户信息的工具。
- 状态管理：使用 Command 对象在工具执行后更新客户信息和消息历史。
- 流程控制：根据查询结果决定下一步执行的节点。



"""

LangGraph Command 工具内部状态更新演示

演示如何在工具内部使用 **Command** 对象更新图状态。

以客户支持应用程序为例，在对话开始时根据客户 ID 查询客户信息。

"""

```

import time
import random
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

```

定义状态

```

class SupportState(TypedDict):
    customer_id: str
    customer_info: dict
    messages: Annotated[list, lambda x, y: x + y]
    issue_resolved: bool

```

模拟客户数据库

```

CUSTOMER_DATABASE = {
    "CUST001": {

```

```
        "name": "张三",
        "email": "zhangsan@example.com",
        "membership_level": "金牌会员",
        "account_status": "正常"
    },
    "CUST002": {
        "name": "李四",
        "email": "lisi@example.com",
        "membership_level": "银牌会员",
        "account_status": "正常"
    },
    "CUST003": {
        "name": "王五",
        "email": "wangwu@example.com",
        "membership_level": "普通会员",
        "account_status": "欠费"
    }
}

# 工具函数：查询客户信息
def lookup_customer_info(customer_id: str) -> dict:
    """
    模拟查询客户信息的工具函数
    在实际应用中，这可能是一个 API 调用或数据库查询
    """
    print(f"正在查询客户 ID: {customer_id} 的信息...")

    # 模拟网络延迟
    time.sleep(1)

    # 从数据库获取客户信息
    customer_info = CUSTOMER_DATABASE.get(customer_id, {})

    if customer_info:
        print(f"找到客户信息: {customer_info}")
    else:
        print(f"未找到客户 ID: {customer_id} 的信息")
        customer_info = {"error": "客户未找到"}

    return customer_info
```

```
# 节点函数：客户信息查询工具
def customer_lookup_tool(state: SupportState) -> Command[SupportState]:
    """客户信息查询工具节点"""
    print("执行节点：customer_lookup_tool")

    customer_id = state["customer_id"]

    # 使用工具查询客户信息
    customer_info = lookup_customer_info(customer_id)

    # 使用 Command 更新状态并决定下一步
    return Command(
        update={
            "customer_info": customer_info,
            "messages": [("system", f"已查询客户 {customer_id} 的信息")]
        },
        goto="support_agent" # 查询完成后转到客服代理节点
    )

# 节点函数：客服代理
def support_agent(state: SupportState) -> Command[SupportState]:
    """客服代理节点"""
    print("执行节点：support_agent")

    customer_info = state["customer_info"]
    messages = state["messages"]

    # 检查是否找到客户信息
    if "error" in customer_info:
        response = "抱歉，我们无法找到您的客户信息，请您确认提供的客户 ID 是否正确。"
        next_node = END
    else:
        # 根据客户等级提供个性化服务
        membership_level = customer_info.get("membership_level", "未知")
        name = customer_info.get("name", "客户")

        if membership_level == "金牌会员":
```

```
        response = f"尊敬的金牌会员{name}，您好！我们非常重视您的问题，将  
优先为您处理。"  
        elif membership_level == "银牌会员":  
            response = f"{name}您好！我们会尽快为您解决问题。"  
        else:  
            response = f"{name}您好！感谢您的咨询。"  
  
        # 模拟处理问题  
        response += "\n\n 我们已经收到您的问题，正在为您处理..."  
        next_node = "issue_resolver"  
  
    return Command(  
        update={  
            "messages": [("assistant", response)]  
        },  
        goto=next_node  
    )
```

节点函数：问题解决器

```
def issue_resolver(state: SupportState) -> Command[SupportState]:  
    """问题解决器节点"""  
    print("执行节点：issue_resolver")  
  
    # 模拟问题解决过程  
    print("正在分析和解决客户问题...")  
    time.sleep(1)  
  
    # 随机决定问题是否解决（模拟）  
    resolved = random.choice([True, False])  
  
    if resolved:  
        response = "您的问题已成功解决！如果还有其他需要帮助的地方，请随时告诉  
我们。"  
        issue_status = True  
    else:  
        response = "我们正在进一步处理您的问题，稍后会有专员与您联系。"  
        issue_status = False  
  
    return Command(  
        update={
```

```
        "messages": [("system", response)],
        "issue_resolved": issue_status
    },
    goto=END
)

def main():
    """演示 Command 工具内部状态更新"""
    print("=== Command 工具内部状态更新演示 ===\n")

    # 创建图
    builder = StateGraph(SupportState)

    # 添加节点
    builder.add_node("customer_lookup_tool", customer_lookup_tool)
    builder.add_node("support_agent", support_agent)
    builder.add_node("issue_resolver", issue_resolver)

    # 添加边
    builder.add_edge(START, "customer_lookup_tool")

    # 编译图
    graph = builder.compile()

    # 测试用例 1: 金牌会员客户
    print("测试 1: 金牌会员客户")
    initial_state = {
        "customer_id": "CUST001",
        "customer_info": {},
        "messages": [("user", "我需要查询我的账户信息")],
        "issue_resolved": False
    }
    print("初始状态:", initial_state)
    result = graph.invoke(initial_state)
    print("最终状态:", result)
    print("\n" + "="*50 + "\n")

    # 测试用例 2: 不存在的客户
    print("测试 2: 不存在的客户")
    initial_state = {
```

```
        "customer_id": "CUST999",
        "customer_info": {},
        "messages": [("user", "我想查询账户信息")],
        "issue_resolved": False
    }
    print("初始状态:", initial_state)
    result = graph.invoke(initial_state)
    print("最终状态:", result)
    print("\n" + "="*50 + "\n")

# 测试用例 3: 普通会员客户
print("测试 3: 普通会员客户")
initial_state = {
    "customer_id": "CUST003",
    "customer_info": {},
    "messages": [("user", "账户有问题需要处理")],
    "issue_resolved": False
}
print("初始状态:", initial_state)
result = graph.invoke(initial_state)
print("最终状态:", result)

if __name__ == "__main__":
    main()
```

从运行结果可以看出：

- （1）测试 1（金牌会员）：
 - 成功查询到客户信息
 - 提供个性化服务（优先处理）
 - 问题成功解决
- （2）测试 2（不存在的客户）：
 - 查询失败，返回错误信息
 - 客服代理提示客户确认 ID
 - 流程结束
- （3）测试 3（普通会员）：
 - 成功查询到客户信息

- 提供标准服务
- 问题需要进一步处理

在这个示例中，使用 `Command` 对象的优势体现在：

- 状态更新与流程控制一体化：在工具执行后同时更新状态和决定下一步流程。
- 避免额外的条件边：不需要额外的条件边来处理工具执行结果。
- 清晰的控制流：每个节点明确知道下一步要执行什么操作。
- 灵活的状态管理：可以在节点执行过程中精确控制状态更新。

这种模式特别适用于需要在工具执行后立即更新状态并根据结果决定下一步操作的场景，如 API 调用、数据库查询、外部服务集成等。

3.6 Runtime Context

创建 `Graph` 时，可以为传递给节点的运行时上下文指定一个 `context_schema`。这对于向节点传递**不属于图状态**的信息很有用。例如，可以传递诸如模型名称或数据库连接之类的依赖项。

1) 使用 `context_schema` 的优势：

- (1) 分离关注点：将运行时配置与图状态分离，保持状态的纯净性
- (2) 类型安全：通过定义数据类，提供类型检查和 IDE 自动补全支持
- (3) 易于管理：统一管理运行时依赖，如数据库连接、API 密钥等

2) 适用场景包括：

- (1) 传递模型配置信息（如模型名称、参数等）
- (2) 传递数据库连接、API 密钥等敏感信息
- (3) 在不同环境中动态切换配置
- (4) 传递用户身份信息或其他运行时上下文

3) 使用方式

- (1) Context Schema（上下文结构）

使用 `@dataclass` 定义了一个 `ContextSchema` 类，定义的内容不属于图的状态，但在运行时需要被节点访问

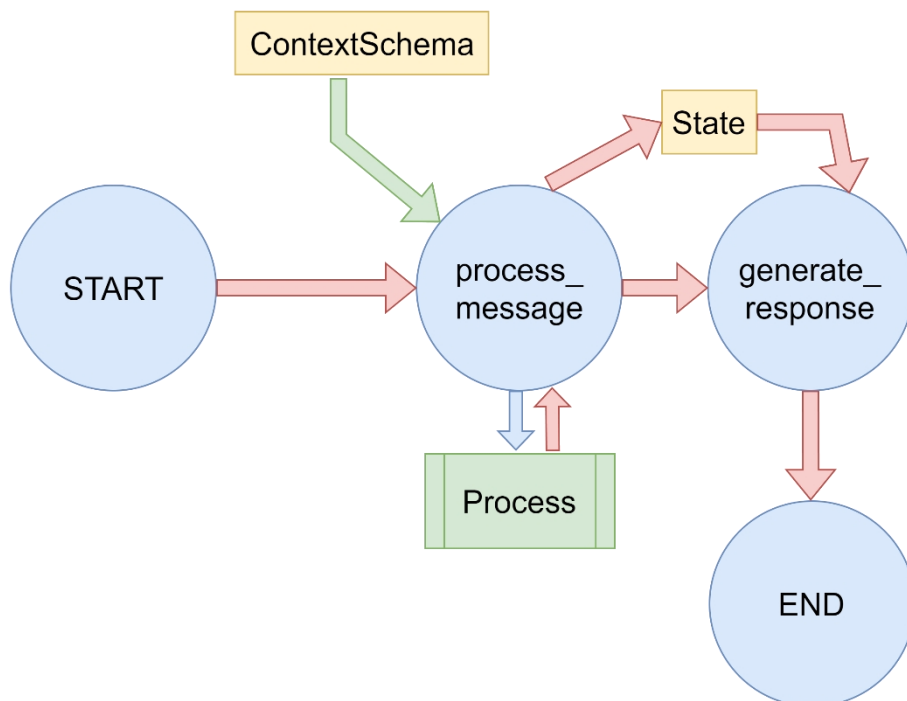
- (2) 节点函数定义

- 节点函数接收两个参数：`state`（图的状态）和 `runtime`（运行时上下文）

- 通过 `runtime.context` 访问上下文信息，如 `runtime.context.model_name`

(3) 图的创建和执行

- 创建 `StateGraph` 时传入 `context_schema=ContextSchema` 参数
- 调用 `graph.invoke()` 时通过 `context` 参数传递上下文数据



4) 案例

"""

LangGraph Context Schema 演示

演示如何在 `LangGraph 1.0` 中使用 `context_schema` 向节点传递不属于图表状态的信息。这在传递模型名称、数据库连接等依赖项时非常有用。

"""

```

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.runtime import Runtime
from langchain_core.messages import AIMessage, HumanMessage
from dataclasses import dataclass
  
```

定义状态结构

```

class AgentState(TypedDict):
  
```

更多 [Java](#) - 大数据 - 前端 - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
messages: Annotated[list, lambda x, y: x + y]
response: str

# 定义上下文结构
@dataclass
class ContextSchema:
    model_name: str
    db_connection: str
    api_key: str

# 节点函数：处理用户消息
def process_message(state: AgentState, runtime: Runtime[ContextSchema]) -> dict:
    """处理用户消息的节点，使用 context 中的信息"""
    print("执行节点：process_message")

    # 获取最新的用户消息
    last_message = state["messages"][-1].content if state["messages"] else ""

    print(f"用户消息：{last_message}")

    # 使用 runtime.context 中的信息
    model_name = runtime.context.model_name
    db_connection = runtime.context.db_connection
    api_key = runtime.context.api_key

    print(f"使用的模型：{model_name}")
    print(f"数据库连接：{db_connection}")
    print(f"API 密钥前缀：{api_key[:5]}***") # 只显示前 5 位，隐藏其余部分

    # 模拟使用这些信息处理请求
    response = f"使用 {model_name} 处理了您的请求，已连接到 {db_connection}"

    return {
        "messages": [AIMessage(content=response)],
        "response": response
    }
```

```
# 节点函数：生成最终响应
def generate_response(state: AgentState, runtime: Runtime[ContextSchema])
-> dict:
    """生成最终响应的节点"""
    print("执行节点：generate_response")

    # 使用 runtime.context 中的信息
    model_name = runtime.context.model_name
    print(f"使用模型 {model_name} 生成最终响应")

    # 获取之前的结果
    previous_response = state["response"]

    # 生成更详细的响应
    final_response = f"{previous_response}\n\n这是使用 {model_name} 生成的完整响应。"

    return {
        "messages": [AIMessage(content=final_response)],
        "response": final_response
    }

def main():
    """演示 context_schema 的使用"""
    print("=== Context Schema 演示 ===\n")

    # 创建图，指定 state_schema 和 context_schema
    builder = StateGraph(AgentState, context_schema=ContextSchema)

    # 添加节点
    builder.add_node("process_message", process_message)
    builder.add_node("generate_response", generate_response)

    # 添加边
    builder.add_edge(START, "process_message")
    builder.add_edge("process_message", "generate_response")
    builder.add_edge("generate_response", END)

    # 编译图
    graph = builder.compile()
```

```
# 定义初始状态
initial_state = {
    "messages": [HumanMessage(content="请帮我查询最新的订单信息")],
    "response": ""
}

# 定义上下文
context = ContextSchema(
    model_name="gpt-4-turbo",
    db_connection="postgresql://user:pass@localhost:5432/orders_db",
    api_key="sk-abcdefghijklmnopqrstuvwxyz123456"
)

print("初始状态:", initial_state)
print("上下文信息:", {
    "model_name": context.model_name,
    "db_connection": context.db_connection,
    "api_key": f"{context.api_key[:5]}***"
})
print("\n" + "-"*50 + "\n")

# 执行图, 通过 context 参数传递上下文
result = graph.invoke(initial_state, context=context)

print("\n" + "="*50)
print("最终状态:", result)
print("\n 最终响应:")
print(result["response"])

if __name__ == "__main__":
    main()
```

3.7 可视化

LangGraph 提供了多种图表可视化方式, 帮助开发者更好地理解和调试工作流。

通过 `graph.get_graph()` 方法可以获取图的结构信息, 包括节点和边的详细信息。基于这个信息, 可以使用如下方式进行可视化:

- 生成 Mermaid 代码来可视化图结构。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

- 生成简单的 ASCII 文本图表，但需要安装额外的依赖。

安装 grandalf 来支持 ASCII 可视化。

```
pip install grandalf
```

案例：

```
"""
简单的 LangGraph 可视化演示

这个演示展示了一个更简单的图结构，用于更好地理解可视化功能
"""

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

# 定义状态
class SimpleState(TypedDict):
    value: int
    messages: Annotated[list, lambda x, y: x + y]

# 定义节点函数
def node_a(state: SimpleState) -> dict:
    """节点 A"""
    print("执行节点 A")
    return {
        "value": state["value"] + 1,
        "messages": [("system", "执行了节点 A")]
    }

def node_b(state: SimpleState) -> dict:
    """节点 B"""
    print("执行节点 B")
    return {
        "value": state["value"] * 2,
        "messages": [("system", "执行了节点 B")]
    }

def node_c(state: SimpleState) -> dict:
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
    """节点 C"""
    print("执行节点 C")
    return {
        "value": state["value"] + 10,
        "messages": [("system", "执行了节点 C")]
    }

def main():
    """简单可视化演示"""
    print("=== 简单 LangGraph 可视化演示 ===\n")

    # 创建图
    builder = StateGraph(SimpleState)

    # 添加节点
    builder.add_node("node_a", node_a)
    builder.add_node("node_b", node_b)
    builder.add_node("node_c", node_c)

    # 添加边
    builder.add_edge(START, "node_a")
    builder.add_edge("node_a", "node_b")
    builder.add_edge("node_b", "node_c")
    builder.add_edge("node_c", END)

    # 编译图
    graph = builder.compile()

    # 获取图结构
    graph_structure = graph.get_graph()

    print("1. Mermaid 图表代码:")
    try:
        mermaid_code = graph_structure.draw_mermaid()
        print(mermaid_code)
    except Exception as e:
        print(f"生成 Mermaid 图表失败: {e}")
    print()

    print("2. ASCII 文本可视化（需要安装 grandalf）:")
```

```
try:
    ascii_graph = graph_structure.draw_ascii()
    print(ascii_graph)
except Exception as e:
    print(f"ASCII 可视化失败: {e}")
    print("提示: 可以通过以下命令安装 grandalf 来支持 ASCII 可视化:")
    print("  pip install grandalf")
print()

print("3. 执行图:")
initial_state = {
    "value": 5,
    "messages": []
}

print("初始状态:", initial_state)
result = graph.invoke(initial_state)
print("最终状态:", result)

if __name__ == "__main__":
    main()
```

打印的 Mermaid 代码可以用支持的工具进行展示，或者用在线网站查看效果

(<https://mermaid.live/>) :

☰

Mermaid Live Editor

🔍

Playground

- more features, no account required

📄

Code

⚙️

Config

📖

Docs

```
1  ---
2  config:
3    flowchart:
4      curve: linear
5  ---
6  graph TD;
7    __start__([<p>__start__</p>])):::first
8    node_a(node_a)
9    node_b(node_b)
10   node_c(node_c)
11   __end__([<p>__end__</p>])):::last
12   __start__ --> node_a;
13   node_a --> node_b;
14   node_b --> node_c;
15   node_c --> __end__;
16   classDef default fill:#f2f0ff,line-height:1.2
17   classDef first fill-opacity:0
18   classDef last fill:#bfb6fc
19
```

把生成的Mermaid代码粘贴到这里

🔍

Sample Diagrams

✕

Flowchart

Class

Sequence

Entity Relationship

State

Mindmap

Architecture

Block

C4

Gantt

Git

Kanban

Packet

Pie

Quadrant

Radar

Requirement

Sankey

Timeline

Treemap

User Journey

XY

ZenUML

自动生成的可视化图片

如果使用 Jupyter Notebook, 可直接查看 Mermaid 图片, 调用方式如下:

```
### 需要先安装 Ipython: pip install ipython
from IPython.display import Image, display
display(Image(graph.get_graph().draw_mermaid_png()))
```

ASCII 文本图表在控制台直接可以看到:

```
2. ASCII 文本可视化 (需要安装 grandalf):  
+-----+  
| __start__ |  
+-----+  
      *  
      *  
      *  
+-----+  
| node_a |  
+-----+  
      *  
      *  
      *  
+-----+  
| node_b |  
+-----+  
      *  
      *  
      *  
+-----+  
| node_c |  
+-----+  
      *  
      *  
      *  
+-----+  
| __end__ |  
+-----+
```

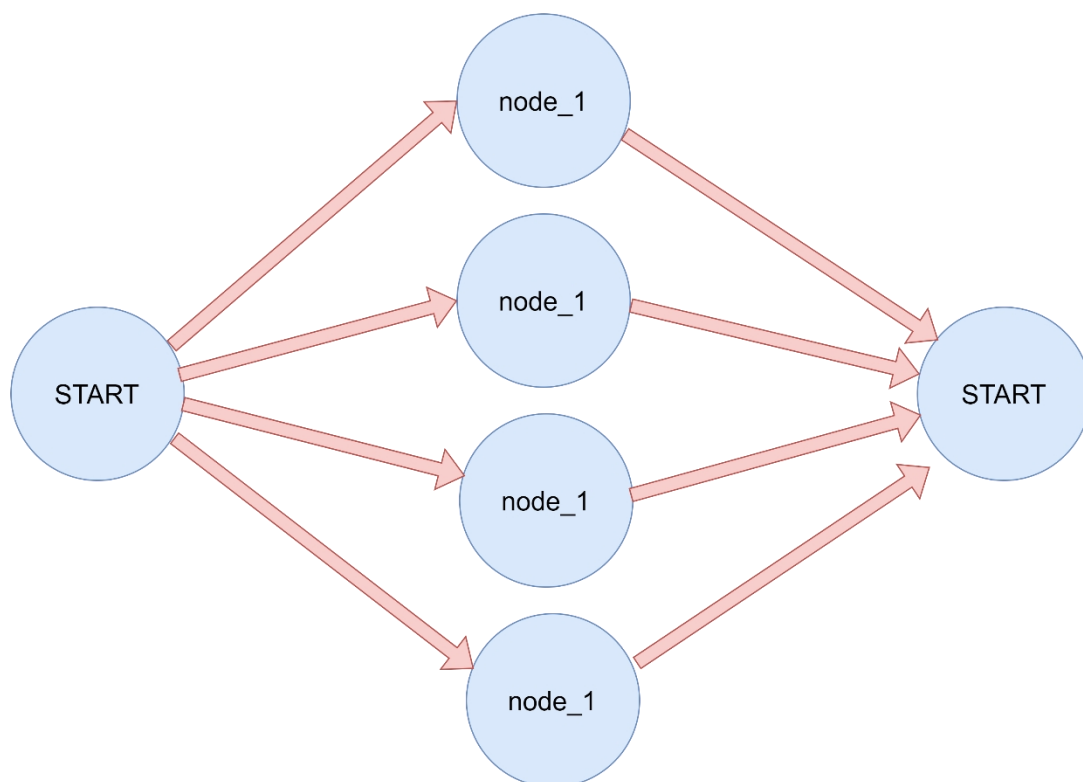
3.8 Async 异步编程

采用异步编程范式在并发运行受 IO 限制的代码时（例如，向聊天模型提供商并发发送 API 请求），可以显著提升性能。

（1）使用 `async/await` 语法实现异步 IO 操作

- 所有节点函数都是 `async def` 定义的异步函数
- 使用 `await graph.ainvoke()` 而不是 `graph.invoke()` 来执行异步图

（2）利用 `asyncio.gather()` 实现任务并发执行



"""

LangGraph 异步并发 IO 任务演示

本示例展示了如何使用 **LangGraph 1.0** 版本和异步编程范式，在处理 IO 密集型任务（如并发 API 请求）时显著提升性能。

"""

```

import asyncio
import time
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
  
```

```

class IOState(TypedDict):
  
```

"""

状态定义，用于跟踪并发 IO 任务的结果

"""

results: Annotated[list, lambda x, y: x + y] # 合并列表结果

task_count: int # 总任务数

```
async def simulated_api_call(task_id: int, delay: float = 1.0) -> str:
    """
    模拟 API 调用的异步函数

    Args:
        task_id: 任务 ID
        delay: 延迟时间（秒），模拟网络请求耗时

    Returns:
        str: API 响应结果
    """
    print(f"开始执行 API 调用 {task_id}...")
    await asyncio.sleep(delay) # 模拟网络 IO 延迟
    result = f"API 调用 {task_id} 的结果"
    print(f"完成 API 调用 {task_id}")
    return result


async def io_task_1(state: IOState) -> dict:
    """
    IO 密集型任务 1 - 模拟 API 调用
    """
    result = await simulated_api_call(1, 1.0)
    return {"results": [result]}


async def io_task_2(state: IOState) -> dict:
    """
    IO 密集型任务 2 - 模拟 API 调用
    """
    result = await simulated_api_call(2, 1.5)
    return {"results": [result]}


async def io_task_3(state: IOState) -> dict:
    """
    IO 密集型任务 3 - 模拟 API 调用
    """
    result = await simulated_api_call(3, 0.8)
    return {"results": [result]}
```

```
async def io_task_4(state: IOState) -> dict:
    """
    IO 密集型任务 4 - 模拟 API 调用
    """
    result = await simulated_api_call(4, 1.2)
    return {"results": [result]}

def summary_node(state: IOState) -> dict:
    """
    汇总节点 - 收集所有并发任务的结果
    """
    print(f"\n=== 所有任务已完成 ===")
    print(f"总共执行了 {state['task_count']} 个任务")
    print(f"收集到 {len(state['results'])} 个结果:")
    for i, result in enumerate(state['results'], 1):
        print(f" {i}. {result}")

    return {"results": state["results"] + ["所有任务已完成"]}

# 同步版本用于对比性能
def sync_version():
    """
    同步版本用于性能对比
    """
    print("=== 同步执行版本 ===")
    start_time = time.time()

    results = []
    delays = [1.0, 1.5, 0.8, 1.2]

    for i, delay in enumerate(delays, 1):
        print(f"开始执行同步任务 {i}...")
        time.sleep(delay) # 模拟阻塞 IO
        result = f"同步任务 {i} 的结果"
        results.append(result)
        print(f"完成同步任务 {i}")

    elapsed = time.time() - start_time
    print(f"\n同步执行总耗时: {elapsed:.2f} 秒")
```

```
    return results

async def async_version():
    """
    异步版本用于性能对比
    """
    print("\n=== 异步执行版本 ===")
    start_time = time.time()

    # 创建所有任务
    tasks = [
        simulated_api_call(1, 1.0),
        simulated_api_call(2, 1.5),
        simulated_api_call(3, 0.8),
        simulated_api_call(4, 1.2)
    ]

    # 并发执行所有任务
    results = await asyncio.gather(*tasks)

    elapsed = time.time() - start_time
    print(f"\n 异步执行总耗时: {elapsed:.2f} 秒")
    return results

def build_async_graph():
    """
    构建 LangGraph 异步 workflow
    """
    builder = StateGraph(IOState)

    # 添加节点
    builder.add_node("io_task_1", io_task_1)
    builder.add_node("io_task_2", io_task_2)
    builder.add_node("io_task_3", io_task_3)
    builder.add_node("io_task_4", io_task_4)
    builder.add_node("summary", summary_node)

    # 添加边 - 所有 IO 任务从 START 并行开始
    builder.add_edge(START, "io_task_1")
```

```
builder.add_edge(START, "io_task_2")
builder.add_edge(START, "io_task_3")
builder.add_edge(START, "io_task_4")

# 所有 IO 任务完成后汇聚到 summary 节点
builder.add_edge("io_task_1", "summary")
builder.add_edge("io_task_2", "summary")
builder.add_edge("io_task_3", "summary")
builder.add_edge("io_task_4", "summary")
builder.add_edge("summary", END)

# 编译图
return builder.compile()

async def main():
    """
    主函数 - 演示异步并发 IO 任务的优势
    """
    print("LangGraph 异步并发 IO 任务演示")
    print("=" * 50)

    # 1. 先展示传统的同步执行方式
    sync_results = sync_version()

    # 2. 展示纯异步执行方式
    async_results = await async_version()

    # 3. 展示使用 LangGraph 的异步工作流
    print("\n=== LangGraph 异步工作流版本 ===")
    start_time = time.time()

    # 构建异步图
    graph = build_async_graph()

    # 执行图
    result = await graph.ainvoke({
        "results": [],
        "task_count": 4
    })
```

```

elapsed = time.time() - start_time
print(f"\nLangGraph 异步工作流总耗时: {elapsed:.2f} 秒")

# 4. 性能对比总结
print("\n=== 性能对比总结 ===")
print("通过异步并发执行 IO 密集型任务，可以显著提升性能:")
print("- 同步执行：任务依次执行，总耗时为各任务耗时之和")
print("- 异步执行：任务并发执行，总耗时近似于最耗时的任务")
print("- LangGraph 异步工作流：提供了结构化的异步任务编排能力")

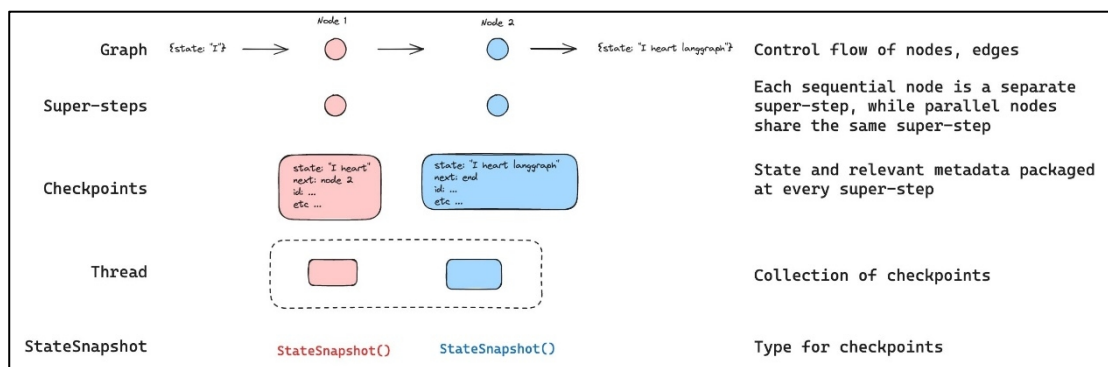
if __name__ == "__main__":
    # 运行主函数
    asyncio.run(main())

```

第 4 章 高级特性

4.1 持久化（Persistence）

LangGraph 具有内置的持久化层，通过检查点工具来实现。当使用检查点工具编译图时，检查点工具会在每个超级步骤保存图状态的 checkpoint。这些检查点会被保存到一个 thread 中，在图执行后可以访问该线程。由于 threads 允许在执行后访问图的状态，因此多种强大功能成为可能，包括人机协同、记忆、时间回溯和容错等。



LangGraph API 会自动处理检查点。使用 LangGraph API 时，无需手动实现或配置检查点工具。该 API 会在后台为您处理所有持久化基础架构。

4.1.1 Threads

线程（Thread）是检查点工具为其保存的每个检查点分配的唯一 ID 或线程标识符，它包含了一系列运行实例的累积状态。当某个运行实例执行时，底层图的状态会持久化到该线程中。使用检查点调用图时，必须在配置的可配置部分指定 `thread_id`。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
{"configurable": {"thread_id": "1"}}
```

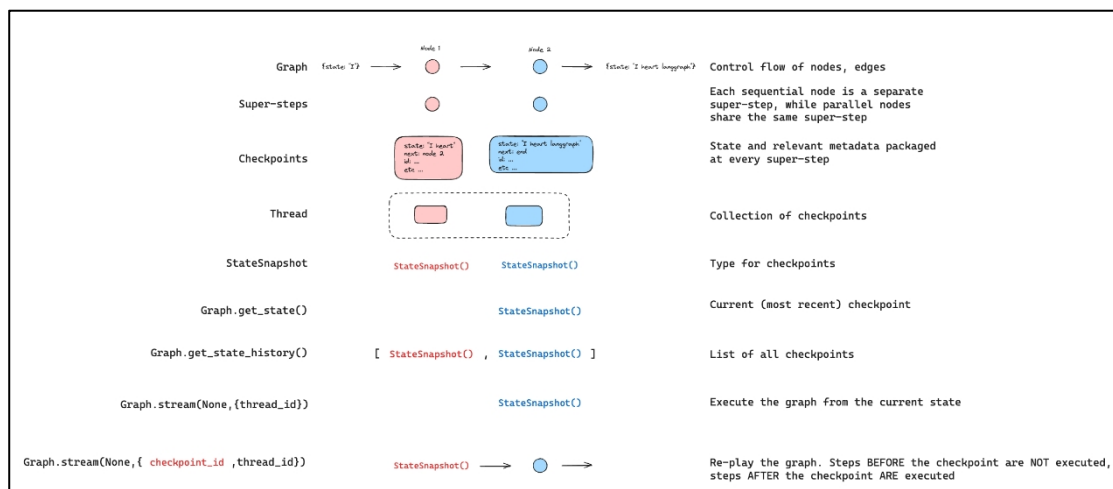
可以检索线程的当前状态和历史状态。要持久化状态，必须在执行运行之前创建线程。

4.1.2 Checkpoints

线程在特定时间点的状态称为检查点。检查点是在每个超级步骤保存的图状态快照，由具有以下关键属性的 StateSnapshot 对象表示：

- **config**: 与此检查点相关联的配置。
- **metadata**: 与此检查点相关联的元数据。
- **values**: 此时状态通道的值。
- **next**: 下一个要在图中执行的节点名称元组。
- **tasks**: 一个包含 PregelTask 对象的元组，这些对象包含关于接下来要执行的任务的信息。如果该步骤之前已尝试过，它将包含错误信息。如果图形从节点内部被动态中断，任务将包含与中断相关的额外数据。

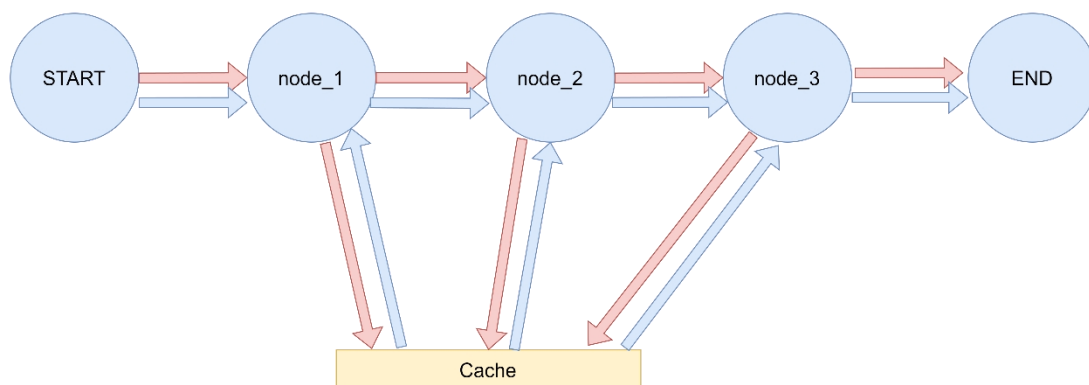
检查点会被持久化，并可用于在之后恢复线程的状态。



- **graph.get_state(config)**: 来查看图的最新状态。
- **graph.get_state_history(config)**: 可以获取特定线程的图执行完整历史。

4.1.3 内存检查点

langgraph-checkpoint: 检查点保存器（BaseCheckpointSaver）的基础接口以及序列化/反序列化接口（SerializerProtocol）。包含用于实验的内存中检查点实现（InMemorySaver）。LangGraph 已内置 langgraph-checkpoint。



"""

LangGraph 1.0 持久化存储演示 - 内存存储 (In-Memory)

特点:

- 数据暂存于内存，程序关闭后丢失
- 无需额外配置
- 适用于本地测试和临时验证 workflows 逻辑

"""

```

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
import operator

```

定义状态

```

class PersistenceDemoState(TypedDict):
    messages: Annotated[list, operator.add]
    step_count: Annotated[int, operator.add]

```

节点函数

```

def step_one(state: PersistenceDemoState) -> dict:
    print("执行步骤 1")
    return {
        "messages": ["执行了步骤 1"],
        "step_count": 1
    }

```

```

def step_two(state: PersistenceDemoState) -> dict:

```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
print("执行步骤 2")
return {
    "messages": ["执行了步骤 2"],
    "step_count": 1
}

def step_three(state: PersistenceDemoState) -> dict:
    print("执行步骤 3")
    return {
        "messages": ["执行了步骤 3"],
        "step_count": 1
    }

# 构建图
def create_graph():
    builder = StateGraph(PersistenceDemoState)
    builder.add_node("step_one", step_one)
    builder.add_node("step_two", step_two)
    builder.add_node("step_three", step_three)

    builder.add_edge(START, "step_one")
    builder.add_edge("step_one", "step_two")
    builder.add_edge("step_two", "step_three")
    builder.add_edge("step_three", END)

    return builder

def main():
    print("=== LangGraph 1.0 内存持久化存储演示 ===\n")

    # 创建内存存储器
    memory = InMemorySaver()

    # 编译图并使用内存存储
    graph = create_graph()
    app = graph.compile(checkpointer=memory)

    # 配置线程 ID 用于存储状态
```

```
config = {"configurable": {"thread_id": "demo_thread_1"}}

print("1. 首次执行工作流:")
result = app.invoke({
    "messages": ["开始执行"],
    "step_count": 0
}, config)

print(f"执行结果: {result}\n")

print("2. 检查存储的状态:")
saved_state = app.get_state(config)
print(f"保存的状态: {saved_state.values}")
print(f"下一个节点: {saved_state.next}\n")

print("3. 恢复执行工作流:")
# 由于工作流已经完成, 这里会直接返回最终结果
result2 = app.invoke(None, config)
print(f"恢复执行结果: {result2}\n")

print("=== 演示结束 ===")

if __name__ == "__main__":
    main()
```

4.1.4 数据库检查点

在底层, 检查点功能由符合 BaseCheckpointSaver 接口的检查点对象提供支持。LangGraph 提供了多种检查点实现, 所有这些实现都通过独立的、可安装的库来完成, 数据库类型的有:

- langgraph-checkpoint-sqlite: 使用 SQLite 数据库 (SqliteSaver / AsyncSqliteSaver) 存储检查点。非常适合实验和本地工作流程。需要单独安装。
- langgraph-checkpoint-postgres : 使用 Postgres 数据库 (PostgresSaver / AsyncPostgresSaver) 存储检查点, 用于 LangSmith。非常适合在生产环境中使用。需要单独安装。

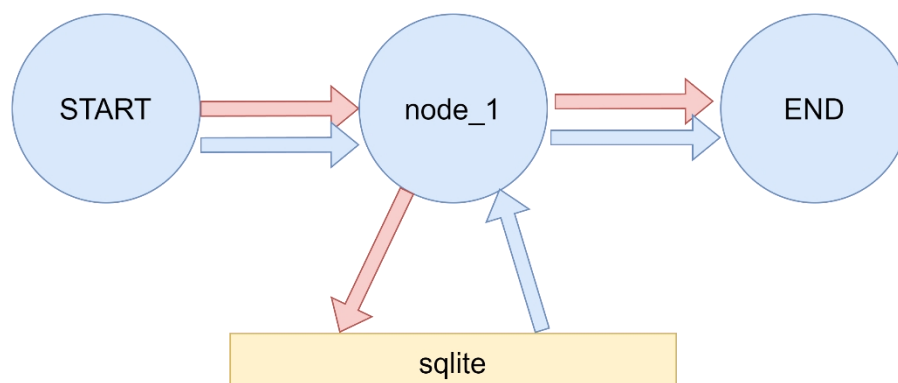
4.1.4.1 sqlite

1) 安装 sqlite 所需依赖

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
pip install langgraph-checkpoint-sqlite
```

2) 案例一



```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import operator
from typing import TypedDict, Annotated

from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import StateGraph, START, END
import sqlite3

class MyState(TypedDict):
    messages: Annotated[list, operator.add]

def node_1(state: MyState):

    return {"messages": ["abc", "def"]}

def main():
    # 数据存储到 sqlite_data 目录下面，需要目录存在
    conn =
    sqlite3.connect(database="./sqlite_data/langgraph_sqlite", check_same_thread=False)
    memory = SqliteSaver(conn=conn)

    builder = StateGraph(MyState)
    builder.add_node("node_1", node_1)
    builder.add_edge(START, "node_1")
    builder.add_edge("node_1", END)

    graph = builder.compile(checkpointer=memory)
```

```
config = {"configurable": {"thread_id": "1"}}

initial_state = graph.get_state(config)
print(f"Initial state: {initial_state}")

# 执行图
result = graph.invoke({"messages": []}, config)
print(f"Result: {result}")

# 查看执行后的状态
final_state = graph.get_state(config)
print(f"Final state: {final_state}")

conn.close()

if __name__ == '__main__':
    main()
```

4.2 持久化执行（Durable execution）

LangGraph 中的“持久化执行”指：即便在工作流执行过程中遭遇系统崩溃、网络中断或服务重启等意外状况，仍能确保工作流从断点处继续执行，而非从头重新运行的能力。它是对“持久化（Persistence）”能力的延伸——持久化侧重“状态保存”，而持久化执行侧重“基于已保存的状态实现可靠续跑”。

持久执行是一种流程或工作流在关键节点保存进度的技术，它允许流程暂停，之后能从暂停的精确位置继续执行。这在需要人机协同的场景中尤为有用，在这类场景中，用户可以在继续执行前检查、验证或修改流程；同时，在可能遇到中断或错误（例如，调用大语言模型超时）的长时间运行任务中也很有帮助。通过保留已完成的工作，持久执行使流程能够在无需重新处理先前步骤的情况下恢复——即使经过较长时间的延迟（例如，一周后）。

LangGraph 的内置持久化层为工作流提供持久执行能力，确保每个执行步骤的状态都能保存到持久化存储中。这一功能保证，无论工作流因系统故障还是人机交互而中断，都能从最后记录的状态恢复执行。

4.2.1 确定性与一致重放

当你恢复工作流运行时，代码不会从执行停止的同一行代码处恢复；相反，它会确定一个合适的起始点，从该点继续执行。这意味着工作流将从起始点重新执行所有步骤，直到到达停止的位置。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

因此，当你为持久化执行编写 workflow 时，必须将任何非确定性操作（例如随机数生成）以及任何具有副作用的操作（例如文件写入、API 调用）封装在任务或节点中。

为确保 workflow 具有确定性且能够被一致地重放，确保以下几点：

1) 避免重复工作

如果一个节点包含多个具有副作用的操作（例如，日志记录、文件写入或网络调用），请将每个操作包装在单独的任务中。这确保了在 workflow 恢复时，这些操作不会被重复执行，并且它们的结果会从持久化层中检索。

2) 封装非确定性操作

将任何可能产生非确定性结果的代码（例如随机数生成）包装在**任务或节点**中。这确保了在恢复时，workflow 会按照确切记录的步骤序列执行，并得到相同的结果。

3) 使用幂等操作

在可能的情况下，确保副作用（例如 API 调用、文件写入）是幂等的。这意味着，如果某个操作在 workflow 中失败后重试，其效果将与第一次执行时相同。这对于导致数据写入的操作尤为重要。如果某个**任务**开始执行但未能成功完成，workflow 的恢复将重新运行该**任务**，并依靠已记录的结果来保持一致性。使用幂等键或验证现有结果，以避免意外的重复，确保 workflow 执行顺畅且可预测。

参照后面 5.12 章节。

4.2.2 持久性模式

LangGraph 支持三种持久性模式，从持久性最低到最高的模式如下：

- "exit"
- "async"
- "sync"

更高的持久性模式会给 workflow 执行增加更多开销。

调用任何图执行方法时，你可以指定持久性模式：

```
graph.stream(  
    {"input": "test"},  
    durability="sync"  
)
```

1) exit

只有当图执行完成（无论是成功完成还是出现错误）时，更改才会被持久化。这为长时间运行的图提供了最佳性能，但意味着中间状态不会被保存，因此您无法从中途执行失败中恢复，也无法中断图的执行。

2) async

在执行下一步时，变更会异步持久化。这提供了良好的性能和耐久性，但存在一个小风险：如果进程在执行期间崩溃，检查点可能无法写入。

3) sync

在下一个步骤开始前，变更会被同步持久化。这确保了每个检查点都在继续执行前写入，以一定的性能开销为代价提供了高持久性。

4.2.3 恢复 workflow

在 workflow 中启用持久化执行后，可以在以下场景中恢复执行：

暂停和恢复 workflow：使用中断函数在特定点暂停 workflow，并使用 `Command` 原语通过更新的状态恢复 workflow（参见 `Graph API` 中的 3.5.5 的示例）。

从故障中恢复：发生异常（例如，大语言模型提供商中断）后，自动从最后一个成功的检查点恢复 workflow。这需要通过提供 `None` 作为输入值，使用相同的线程标识符执行 workflow（参见函数式 API 中 5.8 的示例）。

4.3 流处理 (Streaming)

LangGraph 实现了一个流式传输系统，以呈现实时更新。通过逐步显示输出内容（即便完整响应尚未生成），流式传输能显著提升用户体验 (UX)，尤其在应对大语言模型 (LLMs) 的延迟问题时效果突出。

LangGraph 流式传输可以实现的功能：

- **图状态流式输出：**通过 `updates` 和 `values` 模式获取图状态（也即通过 `state_schema` 指定的状态）的更新。
- **子图流式输出：**将所有父图和嵌套子图输出内容进行流式输出。
- **大模型 tokens 流式输出：**捕获来自于工具调用，子节点，子图等地方所有的 `token`，并进行流式输出。
- **流式传输自定义数据：**直接从工具函数发送自定义更新或进度信号。

- 使用多种流模式：可从 `values`（完整状态）、`updates`（状态增量）、`messages`（大语言模型 `tokens`+元数据）、`custom`（任意用户数据）或 `debug`（详细跟踪）中选择多种模式进行流式输出。

4.3.1 支持的流模式

将以下一种或多种流模式以列表形式传递给 `stream` 或 `astream` 方法：

模式	描述
<code>values</code>	在图的每一步之后流式传输状态的完整值。
<code>updates</code>	在图的每一步之后流式传输状态更新。如果在同一步骤中进行了多项更新（例如，运行了多个节点），这些更新将被单独流式传输。
<code>custom</code>	从图节点内部流式传输自定义数据。
<code>messages</code>	从任何调用了大语言模型的图节点流式传输二元组（大语言模型 <code>token</code> ，元数据）。
<code>debug</code>	在图的整个执行过程中尽可能多地流式传输信息。

4.3.2 基本用法

LangGraph 有 `stream`（同步）和 `astream`（异步）方法，以迭代器的形式生成流式输出。

4.3.2.1 用法介绍

1) 多模式流

将列表作为 `stream_mode` 参数传递，以同时流式传输多种模式。流式输出将是 `(mode, chunk)` 形式的元组，其中 `mode` 是流模式的名称，`chunk` 是该模式所流式传输的数据。

2) 流图状态

使用流模式 `updates` 和 `values` 来流式传输图在执行时的状态。

3) 调试

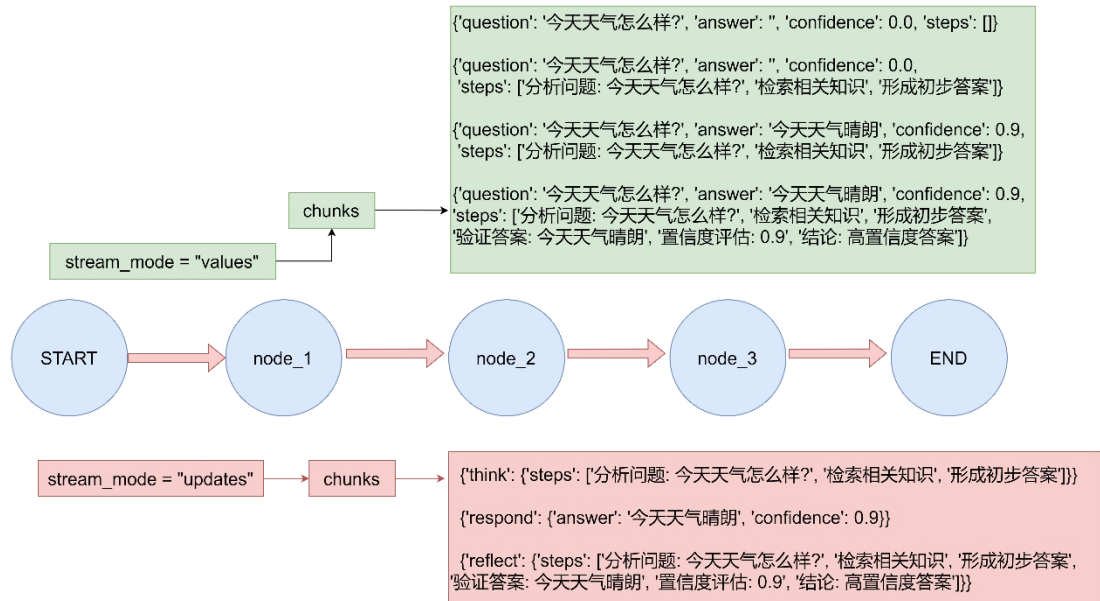
使用 `debug` 流模式，在图的执行过程中流式传输尽可能多的信息。流式输出包括节点名称以及完整状态。

4) 流式输出子图结果

要在流式输出中包含子图的输出，可以在父图的 `.stream()` 方法中设置 `subgraphs=True`。这样将同时流式传输父图和所有子图的输出。

4.3.2.2 案例演示

1) 案例一：多模式流



```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
LangGraph 多模式流式传输演示
展示 LangGraph 支持的各种流模式
"""

from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# 定义状态类型
class State(TypedDict):
    question: str
    answer: str
    confidence: float
    steps: list

def think(state: State) -> State:
    """思考节点"""
    question = state["question"]
    # 模拟思考过程
    steps = [f"分析问题: {question}", "检索相关知识", "形成初步答案"]
```

```
    return {"steps": steps}

def respond(state: State) -> State:
    """回应节点"""
    question = state["question"]
    # 根据问题生成答案
    if "天气" in question:
        answer = "今天天气晴朗"
        confidence = 0.9
    elif "时间" in question:
        answer = "现在是上午 10 点"
        confidence = 0.8
    else:
        answer = "这是一个很好的问题"
        confidence = 0.7

    return {
        "answer": answer,
        "confidence": confidence
    }

def reflect(state: State) -> State:
    """反思节点"""
    answer = state["answer"]
    confidence = state["confidence"]
    steps = state.get("steps", [])

    steps.append(f"验证答案: {answer}")
    steps.append(f"置信度评估: {confidence}")

    if confidence > 0.8:
        conclusion = "高置信度答案"
    elif confidence > 0.5:
        conclusion = "中等置信度答案"
    else:
        conclusion = "低置信度答案"

    steps.append(f"结论: {conclusion}")
```

```
    return {"steps": steps}

def main():
    # 构建图
    builder = StateGraph(State)
    builder.add_node("think", think)
    builder.add_node("respond", respond)
    builder.add_node("reflect", reflect)

    builder.add_edge(START, "think")
    builder.add_edge("think", "respond")
    builder.add_edge("respond", "reflect")
    builder.add_edge("reflect", END)

    graph = builder.compile()

    print("=== LangGraph 多模式流式传输演示 ===\n")

    # 准备输入
    input_state = {
        "question": "今天天气怎么样?",
        "answer": "",
        "confidence": 0.0,
        "steps": []
    }

    print("---- 1. 使用 stream_mode='values' 模式 ----")
    print("显示每一步执行后的完整状态:")
    for chunk in graph.stream(input_state, stream_mode="values"):
        print(f" {chunk}")

    print("\n" + "="*60 + "\n")

    print("---- 2. 使用 stream_mode='updates' 模式 ----")
    print("只显示每一步的状态更新:")
    for chunk in graph.stream(input_state, stream_mode="updates"):
        print(f" {chunk}")

    print("\n" + "="*60 + "\n")
```

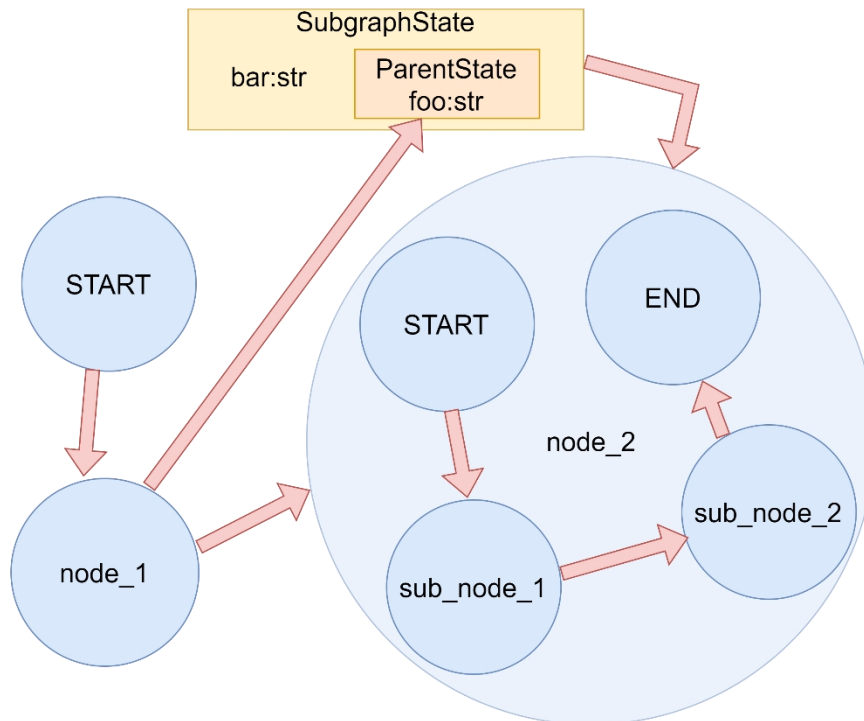
```
print("--- 3. 同时使用多种流模式 ---")
print("同时显示完整状态和状态更新:")
for mode, chunk in graph.stream(input_state, stream_mode=["values",
"updates"]):
    print(f" [{mode}]: {chunk}")

print("\n" + "="*60 + "\n")

print("--- 4. 使用 debug 模式 ---")
print("显示详细的调试信息:")
try:
    for chunk in graph.stream(input_state, stream_mode="debug"):
        print(f" {chunk}")
except Exception as e:
    print(f" Debug 模式可能需要特殊配置: {e}")

if __name__ == "__main__":
    main()
```

2) 案例二：流式输出子图结果



```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

LangGraph 子图流式传输演示

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：尚硅谷官网

展示如何在流式输出中包含子图的输出

```
"""

from langgraph.graph import START, StateGraph, END
from typing import TypedDict

# 定义子图状态
class SubgraphState(TypedDict):
    foo: str # 注意这个键与父图状态共享
    bar: str

def subgraph_node_1(state: SubgraphState):
    """子图节点 1"""
    print(f"  执行子图节点 1, 当前状态: {state}")
    return {"bar": "bar"}

def subgraph_node_2(state: SubgraphState):
    """子图节点 2"""
    print(f"  执行子图节点 2, 当前状态: {state}")
    return {"foo": state["foo"] + state["bar"]}

# 定义父图状态
class ParentState(TypedDict):
    foo: str

def node_1(state: ParentState):
    """父图节点 1"""
    print(f"  执行父图节点 1, 当前状态: {state}")
    return {"foo": "hi! " + state["foo"]}

def main():
    print("=== LangGraph 子图流式传输演示 ===\n")

    # 创建子图
    subgraph_builder = StateGraph(SubgraphState)
    subgraph_builder.add_node("subgraph_node_1", subgraph_node_1)
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
subgraph_builder.add_node("subgraph_node_2", subgraph_node_2)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph_builder.add_edge("subgraph_node_1", "subgraph_node_2")
subgraph_builder.add_edge("subgraph_node_2", END)
subgraph = subgraph_builder.compile()

# 创建父图
builder = StateGraph(ParentState)
builder.add_node("node_1", node_1)
builder.add_node("node_2", subgraph) # 将子图作为节点添加到父图中
builder.add_edge(START, "node_1")
builder.add_edge("node_1", "node_2")
graph = builder.compile()

print("--- 1. 不包含子图的常规流式输出 ---")
for chunk in graph.stream(
    {"foo": "foo"},
    stream_mode="updates"
):
    print(f" 流式输出块: {chunk}")

print("\n" + "="*50 + "\n")

print("--- 2. 包含子图的流式输出 (subgraphs=True) ---")
for chunk in graph.stream(
    {"foo": "foo"},
    stream_mode="updates",
    # 设置 subgraphs=True 来流式传输子图的输出
    subgraphs=True,
):
    print(f" 流式输出块: {chunk}")

print("\n" + "="*50 + "\n")

print("--- 3. 使用 values 模式并包含子图输出 ---")
for chunk in graph.stream(
    {"foo": "foo"},
    stream_mode="values",
    subgraphs=True
):
    print(f" 流式输出块: {chunk}")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
print("\n" + "="*50 + "\n")

print("--- 4. 详细分析子图流式输出 ---")
print("当 subgraphs=True 时, 输出格式为 (namespace, chunk) 元组:")
for chunk in graph.stream(
    {"foo": "foo"},
    stream_mode="updates",
    subgraphs=True
):
    namespace, data = chunk
    if namespace:
        print(f" 子图 '{namespace[0]}' 输出: {data}")
    else:
        print(f" 父图输出: {data}")

if __name__ == "__main__":
    main()
```

4.3.3 流式输出 LLM 响应

使用 messages 流模式, 从图中的任何部分 (包括节点、工具、子图或任务) 逐 **token** 流式传输大型语言模型 (LLM) 的输出。

messages 模式的流式输出是一个元组(message_chunk, metadata), 其中:

- message_chunk: 来自大语言模型 (LLM) 的令牌或消息片段。
- metadata: 一个包含图节点和大语言模型调用详情的字典。

执行 `pip install langchain` `pip install -U langchain-openai`

```
from typing import TypedDict
from langgraph.graph import StateGraph, START
from langchain.chat_models import init_chat_model

model = init_chat_model(model="gpt-4o-mini", model_provider="openai")

class State(TypedDict):
    query:str
    answer:str
```



```
def node(state:State):
    print("开始调用 node 节点")
    llm_result = model.invoke(
        [("user",state["query"])]
    )
    print("llm invoke 结束")
    return {"answer":llm_result}

def main():
    graph = (
        StateGraph(
            state_schema=State
        )
        .add_node(node)
        .add_edge(START,"node")
        .compile()
    )
    inputs = {"query":"帮我生成一个 300 字的小学生作文，主题为我的一天"}
#
    for chunk,meta_data in graph.stream(inputs,stream_mode="messages"):
        # print(f"type of chunk:{type(chunk)}")
        print(chunk.content,end="")

if __name__ == '__main__':
    main()
```

4.3.4 流式传输自定义数据

要从 LangGraph 节点或工具内部发送自定义用户定义数据，请遵循以下步骤：

- 使用 `get_stream_writer` 访问流写入器并发送自定义数据。
- 调用 `.stream()` 或 `.astream()` 时，设置 `stream_mode="custom"` 以在流中获取自定义数据。

你可以组合多种模式（例如 `["updates", "custom"]`），但至少有一种模式必须是 `"custom"`。

1) 从节点内部发送自定义用户定义数据

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
LangGraph 自定义数据流式传输演示
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

展示如何从节点内部发送自定义用户定义数据

```
"""
```

```
from typing import TypedDict
from langgraph.config import get_stream_writer
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    query: str
    answer: str
    progress: list

def node_with_custom_streaming(state: State) -> State:
    """带自定义流式传输的节点"""
    # 获取流写入器以发送自定义数据
    writer = get_stream_writer()

    # 发送自定义数据（例如，进度更新）
    writer({"custom_key": "开始处理查询"})
    writer({"progress": "步骤 1: 分析查询内容", "status": "running"})

    query = state["query"]
    # 模拟处理过程
    result = f"处理结果: {query.upper()}"

    writer({"progress": "步骤 2: 生成结果", "status": "running"})
    writer({"progress": "步骤 3: 完成处理", "status": "completed"})
    writer({"custom_key": "查询处理完成"})

    return {
        "answer": result,
        "progress": state.get("progress", []) + ["处理完成"]
    }

def main():
    print("=== LangGraph 自定义数据流式传输演示 ===\n")

    # 构建图
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
graph = (  
    StateGraph(State)  
        .add_node("node_with_custom_streaming",  
node_with_custom_streaming)  
        .add_edge(START, "node_with_custom_streaming")  
        .add_edge("node_with_custom_streaming", END)  
        .compile()  
)  
  
inputs = {"query": "hello world", "answer": "", "progress": []}  
  
print("--- 1. 使用 custom 流模式 ---")  
try:  
    # 设置 stream_mode="custom" 以在流中接收自定义数据  
    for chunk in graph.stream(inputs, stream_mode="custom"):  
        print(f"自定义数据块: {chunk}")  
except Exception as e:  
    print(f"错误: {e}")  
    print("说明: 在 Graph API 中, 自定义流数据需要在节点中通过特定方式发送")  
")  
  
print("\n" + "="*50 + "\n")  
  
print("--- 2. 使用 updates 流模式 ---")  
for chunk in graph.stream(inputs, stream_mode="updates"):  
    print(f"状态更新: {chunk}")  
  
print("\n" + "="*50 + "\n")  
  
print("--- 3. 同时使用 custom 和 updates 流模式 ---")  
try:  
    for mode, chunk in graph.stream(inputs, stream_mode=["custom",  
"updates"]):  
        print(f"[{mode}]: {chunk}")  
except Exception as e:  
    print(f"错误: {e}")  
    print("说明: 在 Graph API 中, 需要特殊配置才能使用自定义流模式")  
  
if __name__ == "__main__":  
    main()
```

2) 从工具内部发送自定义用户定义数据

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
LangGraph 工具中的自定义数据流式传输演示
展示如何从工具内部发送自定义用户定义数据
"""

from typing import TypedDict
from langchain.tools import tool
from langgraph.config import get_stream_writer
from langgraph.graph import StateGraph, START, END

@tool
def query_database(query: str) -> str:
    """查询数据库工具"""
    # 访问流写入器以发送自定义数据
    writer = get_stream_writer()

    # 发送自定义数据（例如，进度更新）
    writer({"data": "开始查询数据库", "type": "info"})
    writer({"data": "Retrieved 0/100 records", "type": "progress"})

    # 模拟执行查询
    # 发送更多自定义数据
    writer({"data": "Retrieved 50/100 records", "type": "progress"})
    writer({"data": "Retrieved 100/100 records", "type": "progress"})
    writer({"data": "查询完成", "type": "info"})

    return f"查询'{query}'的结果：找到 25 条匹配记录"

class GraphState(TypedDict):
    query: str
    result: str

def create_graph_with_tool():
    """创建使用工具的图"""

    def tool_node(state: GraphState) -> GraphState:
```

```
        """工具节点"""
        # 直接在节点中使用工具
        result = query_database.invoke(state["query"])
        return {"result": result}

    # 构建图
    builder = StateGraph(GraphState)
    builder.add_node("tool_node", tool_node)
    builder.add_edge(START, "tool_node")
    builder.add_edge("tool_node", END)

    return builder.compile()

def graph_api_demo():
    """Graph API 演示"""
    print("\n" + "=" * 60 + "\n")
    print("=== LangGraph Graph API 中的自定义数据流式传输演示 ===\n")

    # 创建图
    graph = create_graph_with_tool()

    inputs = {"query": "产品信息", "result": ""}

    print("---- 从工具中流式传输自定义数据 ----")
    try:
        # 设置 stream_mode="custom" 以在流中接收自定义数据
        for mode, chunk in graph.stream(inputs, stream_mode=["custom",
"updates"]):
            if mode == "custom":
                print(f" [自定义数据] {chunk}")
            elif mode == "updates":
                print(f" [状态更新] {chunk}")
    except Exception as e:
        print(f"错误: {e}")

def main_demo():
    """主演示函数"""
    graph_api_demo()

    print("\n" + "=" * 60)
    print("说明:")
    print("1. 自定义数据流允许从节点或工具内部发送用户定义的数据")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
print("2. 使用 get_stream_writer() 获取流写入器")
print("3. 可以发送进度更新、日志信息等任何自定义数据")
print("4. 在流式传输时设置 stream_mode='custom' 或包含 'custom' 的模式列表")
if __name__ == "__main__":
    main_demo()
```

4.4 中断 (Interrupts)

中断允许在特定点暂停图的执行，并在继续之前等待外部输入。这支持了“人在回路”模式，即需要外部输入才能继续的场景。当中断被触发时，LangGraph 会利用其持久化层保存图的状态，并无限期等待，直到恢复执行。

通过在图形节点中的任意位置调用 `interrupt()` 函数，即可实现中断功能。该函数接受任何可序列化为 JSON 的值，并将其提供给调用者。

当需要从中断点继续时，只需调用 `Command` 即可。该命令的 `resume` 参数将会作为 `interrupt()` 函数的返回值，`graph` 随之从当前点继续往下执行。

与静态断点（在特定节点之前或之后暂停，调用 `compile()` 时，所传递的 `interrupt_before` / `interrupt_after` 参数）不同，中断是动态的——它们可以放在代码中的任何位置，并且可以根据应用程序逻辑设置为条件性的。

4.4.1 使用 `interrupt` 暂停

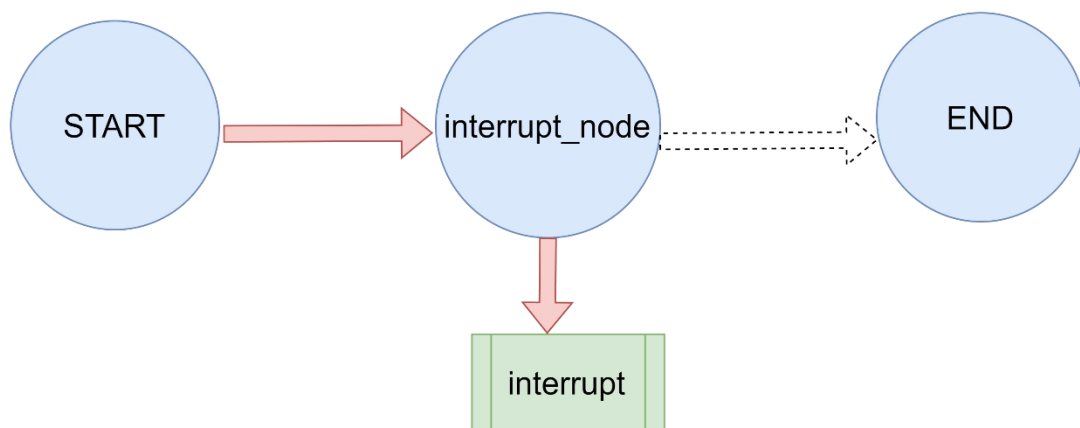
`interrupt` 函数会暂停图的执行，并向调用者返回一个值。当在节点内调用 `interrupt` 时，LangGraph 会保存当前的图状态，并等待用户通过输入来恢复执行。

使用 `interrupt` 时，需要使用 `checkpoint` 来保存图状态，`langgraph` 会将图状态保存到特定的 `thread ID` 当中。

当调用 `interrupt` 时，会发生以下情况：

- 图执行会在调用 `interrupt` 的确切位置暂停。
- 保存状态，以便之后可以恢复执行。
- 值会被返回给处于 `__interrupt__` 状态下的调用方；该值可以是任何可序列化为 JSON 的值（字符串、对象、数组等）。
- 图会无限期等待，直到你通过响应恢复执行
- 当恢复时，响应会传递回节点，成为 `interrupt()` 调用的返回值。

代码示例如下：



```
from typing import TypedDict, Annotated
from langgraph.types import interrupt
from langgraph.graph import StateGraph
from langgraph.constants import START
from langgraph.checkpoint.memory import MemorySaver

class MyState(TypedDict):
    state_1: str
    state_2: Annotated[list, lambda x, y: x+y]

def node_1(state: MyState):

    print("entering node_1")
    res = interrupt(
        {
            "key_1": "value_1",
            "key_2": "value_2"
        }
    )
    return {"state_2": res}

graph = StateGraph(MyState)
graph.add_node(node_1)
graph.add_edge(START, "node_1")
checkpointer = MemorySaver()
graph = graph.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": 1}}
invoke_result = graph.invoke(
    {
        "state_1": "test",
        "state_2": ["1"]
    },
```

```
config=config
)
# 打印结果: [Interrupt(value={'key_1': 'value_1', 'key_2': 'value_2'},
id='d6cb4b6d0bc74b831f81861a50187c87')]
print(invoke_result['__interrupt__'])
```

4.4.2 恢复中断

当中断暂停执行后，可以通过再次调用图并传入包含恢复值的 `Command` 来恢复图的运行。恢复值会被传回 `interrupt` 调用，使节点能够利用外部输入继续执行。

关于恢复的要点：

- 恢复时必须使用与中断发生时相同的**线程 ID**。
- 传递给 `Command(resume=...)` 的值会成为 `interrupt` 调用的返回值。
- 节点在恢复时会从调用 `interrupt` 的节点开头重新启动，因此 `interrupt` 之前的所有代码会再次运行。
- 可以将任何可 JSON 序列化的值作为恢复值传递。

恢复 4.4.1 节中断的图的执行：

```
...

from langgraph.types import Command
#打印结果: {'state_1': 'test', 'state_2': ['1', 'the value returned to
interrupt invoke']}
graph.invoke(Command(resume=["the value returned to interrupt
invoke"]),config=config)
```

4.4.3 常见用法

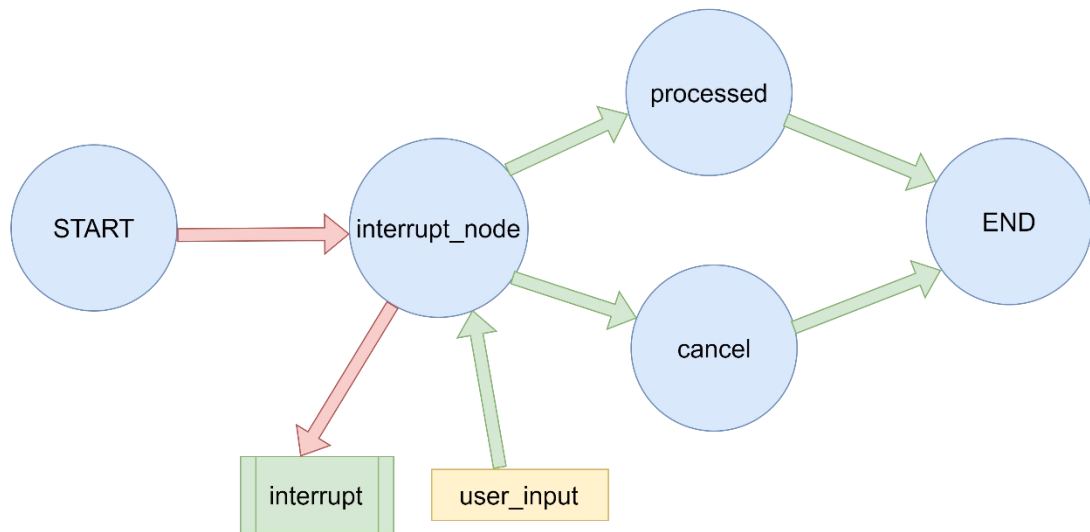
中断的关键在于能够暂停执行并等待外部输入。这在多种使用场景中都很有用，包括：

- 审批工作流：在执行关键操作（例如 API 调用、数据库更改、财务交易等风险等级较高的操作）前暂停。
- 审阅和编辑：让人类在继续之前审阅并修改大模型的输出或工具调用。
- 中断工具调用：在执行工具调用前暂停，以便在执行前检查和编辑工具调用。
- 验证人工输入：在进入下一步之前暂停，以验证人工输入。

4.4.3.1 审批工作流

中断最常见的用途之一是在执行关键操作前暂停并请求批准。例如，希望让用户批准一项 API 调用、一次数据库更改或其他任何高风险操作。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)



```

"""
LangGraph 审批工作流演示

该演示展示了如何使用 LangGraph 的中断功能实现需要人工审批的工作流。
当工作流遇到关键操作时会暂停，并等待用户的批准或拒绝。
"""

import sqlite3
from typing import Literal, Optional, TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class ApprovalState(TypedDict):
    """审批状态定义"""
    action_details: str
    status: Optional[Literal["pending", "approved", "rejected"]]

def approval_node(state: ApprovalState) -> Command[Literal["proceed", "cancel"]]:
    """
    审批节点

    Args:
        state: 当前状态
    """

```

```
Returns:
    Command: 包含中断信息和后续路由指令的命令对象
    """

print(f"执行节点: approval_node")
print(f"操作详情: {state['action_details']}")
print(" workflow暂停, 等待用户审批...")

# 中断执行并暴露详细信息供调用方在 UI 中渲染
decision = interrupt({
    "question": "批准此操作吗?",
    "details": state["action_details"],
})

# 根据恢复值路由到适当的节点
next_node = "proceed" if decision else "cancel"
print(f"审批决定: {'批准' if decision else '拒绝'}, 路由到节点:
{next_node}")

return Command(goto=next_node)

def proceed_node(state: ApprovalState):
    """
    执行节点 - 当审批被批准时执行

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: proceed_node")
    print("操作已被批准, 正在执行...")
    return {"status": "approved"}

def cancel_node(state: ApprovalState):
    """
    取消节点 - 当审批被拒绝时执行
```

```
Args:
    state: 当前状态

Returns:
    dict: 更新后的状态
"""
print("执行节点: cancel_node")
print("操作已被拒绝, 正在取消...")
return {"status": "rejected"}

def main():
    """主函数 - 演示审批工作流"""
    print("=== LangGraph 审批工作流演示 ===\n")

    # 创建状态图
    builder = StateGraph(ApprovalState)
    builder.add_node("approval", approval_node)
    builder.add_node("proceed", proceed_node)
    builder.add_node("cancel", cancel_node)
    builder.add_edge(START, "approval")

    # 注意: 这里我们不直接连接 approval 到 proceed 和 cancel
    # 而是通过 Command(goto=...) 在 approval_node 中动态决定

    builder.add_edge("proceed", END)
    builder.add_edge("cancel", END)

    # 使用内存保存器作为检查点
    checkpointer = MemorySaver()

    # 编译图
    graph = builder.compile(checkpointer=checkpointer)

    # 配置线程 ID
    config = {"configurable": {"thread_id": "approval-123"}}

    # 初始化状态并执行图
    print("1. 启动审批工作流...")
    initial = graph.invoke(
        {"action_details": "转账 $500", "status": "pending"},
```

```
        config=config,
    )

    # 显示中断信息
    print(f" workflow中断信息: {initial['__interrupt__']}\n")

    # 模拟用户审批过程
    print("2. 模拟用户审批过程...")
    interrupt_value = initial["__interrupt__"][0].value
    print("操作详情:", interrupt_value["details"])
    print("问题:", interrupt_value["question"])

    # 获取用户输入
    while True:
        user_input = input("\n 请输入审批决定 (y/n): ").strip().lower()
        if user_input in ['y', 'yes', '是']:
            decision = True
            break
        elif user_input in ['n', 'no', '否']:
            decision = False
            break
        else:
            print("无效输入, 请输入 y/yes/是 或 n/no/否")

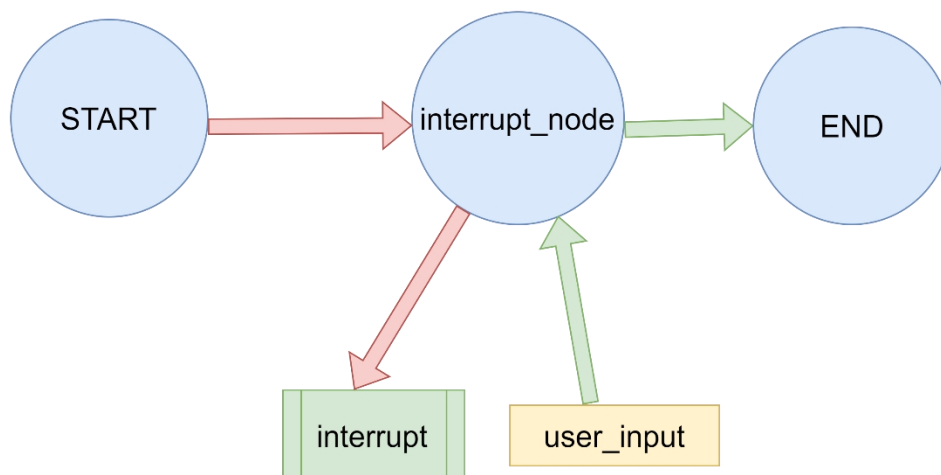
    # 使用用户决定恢复执行
    print(f"\n3. 使用审批决定恢复 workflow 执行...")
    resumed = graph.invoke(Command(resume=decision), config=config)

    # 显示最终结果
    print(f"最终状态: {resumed}")
    print(f"操作状态: {resumed['status']}")
    print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.4.3.2 审阅和编辑

有时希望让人类在继续之前审核并编辑部分图状态。这在纠正大语言模型、补充缺失信息或进行调整等场景下很有用。



"""

LangGraph 审阅和编辑工作流演示

该演示展示了如何使用 **LangGraph** 的中断功能实现人工审阅和编辑工作流。这对于让人类在继续之前审核并编辑图状态非常有用。

"""

```

import sqlite3
from typing import TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt
    
```

```

class ReviewState(TypedDict):
    """审阅状态定义"""
    generated_text: str

def review_node(state: ReviewState):
    """
    审阅节点

    Args:
        state: 当前状态，包含生成的文本内容

    Returns:
        dict: 更新后的状态
    """
    
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
"""

print(f"执行节点: review_node")
print(f"当前文本内容: {state['generated_text']}")
print(" workflows 暂停, 等待用户审阅和编辑...")

# 请求审阅者编辑生成的内容
updated = interrupt({
    "instruction": "请审阅并编辑以下内容",
    "content": state["generated_text"],
})

print(f"收到编辑后的内容: {updated}")
return {"generated_text": updated}

def main():
    """主函数 - 演示审阅和编辑工作流"""
    print("=== LangGraph 审阅和编辑工作流演示 ===\n")

    # 创建状态图
    builder = StateGraph(ReviewState)
    builder.add_node("review", review_node)
    builder.add_edge(START, "review")
    builder.add_edge("review", END)

    # 使用内存保存器作为检查点
    checkpointer = MemorySaver()

    # 编译图
    graph = builder.compile(checkpointer=checkpointer)

    # 配置线程 ID
    config = {"configurable": {"thread_id": "review-42"}}

    # 初始化状态并执行图
    print("1. 启动审阅工作流...")
    initial = graph.invoke({"generated_text": "这是初始草稿内容"},
config=config)

    # 显示中断信息
    print(f" workflows 中断信息: {initial['__interrupt__']}\n")
```

```
# 模拟用户审阅和编辑过程
print("2. 模拟用户审阅和编辑过程...")
interrupt_value = initial["__interrupt__"][0].value
print("指导说明:", interrupt_value["instruction"])
print("原文内容:", interrupt_value["content"])

# 获取用户编辑后的内容
edited_text = input("\n 请输入编辑后的内容: ").strip()

# 使用用户编辑后的内容恢复执行
print(f"\n3. 使用编辑后的内容恢复 workflow 执行...")
final_state = graph.invoke(
    Command(resume=edited_text),
    config=config,
)

# 显示最终结果
print(f"最终状态: {final_state}")
print(f"最终文本内容: {final_state['generated_text']}")
print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.4.3.3 工具中的中断

将中断直接放在工具函数内部。这会使工具在每次被调用时暂停以等待批准，并允许在执行工具调用之前进行人工检查和编辑。

```
"""
LangGraph 工具中断演示
```

该演示展示了如何在工具函数内部使用中断功能，
使工具在每次被调用时暂停以等待批准，并允许在执行前进行人工检查和编辑。

```
"""
```

```
import sqlite3
from typing import TypedDict

from langchain.tools import tool
from langgraph.checkpoint.memory import MemorySaver
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class AgentState(TypedDict):
    """代理状态定义"""
    messages: list[dict]

@tool
def send_email(to: str, subject: str, body: str):
    """
    发送邮件给收件人。

    Args:
        to (str): 收件人邮箱地址
        subject (str): 邮件主题
        body (str): 邮件正文

    Returns:
        str: 发送结果信息
    """
    print(f"执行工具: send_email")
    print(f"收件人: {to}")
    print(f"主题: {subject}")
    print(f"正文: {body}")

    # 在发送前暂停；有效载荷会出现在 result["__interrupt__"] 中
    response = interrupt({
        "action": "send_email",
        "to": to,
        "subject": subject,
        "body": body,
        "message": "是否批准发送此邮件？",
    })

    if response.get("action") == "approve":
        final_to = response.get("to", to)
        final_subject = response.get("subject", subject)
        final_body = response.get("body", body)
```



```
# 实际发送邮件（这里只是模拟）
print(f"[send_email] to={final_to} subject={final_subject}
body={final_body}")
return f"邮件已发送至 {final_to}"

return "用户取消了邮件发送"

def agent_node(state: AgentState):
    """
    代理节点函数

    Args:
        state: 当前状态，包含消息历史

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: agent_node")

    # 模拟 LLM 决定调用工具
    # 在实际应用中，这里会使用 LLM 来决定是否调用工具
    if len(state["messages"]) == 1: # 第一次调用
        # 模拟 LLM 决定调用 send_email 工具
        tool_call = {
            "name": "send_email",
            "arguments": {
                "to": "alice@example.com",
                "subject": "会议安排",
                "body": "你好，我想安排一个会议讨论项目进展。"
            }
        }

    # 调用工具（这会触发中断）
    try:
        result = send_email.invoke(tool_call["arguments"])
        return {
            "messages": state["messages"] + [
                {"role": "assistant", "content": f"调用工具:
{tool_call['name']}"}],
```

```
        {"role": "tool", "name": tool_call["name"], "content":
result}
    ]
}
except Exception as e:
    # 捕获中断异常, 让工作流暂停
    raise e
else:
    # 后续调用, 返回最终结果
    return {"messages": state["messages"]}]

def main():
    """主函数 - 演示工具中断功能"""
    print("=== LangGraph 工具中断演示 ===\n")

    # 创建状态图
    builder = StateGraph(AgentState)
    builder.add_node("agent", agent_node)
    builder.add_edge(START, "agent")
    builder.add_edge("agent", END)

    # 使用内存保存器作为检查点
    checkpointer = MemorySaver()

    # 编译图
    graph = builder.compile(checkpointer=checkpointer)

    # 配置线程 ID
    config = {"configurable": {"thread_id": "email-workflow"}}

    # 初始化状态并执行图
    print("1. 启动邮件发送工作流...")
    try:
        initial = graph.invoke(
            {
                "messages": [
                    {"role": "user", "content": "请发送邮件给
alice@example.com 关于会议安排"}
                ]
            },
```

```
        config=config,
    )
    print(f" workflow中断信息: {initial['__interrupt__']}\n")

    # 模拟用户审批过程
    print("2. 模拟用户审批过程...")
    interrupt_value = initial["__interrupt__"][0].value
    print("操作:", interrupt_value["action"])
    print("消息:", interrupt_value["message"])
    print("收件人:", interrupt_value["to"])
    print("主题:", interrupt_value["subject"])
    print("正文:", interrupt_value["body"])

    # 获取用户输入
    while True:
        user_input = input("\n 是否批准发送邮件? (y/n): ")
        user_input = user_input.strip().lower()
        if user_input in ['y', 'yes', '是']:
            # 用户批准, 可以编辑参数
            new_subject = input("请输入新主题 (直接回车保持原主题): ")
            new_subject = new_subject.strip()
            if not new_subject:
                approval_response = {"action": "approve"}
            else:
                approval_response = {"action": "approve", "subject":
new_subject}
            break
        elif user_input in ['n', 'no', '否']:
            approval_response = {"action": "reject"}
            break
        else:
            print("无效输入, 请输入 y/yes/是 或 n/no/否")

    # 使用用户决定恢复执行
    print(f"\n3. 使用审批决定恢复 workflow 执行...")
    resumed = graph.invoke(
        Command(resume=approval_response),
        config=config,
    )

    # 显示最终结果
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
print(f"最终消息: {resumed['messages'][-1]}")
print("\n=== 演示完成 ===")

except Exception as e:
    print(f"执行过程中出现错误: {e}")
    print("\n=== 演示结束 ===")

if __name__ == "__main__":
    main()
```

4.4.3.4 验证人工输入

验证来自人类的输入，如果输入无效，就再次询问。通过在循环中多次调用 `interrupt` 来实现这一点。

```
"""
LangGraph 高级表单验证演示

该演示展示了如何验证来自人类的多个输入字段，
如果输入无效，就再次询问。通过在循环中多次调用 interrupt 来实现这一点。
"""

import sqlite3
from typing import TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class AdvancedFormState(TypedDict):
    """高级表单状态定义"""
    name: str | None
    age: int | None
    email: str | None

def get_name_node(state: AdvancedFormState):
    """
    获取姓名节点函数

    Args:
    """
```

```
state: 当前状态

Returns:
    dict: 更新后的状态
"""
print("执行节点: get_name_node")

prompt = "请输入您的姓名:"

while True:
    answer = interrupt(prompt)
    print(f"收到用户输入: {answer}")

    # 验证输入是否为非空字符串
    if isinstance(answer, str) and len(answer.strip()) > 0:
        name = answer.strip()
        print(f"姓名验证通过: {name}")
        return {"name": name}

    prompt = "姓名不能为空, 请重新输入您的姓名:"

def get_age_node(state: AdvancedFormState):
    """
    获取年龄节点函数

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: get_age_node")

    prompt = f"您好 {state['name']}, 请输入您的年龄:"

    while True:
        answer = interrupt(prompt)
        print(f"收到用户输入: {answer}")

        # 验证输入是否为正整数
```

```
if isinstance(answer, int) and answer > 0:
    print(f"年龄验证通过: {answer}")
    return {"age": answer}

# 尝试将字符串转换为整数
if isinstance(answer, str):
    try:
        age_int = int(answer)
        if age_int > 0:
            print(f"年龄验证通过: {age_int}")
            return {"age": age_int}
    except ValueError:
        pass

prompt = f"'{answer}' 不是有效的年龄。请输入一个正整数:"

def get_email_node(state: AdvancedFormState):
    """
    获取邮箱节点函数

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: get_email_node")

    prompt = f"您好 {state['name']} ({state['age']}岁), 请输入您的邮箱地址:"

    while True:
        answer = interrupt(prompt)
        print(f"收到用户输入: {answer}")

        # 简单验证邮箱格式
        if isinstance(answer, str) and "@" in answer and "." in answer:
            email = answer.strip()
            print(f"邮箱验证通过: {email}")
            return {"email": email}
```

```
prompt = f'"{answer}" 不是有效的邮箱地址。请输入正确的邮箱格式\n(example@domain.com):'

def main():
    """主函数 - 演示高级表单验证功能"""
    print("=== LangGraph 高级表单验证演示 ===\n")

    # 创建状态图
    builder = StateGraph(AdvancedFormState)
    builder.add_node("collect_name", get_name_node)
    builder.add_node("collect_age", get_age_node)
    builder.add_node("collect_email", get_email_node)

    builder.add_edge(START, "collect_name")
    builder.add_edge("collect_name", "collect_age")
    builder.add_edge("collect_age", "collect_email")
    builder.add_edge("collect_email", END)

    # 使用内存保存器作为检查点
    checkpointer = MemorySaver()

    # 编译图
    graph = builder.compile(checkpointer=checkpointer)

    # 配置线程 ID
    config = {"configurable": {"thread_id": "advanced-form-1"}}

    # 初始化状态并执行图
    print("1. 启动高级表单收集工作流...")
    try:
        # 收集姓名
        state_after_name = graph.invoke({"name": None, "age": None, "email":
None}, config=config)
        print(f"工作流中断信息: {state_after_name['__interrupt__']}\n")

        # 输入有效姓名
        print("2. 输入有效姓名...")
        state_after_name = graph.invoke(Command(resume="张三"),
config=config)
        print(f"工作流中断信息: {state_after_name['__interrupt__']}\n")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
# 输入无效年龄
print("3. 输入无效年龄...")
state_after_invalid_age = graph.invoke(Command(resume="二十"),
config=config)
print(f" workflow中断信息:
{state_after_invalid_age['__interrupt__']}\n")

# 输入有效年龄
print("4. 输入有效年龄...")
state_after_age = graph.invoke(Command(resume=25), config=config)
print(f" workflow中断信息: {state_after_age['__interrupt__']}\n")

# 输入无效邮箱
print("5. 输入无效邮箱...")
state_after_invalid_email =
graph.invoke(Command(resume="zhangsan"), config=config)
print(f" workflow中断信息:
{state_after_invalid_email['__interrupt__']}\n")

# 输入有效邮箱
print("6. 输入有效邮箱...")
final_state = graph.invoke(Command(resume="zhangsan@example.com"),
config=config)
print(f"最终状态: {final_state}")
print(f"姓名: {final_state['name']}")
print(f"年龄: {final_state['age']}")
print(f"邮箱: {final_state['email']}")
print("\n=== 演示完成 ===")

except Exception as e:
    print(f"执行过程中出现错误: {e}")
    print("\n=== 演示结束 ===")

if __name__ == "__main__":
    main()
```

4.4.4 中断规则

在节点内调用 `interrupt` 时，LangGraph 会通过抛出一个异常来暂停执行，该异常会通知运行时暂停。此异常会沿调用栈向上传播，并被运行时捕获，运行时随后会通知图保存当前状态并等待外部输入。

当执行恢复时（在提供请求的输入后），运行时会从头重新启动整个节点——它不会从调用 `interrupt` 的确切行继续。这意味着在 `interrupt` 之前运行的任何代码都会再次执行。因此，在使用中断时需要遵循一些重要规则，以确保它们按预期运行。

- （1）不要将 `interrupt` 调用包裹在 `try/except` 中。
- （2）不要在节点内重新排序 `interrupt` 调用。
- （3）不要在 `interrupt` 调用中返回复杂值。
- （4）在 `interrupt` 前调用的副作用必须具有幂等性。

4.5 时间旅行（Time travel）

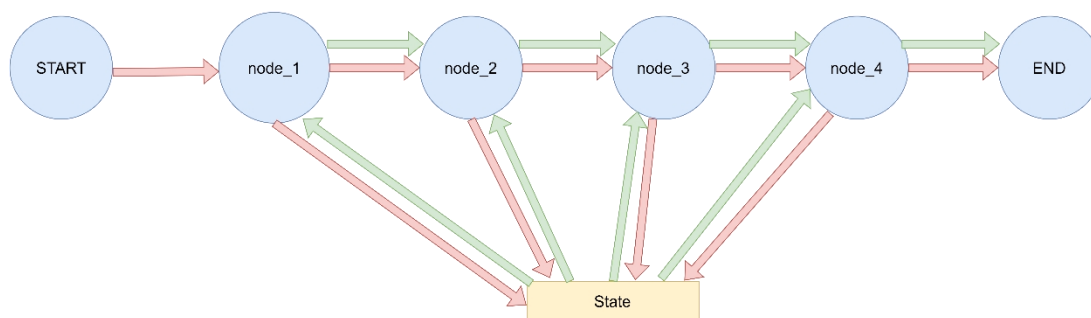
在处理基于模型做决策的非确定性系统（例如由大语言模型驱动的智能体）时，详细检查它们的决策过程可能会很有用：

- （1）理解推理过程：分析达成成功结果的各个步骤。
- （2）调试错误：确定错误发生的位置和原因。
- （3）探索替代方案：测试不同的路径以发现更好的解决方案。

LangGraph 提供了时间回溯功能来支持这些使用场景。具体来说，可以从之前的检查点恢复执行——要么重放相同的状态，要么对其进行修改以探索其他可能性。在所有情况下，恢复过去的执行都会在历史记录中产生一个新的分支。

要在 LangGraph 中使用时间旅行：

- （1）使用 `invoke` 或 `stream` 方法，以初始输入来运行图表。
- （2）识别现有线程中的检查点：使用 `get_state_history` 方法检索特定 `thread_id` 的执行历史，并找到所需的 `checkpoint_id`。或者，在希望执行暂停的节点之前设置一个 `interrupt`。然后，你可以找到截至该中断记录的最新检查点。
- （3）更新图状态（可选）：使用 `update_state` 方法在检查点修改图的状态，并从替代状态恢复执行。
- （4）从检查点恢复执行：使用 `invoke` 或 `stream` 方法，输入为 `None`，配置中包含适当的 `thread_id` 和检查点 ID



"""

LangGraph 高级时间旅行演示

该演示展示了更复杂的时间旅行功能，包括：

1. 运行图并生成多个状态
2. 查看历史状态
3. 从不同历史点恢复执行
4. 比较不同执行路径的结果

"""

```
import uuid
from typing_extensions import TypedDict, NotRequired

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
```

```
class StoryState(TypedDict):
```

"""故事状态定义"""

character: NotRequired[str]

setting: NotRequired[str]

plot: NotRequired[str]

ending: NotRequired[str]

```
def create_character(state: StoryState):
```

"""

创建故事角色

Args:

state: 当前状态

Returns:

dict: 更新后的状态

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
"""
print("执行节点: create_character")

# 模拟 LLM 调用
mock_character = "一只会说话的猫"
print(f"创建的角色: {mock_character}")
return {"character": mock_character}

def set_setting(state: StoryState):
    """
    设置故事背景

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: set_setting")

    # 模拟 LLM 调用
    mock_setting = "在一个神秘的图书馆里"
    print(f"设置的背景: {mock_setting}")
    return {"setting": mock_setting}

def develop_plot(state: StoryState):
    """
    发展故事情节

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: develop_plot")

    # 模拟 LLM 调用
    character = state.get("character", "未知角色")
```

```
    setting = state.get("setting", "未知背景")
    mock_plot = f"{character}在{setting}发现了一本会发光的书"
    print(f"发展的剧情: {mock_plot}")
    return {"plot": mock_plot}

def write_ending(state: StoryState):
    """
    编写故事结局

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: write_ending")

    # 模拟 LLM 调用
    plot = state.get("plot", "未知剧情")
    mock_ending = f"当{plot}时, 整个图书馆都被魔法光芒照亮了"
    print(f"编写的结局: {mock_ending}")
    return {"ending": mock_ending}

def main():
    """主函数 - 演示高级时间旅行功能"""
    print("=== LangGraph 高级时间旅行演示 ===\n")

    # 构建工作流
    workflow = StateGraph(StoryState)

    # 添加节点
    workflow.add_node("create_character", create_character)
    workflow.add_node("set_setting", set_setting)
    workflow.add_node("develop_plot", develop_plot)
    workflow.add_node("write_ending", write_ending)

    # 添加边来连接节点
    workflow.add_edge(START, "create_character")
    workflow.add_edge("create_character", "set_setting")
```

```
workflow.add_edge("set_setting", "develop_plot")
workflow.add_edge("develop_plot", "write_ending")
workflow.add_edge("write_ending", END)

# 编译
checkpointer = MemorySaver()
graph = workflow.compile(checkpointer=checkpointer)

# 1. 运行图表生成第一个故事
print("1. 生成第一个故事...")
config1 = {
    "configurable": {
        "thread_id": str(uuid.uuid4()),
    }
}

story1 = graph.invoke({}, config1)
print(f"角色: {story1['character']}")
print(f"背景: {story1['setting']}")
print(f"剧情: {story1['plot']}")
print(f"结局: {story1['ending']}")
print()

# 2. 查看历史状态
print("2. 查看第一个故事的历史状态...")
states1 = list(graph.get_state_history(config1))

print("历史状态:")
for i, state in enumerate(states1):
    print(f" {i}. 下一步节点: {state.next}")
    print(f"    检查点 ID: {state.config['configurable']['checkpoint_id']}")
    if state.values:
        print(f"    状态值: {state.values}")
    print()

# 3. 从中间状态恢复执行, 创建第二个故事
print("3. 从中间状态恢复执行, 创建第二个故事...")

# 选择 create_character 执行后的状态
character_state = states1[2] # 索引 2 对应 create_character 执行后的状态
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网

```
print(f"选中的状态: {character_state.next}")
print(f"选中的状态值: {character_state.values}")

# 更新状态, 改变角色
new_config = graph.update_state(
    character_state.config,
    values={"character": "一只会飞的龙"}
)
print(f"新配置: {new_config}")
print()

# 4. 从新检查点恢复执行
print("4. 从新检查点恢复执行, 生成第二个故事...")
story2 = graph.invoke(None, new_config)
print(f"新角色: {story2['character']}")
print(f"背景: {story2['setting']}")
print(f"剧情: {story2['plot']}")
print(f"结局: {story2['ending']}")
print()

# 5. 比较两个故事
print("5. 比较两个故事:")
print("  故事 1:")
print(f"    角色: {story1['character']}")
print(f"    背景: {story1['setting']}")
print(f"    剧情: {story1['plot']}")
print(f"    结局: {story1['ending']}")
print()

print("  故事 2:")
print(f"    角色: {story2['character']}")
print(f"    背景: {story2['setting']}")
print(f"    剧情: {story2['plot']}")
print(f"    结局: {story2['ending']}")
print()

print("=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.6 记忆（Memory）

人工智能应用需要内存来在多次交互中共享上下文。在 LangGraph 中，你可以添加两种类型的内存：

- （1）将短期记忆（**Checkpoint**）作为智能体状态的一部分添加，以实现多轮对话。
 - 作用域：单个 thread（对话线程）
 - 存储内容：对话历史、状态数据
 - 生命周期：仅在当前 thread 内有效
 - 实现方式：通过 checkpointer 持久化 state
- （2）添加长期记忆（**Store**）以跨会话存储用户特定数据或应用程序级数据。
 - 作用域：跨 thread，可在任何对话中访问
 - 存储内容：用户偏好、知识库等跨会话数据
 - 生命周期：跨会话持久化
 - 实现方式：通过 store 存储，支持 namespace 组织

4.6.1 添加短期记忆

短期记忆（线程级持久性）使智能体能够跟踪多轮对话。

添加短期记忆就是制定检查点存储，并且指定线程 id，按 thread_id 隔离（4.1.3 和 4.1.4 的案例就是属于短期记忆的实现）。

如果图包含子图，只需在编译父图时提供检查点工具即可。LangGraph 会自动将检查点工具传播到子图中。

以下是新的案例：

1) 案例一：使用内存检查点

```
"""
```

LangGraph 短期记忆演示

该演示展示了如何使用短期记忆（线程级持久性）使智能体能够跟踪多轮对话。

```
"""
```

```
from typing import Annotated
from typing_extensions import TypedDict
import operator
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import StateGraph, START, END

# 定义状态，包含消息历史
class ChatState(TypedDict):
    """聊天状态定义"""
    messages: Annotated[list, operator.add]
    user_name: str

def greeting_node(state: ChatState) -> dict:
    """
    问候节点

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: greeting_node")

    user_name = state.get("user_name", "访客")
    greeting_message = f"你好, {user_name}! 我是你的 AI 助手。"

    return {
        "messages": [("assistant", greeting_message)]
    }

def respond_node(state: ChatState) -> dict:
    """
    回应节点

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
```



```
"""

print("执行节点: respond_node")

# 获取最新的用户消息
user_messages = [msg for msg in state["messages"] if msg[0] == "user"]
if user_messages:
    latest_user_message = user_messages[-1][1]
    user_name = state.get("user_name", "访客")

    # 根据用户消息生成回应
    if "你好" in latest_user_message or "hello" in latest_user_message.lower():
        response = f"你好, {user_name}! 有什么我可以帮助你的吗?"
    elif "天气" in latest_user_message:
        response = f"抱歉, {user_name}, 我无法获取实时天气信息。"
    elif "名字" in latest_user_message or "我是" in latest_user_message:
        response = f"我知道你叫{user_name}, 很高兴认识你!"
    else:
        response = f"我理解你说的, {user_name}。能告诉我更多吗?"
else:
    response = "我没有看到你的消息, 请再说一遍。"

return {
    "messages": [("assistant", response)]
}

def main():
    """主函数 - 演示短期记忆功能"""
    print("=== LangGraph 短期记忆演示 ===\n")

    # 创建内存存储器（短期记忆）
    memory = InMemorySaver()

    # 构建图
    builder = StateGraph(ChatState)
    builder.add_node("greeting", greeting_node)
    builder.add_node("respond", respond_node)

    builder.add_edge(START, "greeting")
    builder.add_edge("greeting", "respond")
```

```
builder.add_edge("respond", END)

# 编译图并使用内存存储
graph = builder.compile(checkpointer=memory)

# 配置线程 ID 用于存储状态
config = {"configurable": {"thread_id": "chat_1"}}

# 第一轮对话
print("1. 第一轮对话:")
result1 = graph.invoke({
    "messages": [("user", "你好! 我叫张三)],
    "user_name": "张三"
}, config)

print("对话历史:")
for role, message in result1["messages"]:
    print(f" {role}: {message}")
print()

# 查看存储的状态
print("2. 检查存储的状态:")
saved_state = graph.get_state(config)
print("保存的对话历史:")
for role, message in saved_state.values["messages"]:
    print(f" {role}: {message}")
print()

# 第二轮对话（继续之前的对话）
print("3. 第二轮对话（继续之前的对话）:")
result2 = graph.invoke({
    "messages": [("user", "今天天气怎么样? ")],
    "user_name": "张三"
}, config)

print("对话历史:")
for role, message in result2["messages"]:
    print(f" {role}: {message}")
print()

# 第三轮对话
```

```
print("4. 第三轮对话:")
result3 = graph.invoke({
    "messages": [("user", "你能记住我的名字吗? ")],
    "user_name": "张三"
}, config)

print("对话历史:")
for role, message in result3["messages"]:
    print(f" {role}: {message}")
print()

# 查看最终状态
print("5. 最终状态:")
final_state = graph.get_state(config)
print("完整的对话历史:")
for role, message in final_state.values["messages"]:
    print(f" {role}: {message}")
print()

print("=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

2) 案例二：使用数据库检查点

"""

LangGraph SQLite 短期记忆演示

该演示展示了如何在生产环境中使用 SQLite 数据库作为检查点存储，
使智能体能够跟踪多轮对话。

"""

```
import sqlite3
from typing import Annotated
from typing_extensions import TypedDict
import operator

from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import StateGraph, START, END
```

```
# 定义状态，包含消息历史
class ChatState(TypedDict):
    """聊天状态定义"""
    messages: Annotated[list, operator.add]
    user_name: str

def greeting_node(state: ChatState) -> dict:
    """
    问候节点

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: greeting_node")

    user_name = state.get("user_name", "访客")
    greeting_message = f"你好, {user_name}! 我是你的 AI 助手。"

    return {
        "messages": [("assistant", greeting_message)]
    }

def respond_node(state: ChatState) -> dict:
    """
    回应节点

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: respond_node")

    # 获取最新的用户消息
    user_messages = [msg for msg in state["messages"] if msg[0] == "user"]
```

```
if user_messages:
    latest_user_message = user_messages[-1][1]
    user_name = state.get("user_name", "访客")

    # 根据用户消息生成回应
    if "你好" in latest_user_message or "hello" in
latest_user_message.lower():
        response = f"你好, {user_name}! 有什么我可以帮助你的吗? "
    elif "天气" in latest_user_message:
        response = f"抱歉, {user_name}, 我无法获取实时天气信息。"
    elif "名字" in latest_user_message or "我是" in latest_user_message:
        response = f"我知道你叫{user_name}, 很高兴认识你! "
    else:
        response = f"我理解你说的, {user_name}。能告诉我更多吗? "
else:
    response = "我没有看到你的消息, 请再说一遍。"

return {
    "messages": [("assistant", response)]
}

def main():
    """主函数 - 演示 SQLite 短期记忆功能"""
    print("=== LangGraph SQLite 短期记忆演示 ===\n")

    # 创建或连接到 SQLite 数据库
    # 注意: check_same_thread=False 是可以的, 因为实现使用锁来确保线程安全
    conn = sqlite3.connect("../chat_checkpoints.sqlite",
check_same_thread=False)

    # 创建 SqliteSaver 实例
    sqlite_saver = SqliteSaver(conn)

    # 构建图
    builder = StateGraph(ChatState)
    builder.add_node("greeting", greeting_node)
    builder.add_node("respond", respond_node)

    builder.add_edge(START, "greeting")
    builder.add_edge("greeting", "respond")
```

```
builder.add_edge("respond", END)

# 编译图并使用 SQLite 作为检查点存储
graph = builder.compile(checkpointer=sqlite_saver)

# 配置线程 ID 用于存储状态
config = {"configurable": {"thread_id": "sqlite_chat_1"}}

# 第一轮对话
print("1. 第一轮对话:")
result1 = graph.invoke({
    "messages": [("user", "你好! 我叫李四")],
    "user_name": "李四"
}, config)

print("对话历史:")
for role, message in result1["messages"]:
    print(f" {role}: {message}")
print()

# 查看存储的状态
print("2. 检查存储的状态:")
saved_state = graph.get_state(config)
print("保存的对话历史:")
for role, message in saved_state.values["messages"]:
    print(f" {role}: {message}")
print()

# 第二轮对话（继续之前的对话）
print("3. 第二轮对话（继续之前的对话）:")
result2 = graph.invoke({
    "messages": [("user", "今天天气怎么样? ")],
    "user_name": "李四"
}, config)

print("对话历史:")
for role, message in result2["messages"]:
    print(f" {role}: {message}")
print()

# 使用不同的线程 ID
```

```
print("4. 使用不同的线程 ID（新对话）:")
config2 = {"configurable": {"thread_id": "sqlite_chat_2"}}
result3 = graph.invoke({
    "messages": [("user", "你好，我是王五")],
    "user_name": "王五"
}, config2)

print("新对话历史:")
for role, message in result3["messages"]:
    print(f" {role}: {message}")
print()

# 查看不同线程的状态
print("5. 查看不同线程的状态:")
thread1_state = graph.get_state(config)
thread2_state = graph.get_state(config2)

print("线程 1 对话历史:")
for role, message in thread1_state.values["messages"]:
    print(f" {role}: {message}")

print("\n线程 2 对话历史:")
for role, message in thread2_state.values["messages"]:
    print(f" {role}: {message}")
print()

# 关闭数据库连接
conn.close()

print("=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.6.2 添加长期记忆

利用长期记忆跨对话存储用户特定或应用程序特定的数据。

1) 案例一：使用内存存储

```
"""
LangGraph 长期记忆演示
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

该演示展示了如何使用长期记忆跨对话存储用户特定或应用程序特定的数据。

```
"""
```

```
from typing import Annotated
from typing_extensions import TypedDict
from langchain_core.messages import HumanMessage, AIMessage
from langgraph.graph import StateGraph, START
from langgraph.store.memory import InMemoryStore
```

```
# 定义状态
```

```
class ChatState(TypedDict):
    """聊天状态定义"""
    messages: Annotated[list, lambda x, y: x + y]
```

```
def chat_node(state: ChatState, *, store):
```

```
    """
```

```
    聊天节点
```

```
    Args:
```

```
        state: 当前状态
```

```
        store: 存储对象
```

```
    Returns:
```

```
        dict: 更新后的状态
```

```
    """
```

```
    print("执行节点: chat_node")
```

```
# 获取用户 ID（这里我们使用固定 ID 进行演示）
```

```
user_id = "user_123"
```

```
# 从存储中获取用户信息
```

```
try:
```

```
    user_info_item = store.get(("users",), user_id)
```

```
    user_info = user_info_item.value if user_info_item else {}
```

```
    print(f"从存储中获取用户信息: {user_info}")
```

```
except Exception as e:
```

```
    print(f"获取用户信息时出错: {e}")
```

```
    user_info = {}
```



```
# 获取最新的用户消息
user_messages = [msg for msg in state["messages"] if isinstance(msg,
HumanMessage)]
if user_messages:
    latest_message = user_messages[-1].content
    print(f"用户消息: {latest_message}")

# 从消息中提取信息
if "我叫" in latest_message or "我是" in latest_message:
    # 简单提取姓名
    if "我叫" in latest_message:
        name_start = latest_message.find("我叫") + 2
    else:
        name_start = latest_message.find("我是") + 2

    name_end = len(latest_message)
    for i in range(name_start, len(latest_message)):
        if latest_message[i] in [",", " ", ".", "。", "!", "!", " ", "?",
"? "]:
            name_end = i
            break
    name = latest_message[name_start:name_end].strip()
    if name:
        user_info["name"] = name

if "岁" in latest_message:
    # 提取年龄
    age_pos = latest_message.find("岁")
    age_str = ""
    for i in range(age_pos - 1, -1, -1):
        if latest_message[i].isdigit():
            age_str = latest_message[i] + age_str
        else:
            break
    if age_str and age_str.isdigit():
        user_info["age"] = int(age_str)

if "来自" in latest_message:
    # 提取位置
    location_start = latest_message.find("来自") + 2
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
        location_end = len(latest_message)
    for i in range(location_start, len(latest_message)):
        if latest_message[i] in [",", " ", ".", "。", "!", "!", " ", "?",
"? "]:
            location_end = i
            break
    location =
latest_message[location_start:location_end].strip()
    if location:
        user_info["location"] = location

# 保存更新后的用户信息
if user_info:
    try:
        store.put(("users",), user_id, user_info)
        print(f"保存用户信息到存储: {user_info}")
    except Exception as e:
        print(f"保存用户信息时出错: {e}")

# 生成回复
if "你好" in latest_message or "hello" in latest_message.lower():
    if user_info.get("name"):
        response = f"你好, {user_info['name']}! 很高兴再次见到你。"
    else:
        response = "你好! 我是 AI 助手。能告诉我你的名字吗?"
elif "我叫" in latest_message or "我是" in latest_message:
    name = user_info.get("name", "朋友")
    response = f"很高兴认识你, {name}! 有什么我可以帮助你的吗?"
elif "再见" in latest_message or "bye" in latest_message.lower():
    name = user_info.get("name", "朋友")
    response = f"再见, {name}! 期待下次与你交流。"
else:
    # 基于用户信息的个性化回复
    info_parts = []
    if user_info.get("name"):
        info_parts.append(f"名字是{user_info['name']}")
    if user_info.get("age"):
        info_parts.append(f"年龄是{user_info['age']}岁")
    if user_info.get("location"):
        info_parts.append(f"来自{user_info['location']}")
```

```
        if info_parts:
            info_summary = ", 而且我知道你" + ", ".join(info_parts)
            response = f"我理解你的问题。{info_summary}。让我来帮助你解
答。"
        else:
            response = "我理解你的问题。让我来帮助你解答。"
    else:
        response = "我没有收到你的消息，请再说一遍。"

    print(f"生成的回复: {response}")
    return {"messages": [AIMessage(content=response)]}

def main():
    """主函数 - 演示长期记忆功能"""
    print("=== LangGraph 长期记忆演示 ===\n")

    # 创建内存存储
    store = InMemoryStore()

    # 构建图
    builder = StateGraph(ChatState)
    builder.add_node("chat", chat_node)
    builder.add_edge(START, "chat")

    # 编译图并使用存储
    graph = builder.compile(store=store)

    # 第一轮对话
    print("1. 第一轮对话:")
    result1 = graph.invoke({
        "messages": [HumanMessage(content="你好，我叫张三，来自北京。")]
    })

    print("对话历史:")
    for msg in result1["messages"]:
        print(f"    {type(msg).__name__}: {msg.content}")
    print()

    # 第二轮对话
    print("2. 第二轮对话:")
```

```
result2 = graph.invoke({
    "messages": [HumanMessage(content="我今年 25 岁了。")]
})

print("对话历史:")
for msg in result2["messages"]:
    print(f" {type(msg).__name__}: {msg.content}")
print()

# 第三轮对话
print("3. 第三轮对话:")
result3 = graph.invoke({
    "messages": [HumanMessage(content="你好! ")]
})

print("对话历史:")
for msg in result3["messages"]:
    print(f" {type(msg).__name__}: {msg.content}")
print()

# 查看存储的内容
print("4. 查看存储的内容:")
try:
    user_info_item = store.get(("users",), "user_123")
    if user_info_item:
        print(f"存储的用户信息: {user_info_item.value}")
    else:
        print("未找到用户信息")
except Exception as e:
    print(f"查看存储内容时出错: {e}")

print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

2) 案例二：使用数据库存储

"""
LangGraph SQLite 长期记忆演示

该演示展示了如何在生产环境中使用 SQLite 数据库作为长期记忆存储。

更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可百度访问：[尚硅谷官网](#)

参考 PostgreSQL 的正确用法，使用上下文管理器来避免事务冲突。

```
"""

import sqlite3
import uuid
from typing import Annotated
from typing_extensions import TypedDict
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.runnables import RunnableConfig
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.store.sqlite import SqliteStore
from langgraph.store.base import BaseStore

# 定义状态（使用 MessagesState 简化代码）
class ChatState(MessagesState):
    """聊天状态定义 - 继承自 MessagesState"""
    pass

def call_model(
    state: ChatState,
    config: RunnableConfig,
    *,
    store: BaseStore,
):
    """
    调用模型的节点函数

    Args:
        state: 当前状态
        config: 配置信息
        store: 存储对象

    Returns:
        dict: 更新后的状态
    """
    print("执行节点: call_model")

    # 从配置中获取用户 ID
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
user_id = config["configurable"]["user_id"]
namespace = ("memories", user_id)

# 从存储中搜索相关记忆
try:
    memories = store.search(namespace,
query=str(state["messages"][-1].content))
    info = "\n".join([d.value["data"] for d in memories])
    print(f"检索到的记忆: {info}")
except Exception as e:
    print(f"检索记忆时出错: {e}")
    info = ""

system_msg = f"你是一个有帮助的助手，正在与用户交谈。用户信息: {info}" if
info else "你是一个有帮助的助手。"
print(f"系统消息: {system_msg}")

# 检查用户是否要求记住某些信息
last_message = state["messages"][-1]
if "记住" in last_message.content.lower() or "remember" in
last_message.content.lower():
    # 提取需要记住的信息（这里简化处理）
    memory = "用户的名字是张三" if "张三" in last_message.content else "
用户要求记住一些信息"
    try:
        store.put(namespace, str(uuid.uuid4()), {"data": memory})
        print(f"已存储记忆: {memory}")
    except Exception as e:
        print(f"存储记忆时出错: {e}")

# 生成回复（这里使用模拟回复代替实际模型调用）
user_message = last_message.content
if "你好" in user_message or "hello" in user_message.lower():
    response = "你好！我是 AI 助手。有什么我可以帮助你的吗？"
elif "记住" in user_message.lower() or "remember" in
user_message.lower():
    response = "好的，我已经记住了你说的信息。"
elif "名字" in user_message or "name" in user_message.lower():
    if info:
        response = f"根据我的记忆，你的名字是张三。"
    else:
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
        response = "我还不知道你的名字，能告诉我吗？"
    else:
        response = "我理解你的问题。让我来帮助你解答。"

    print(f"生成的回复: {response}")
    return {"messages": [AIMessage(content=response)]}

def main():
    """主函数 - 演示 SQLite 长期记忆功能"""
    print("=== LangGraph SQLite 长期记忆演示 ===\n")

    DB_PATH = "long_term_memory.db"

    # 使用上下文管理器确保正确初始化和清理资源
    with (
        SqliteStore.from_conn_string(DB_PATH) as store,
        SqliteSaver.from_conn_string(DB_PATH) as checkpointer,
    ):
        # 构建图
        builder = StateGraph(ChatState)
        builder.add_node(call_model)
        builder.add_edge(START, "call_model")

        # 编译图，同时使用检查点和存储
        graph = builder.compile(
            checkpointer=checkpointer,
            store=store,
        )

        # 第一次对话 - 要求记住信息
        print("1. 第一次对话 - 要求记住信息:")
        config1 = {
            "configurable": {
                "thread_id": "1",
                "user_id": "user_123",
            }
        }

        for chunk in graph.stream(
```

```
        {"messages": [HumanMessage(content="你好！请记住：我的名字是张三")]}],
        config1,
        stream_mode="values",
    ):
        if chunk["messages"]:
            last_message = chunk["messages"][-1]
            if hasattr(last_message, 'content'):
                print(f" {type(last_message).__name__}: {last_message.content}")
            else:
                print(f" {type(last_message).__name__}: {last_message}")
            print()

    # 第二次对话 - 查询记忆
    print("2. 第二次对话 - 查询记忆:")
    config2 = {
        "configurable": {
            "thread_id": "2",
            "user_id": "user_123",
        }
    }

    for chunk in graph.stream(
        {"messages": [HumanMessage(content="我的名字是什么? ") ]}],
        config2,
        stream_mode="values",
    ):
        if chunk["messages"]:
            last_message = chunk["messages"][-1]
            if hasattr(last_message, 'content'):
                print(f" {type(last_message).__name__}: {last_message.content}")
            else:
                print(f" {type(last_message).__name__}: {last_message}")
            print()

    # 第三次对话 - 不同用户
    print("3. 第三次对话 - 不同用户:")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
config3 = {
    "configurable": {
        "thread_id": "3",
        "user_id": "user_456", # 不同的用户 ID
    }
}

for chunk in graph.stream(
    {"messages": [HumanMessage(content="我的名字是什么? ") ]},
    config3,
    stream_mode="values",
):
    if chunk["messages"]:
        last_message = chunk["messages"][-1]
        if hasattr(last_message, 'content'):
            print(f" {type(last_message).__name__}: {last_message.content}")
        else:
            print(f" {type(last_message).__name__}: {last_message}")
        print()

    print("=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.6.3 管理短期记忆

启用短期记忆后，长对话可能会超出大语言模型（LLM）的上下文窗口。常见的解决方案包括：

- 修剪消息：删除开头或结尾的 N 条消息（在调用大语言模型之前）
- 从 LangGraph 状态中永久删除消息
- 总结消息：总结历史记录中较早的消息，并用摘要替换它们
- 管理检查点以存储和检索消息历史
- 自定义策略（例如，消息过滤等）

这使得智能体能够跟踪对话，同时不会超出大语言模型的上下文窗口。

4.6.3.1 修剪消息

大多数大语言模型都有一个最大支持的上下文窗口（以 token 为单位）。决定何时截断消息的一种方法是计算消息历史中的 token 数量，并在其接近该限制时进行截断。如果您使用 LangChain，可以使用修剪消息工具，并指定要从列表中保留的 token 数量，以及用于处理边界的 strategy（例如，保留最后的 max_tokens）。

要修剪消息历史，请使用 trim_messages 函数：

```
"""
LangGraph 消息修剪演示

展示了如何使用 trim_messages 函数来管理消息历史，
确保消息历史不会超过模型的最大上下文窗口限制。
如果环境中配置了 API 密钥，将使用百炼平台的通义大模型；否则使用模拟响应。
"""

import os
from typing import List
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.messages.utils import (
    trim_messages,
    count_tokens_approximately
)
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, MessagesState
from langgraph.checkpoint.memory import InMemorySaver

# 初始化模型
model = None
try:
    # 尝试初始化百炼平台的通义大模型
    api_key = "替换成你的百炼平台 API-KEY:sk-xxx"
    model = init_chat_model(
        "qwen-plus",
        model_provider="openai", # 使用 openai 提供者，但配置为百炼平台
        api_key=api_key,
        base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
        temperature=0.7
    )
```

```
    print("成功初始化百炼平台的通义大模型")
except Exception as e:
    print(f"初始化模型失败: {e}")
    print("将使用模拟响应模式")

def call_model(state: MessagesState):
    """
    调用模型的节点函数

    Args:
        state: 当前状态，包含消息历史

    Returns:
        dict: 更新后的状态
    """
    print("\n\n 执行节点: call_model")

    # 显示原始消息数量
    print(f"原始消息数量: {len(state['messages'])}")

    # 修剪消息历史，保留最后的 128 个 token
    messages = trim_messages(
        state["messages"],
        strategy="last",
        token_counter=count_tokens_approximately,
        max_tokens=128,
        start_on="human",
        end_on=("human", "tool"),
    )

    # 显示修剪后的消息数量
    print(f"修剪后消息数量: {len(messages)}")

    # 如果有模型则调用，否则使用模拟响应
    if model:
        try:
            response = model.invoke(messages)
            print(f"生成的回复: {response.content}")
            return {"messages": [response]}
        except Exception as e:
```

```
        print(f"调用模型出错: {e}")
        # 出错时使用模拟响应
        pass

    # 模拟模型调用
    last_message = state["messages"][-1].content if state["messages"] else ""

    # 根据消息内容生成响应
    if "名字" in last_message or "name" in last_message.lower():
        response = "我记得你的名字是 bob。"
    elif "诗" in last_message or "poem" in last_message.lower():
        if "猫" in last_message or "cat" in last_message.lower():
            response = "这里是一首关于猫的短诗: \n 小猫咪咪叫, \n 尾巴摇啊摇, \n 捉鼠本领高, \n 主人乐陶陶。"
        elif "狗" in last_message or "dog" in last_message.lower():
            response = "这里是一首关于狗的短诗: \n 小狗汪汪叫, \n 忠诚又可靠, \n 看家护院好, \n 人类好朋友。"
        else:
            response = "我可以为你写一首关于猫或狗的诗。"
    elif "你好" in last_message or "hi" in last_message.lower():
        response = "你好! 我是 AI 助手。"
    else:
        response = "我理解你的问题, 让我来帮助你解答。"

    print(f"生成的模拟回复: {response}")
    return {"messages": [AIMessage(content=response)]}

def main():
    """主函数 - 演示消息修剪功能"""
    print("=== LangGraph 消息修剪演示 (基于参考资料) ===\n")

    # 创建检查点保存器
    checkpointer = InMemorySaver()

    # 构建图
    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_edge(START, "call_model")
```

```
# 编译图
graph = builder.compile(checkpointer=checkpointer)

# 配置线程 ID
config = {"configurable": {"thread_id": "1"}}

# 第一次调用 - 问候
print("\n1. 第一次调用 - 问候:")
result1 = graph.invoke({
    "messages": [HumanMessage(content="hi, my name is bob")]
}, config)
print(f"回复: {result1['messages'][-1].content}")

# 第二次调用 - 请求写诗（关于猫）
print("\n2. 第二次调用 - 请求写诗（关于猫）:")
result2 = graph.invoke({
    "messages": [HumanMessage(content="write a short poem about cats")]
}, config)
print(f"回复: {result2['messages'][-1].content}")

# 第三次调用 - 请求写诗（关于狗）
print("\n3. 第三次调用 - 请求写诗（关于狗）:")
result3 = graph.invoke({
    "messages": [HumanMessage(content="now do the same but for dogs")]
}, config)
print(f"回复: {result3['messages'][-1].content}")

# 第四次调用 - 询问名字
print("\n4. 第四次调用 - 询问名字:")
final_response = graph.invoke({
    "messages": [HumanMessage(content="what's my name?")]
}, config)
print(f"回复: {final_response['messages'][-1].content}")

# 模拟大量消息以展示修剪效果
print("\n5. 模拟大量消息以展示修剪效果:")
# 添加大量消息
many_messages: List[HumanMessage] = []
for i in range(20):
```

```
        many_messages.append(HumanMessage(content=f"这是第{i+1}条测试消息,
内容很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长很长"))

    result5 = graph.invoke({
        "messages": many_messages + [HumanMessage(content="what's my
name?")]
    }, config)
    print(f"回复: {result5['messages'][-1].content}")

    print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.6.3.2 删除消息

要从图状态中删除消息，可以使用 `RemoveMessage`。`RemoveMessage` 要正常工作，需要 state 的 key 带有 `add_messages` 这个 reducer，例如 `MessagesState`。

```
"""
LangGraph 消息删除演示

该演示展示了如何使用 RemoveMessage 从图状态中删除消息。
当状态的 key 带有 add_messages 这个 reducer 时（例如 MessagesState），
RemoveMessage 可以正常工作。
"""

from typing import Annotated, Sequence
from langchain_core.messages import (
    HumanMessage,
    AIMessage,
    RemoveMessage,
    BaseMessage
)
from langchain_core.messages.utils import count_tokens_approximately
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from typing_extensions import TypedDict
```

```
# 定义状态类型
class CustomMessagesState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], "messages"]

# 初始化模型（使用模拟模型）
model = None
try:
    # 尝试初始化百炼平台的通义大模型
    api_key = "替换成你的百炼平台 API-KEY:sk-xxx"
    model = init_chat_model(
        "qwen-plus",
        model_provider="openai", # 使用 openai 提供者，但配置为百炼平台
        api_key=api_key,
        base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
        temperature=0.7
    )
    print("成功初始化百炼平台的通义大模型")
except Exception as e:
    print(f"初始化模型失败: {e}")
    print("将使用模拟响应模式")

def call_model(state: MessagesState):
    """
    调用模型的节点函数

    Args:
        state: 当前状态，包含消息历史

    Returns:
        dict: 更新后的状态
    """
    print("\n\n执行节点: call_model")
    print(f"当前消息数量: {len(state['messages'])}")

    # 显示所有消息
    for i, msg in enumerate(state["messages"]):
        print(f" 消息 {i+1}: {type(msg).__name__} - {msg.content[:50]}{'...' if len(msg.content) > 50 else ''}")
```

```
# 如果有模型则调用，否则使用模拟响应
if model:
    try:
        response = model.invoke(state["messages"])
        print(f"生成的回复: {response.content}")
        return {"messages": [response]}
    except Exception as e:
        print(f"调用模型出错: {e}")
        # 出错时使用模拟响应
        pass

# 模拟模型调用
last_message = state["messages"][-1].content if state["messages"] else ""

# 根据消息内容生成响应
if "名字" in last_message or "name" in last_message.lower():
    response = "我记得你的名字是 bob。"
elif "诗" in last_message or "poem" in last_message.lower():
    if "猫" in last_message or "cat" in last_message.lower():
        response = "这里是一首关于猫的短诗: \n 小猫咪咪叫, \n 尾巴摇啊摇, \n 捉鼠本领高, \n 主人乐陶陶。"
    elif "狗" in last_message or "dog" in last_message.lower():
        response = "这里是一首关于狗的短诗: \n 小狗汪汪叫, \n 忠诚又可靠, \n 看家护院好, \n 人类好朋友。"
    else:
        response = "我可以为你写一首关于猫或狗的诗。"
elif "你好" in last_message or "hi" in last_message.lower():
    response = "你好! 我是 AI 助手。"
else:
    response = "我理解你的问题, 让我来帮助你解答。"

print(f"生成的模拟回复: {response}")
return {"messages": [AIMessage(content=response)]}

def delete_messages(state: MessagesState):
    """
    删除消息的节点函数

    Args:
```



```
state: 当前状态，包含消息历史

Returns:
    dict: 更新后的状态
"""
print("\n 执行节点: delete_messages")
messages = state["messages"]
print(f"删除前消息数量: {len(messages)}")

if len(messages) > 2:
    # 删除最早的两条消息
    to_remove = [RemoveMessage(id=m.id) for m in messages[:2]]
    print(f"将删除 {len(to_remove)} 条消息")
    # 显示要删除的消息
    for i, msg in enumerate(messages[:2]):
        print(f"  删除消息 {i+1}: {type(msg).__name__} - {msg.content[:50]}{'...' if len(msg.content) > 50 else ''}")
    return {"messages": to_remove}
else:
    print("消息数量不足，无需删除")
    return {}

def main():
    """主函数 - 演示消息删除功能"""
    print("=== LangGraph 消息删除演示 ===\n")

    # 创建检查点保存器
    checkpointer = InMemorySaver()

    # 构建图
    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_node(delete_messages)

    # 添加边
    builder.add_edge(START, "call_model")
    builder.add_edge("call_model", "delete_messages")

    # 编译图
    app = builder.compile(checkpointer=checkpointer)
```

```
# 配置线程 ID
config = {"configurable": {"thread_id": "1"}}

# 第一次调用 - 问候
print("1. 第一次调用 - 问候:")
for event in app.stream(
    {"messages": [HumanMessage(content="hi! I'm bob")]},
    config,
    stream_mode="values"
):
    print(f"当前状态中的消息数量: {len(event['messages'])}")
    if event["messages"]:
        last_message = event["messages"][-1]
        print(f"最新消息: {type(last_message).__name__} - {last_message.content}")

    print("\n" + "="*50 + "\n")

# 第二次调用 - 询问名字
print("2. 第二次调用 - 询问名字:")
for event in app.stream(
    {"messages": [HumanMessage(content="what's my name?")]},
    config,
    stream_mode="values"
):
    print(f"当前状态中的消息数量: {len(event['messages'])}")
    if event["messages"]:
        last_message = event["messages"][-1]
        print(f"最新消息: {type(last_message).__name__} - {last_message.content}")

    print("\n" + "="*50 + "\n")

# 第三次调用 - 请求写诗
print("3. 第三次调用 - 请求写诗:")
for event in app.stream(
    {"messages": [HumanMessage(content="write a short poem about cats")]},
    config,
    stream_mode="values"
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
    ):
        print(f"当前状态中的消息数量: {len(event['messages'])}")
        if event["messages"]:
            last_message = event["messages"][-1]
            print(f"最新消息: {type(last_message).__name__} - {last_message.content}")

        print("\n" + "="*50 + "\n")

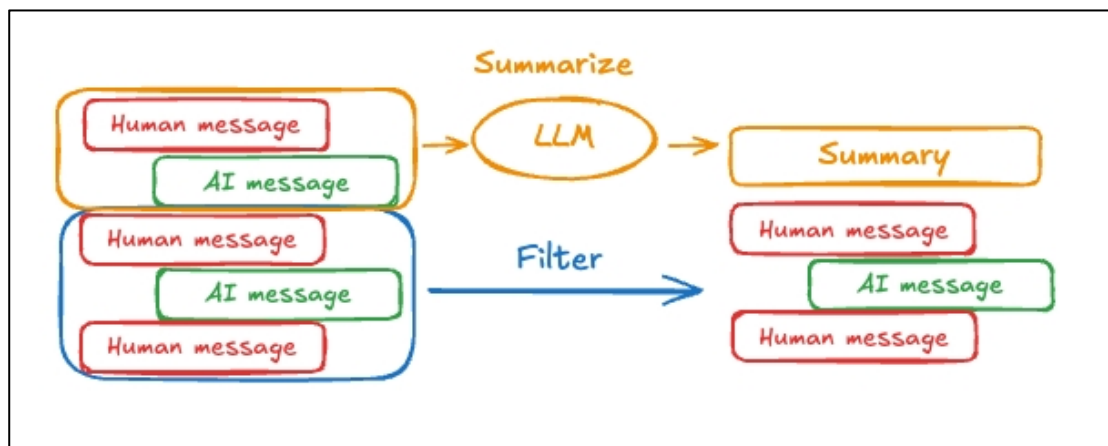
        # 第四次调用 - 请求写诗（关于狗）
        print("4. 第四次调用 - 请求写诗（关于狗）:")
        for event in app.stream(
            {"messages": [HumanMessage(content="now do the same but for dogs")]},
            config,
            stream_mode="values"
        ):
            print(f"当前状态中的消息数量: {len(event['messages'])}")
            if event["messages"]:
                last_message = event["messages"][-1]
                print(f"最新消息: {type(last_message).__name__} - {last_message.content}")

        print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.6.3.3 总结消息

修剪或删除消息存在一个问题，即清理消息队列时可能会丢失信息。正因为如此，一些应用程序会受益于一种更复杂的方法——使用聊天模型来总结消息历史。



提示词和编排逻辑可用于总结消息历史。

例如，在 LangGraph 中，你可以扩展 MessagesState 以包含一个 summary 键；然后，可以生成聊天历史的摘要，并将任何现有的摘要作为下一次摘要的上下文。在 messages 状态键中积累了一定数量的消息后，可以调用这个 summarize_conversation 节点。

"""

LangGraph 对话总结演示

该演示展示了如何使用聊天模型来总结消息历史，而不是简单地修剪或删除消息。这种方法可以避免在清理消息队列时丢失信息。

"""

```

from typing import Annotated, Sequence, TypedDict
from langchain_core.messages import HumanMessage, AIMessage, BaseMessage, SystemMessage
from langchain_core.messages.utils import count_tokens_approximately
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START
from langgraph.checkpoint.memory import InMemorySaver
from typing_extensions import TypedDict

```

定义状态类型

```

class SummaryState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], "messages"]
    summary: str

```

初始化模型（使用模拟模型）

```

model = None

```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
summarization_model = None

try:
    # 尝试初始化百炼平台的通义大模型
    api_key = "替换成你的百炼平台 API-KEY:sk-xxx"
    model = init_chat_model(
        "qwen-plus",
        model_provider="openai", # 使用 openai 提供者，但配置为百炼平台
        api_key=api_key,
        base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
        temperature=0.7
    )

    summarization_model = model.bind(max_tokens=128)
    print("成功初始化百炼平台的通义大模型")
except Exception as e:
    print(f"初始化模型失败: {e}")
    print("将使用模拟响应模式")

def summarize_conversation(messages: Sequence[BaseMessage],
                           current_summary: str = "") -> str:
    """
    使用模型总结对话历史

    Args:
        messages: 消息列表
        current_summary: 当前摘要

    Returns:
        str: 更新后的摘要
    """
    if not messages:
        return current_summary

    # 如果有模型则调用，否则使用模拟摘要
    if summarization_model:
        try:
            # 构造总结提示
            summary_prompt = f"当前摘要: {current_summary}\\n\\n 新对话:\\n"
            for msg in messages:
```

```
        if isinstance(msg, HumanMessage):
            summary_prompt += f"人类: {msg.content}\\n"
        elif isinstance(msg, AIMessage):
            summary_prompt += f"AI: {msg.content}\\n"

    summary_prompt += "\\n 请提供一个简洁的摘要，包含重要的信息和上下文:"

    response =
summarization_model.invoke([SystemMessage(content=summary_prompt)])
    return response.content
except Exception as e:
    print(f"调用总结模型出错: {e}")
    # 出错时使用模拟摘要
    pass

# 模拟摘要生成
summary_content = " ".join([msg.content for msg in messages[-3:]]) #
取最后 3 条消息
return f"对话摘要: {summary_content[:100]}..." # 简单截取前 100 个字符

def summarize_node(state: SummaryState):
    """
    总结节点函数

    Args:
        state: 当前状态

    Returns:
        dict: 更新后的状态
    """
    print("\\n 执行节点: summarize_node")
    messages = state["messages"]
    current_summary = state.get("summary", "")

    print(f"当前消息数量: {len(messages)}")
    print(f"当前摘要: {current_summary}")

    # 如果消息数量超过阈值，进行总结
    if len(messages) > 4: # 当消息数量超过 4 条时进行总结
```

```
print("消息数量超过阈值，开始总结对话历史...")
# 取最近的几条消息进行总结
recent_messages = messages[-4:] # 最近 4 条消息
new_summary = summarize_conversation(recent_messages,
current_summary)
print(f"生成的新摘要: {new_summary}")

# 返回更新后的摘要和保留最近的几条消息
return {
    "summary": new_summary,
    "messages": messages[-2:] # 保留最近 2 条消息
}
else:
    print("消息数量未超过阈值，无需总结")
    return {"summary": current_summary}

def call_model(state: SummaryState):
    """
    调用模型的节点函数

    Args:
        state: 当前状态，包含消息历史和摘要

    Returns:
        dict: 更新后的状态
    """
    print("\n执行节点: call_model")
    messages = state["messages"]
    summary = state.get("summary", "")

    print(f"当前消息数量: {len(messages)}")
    print(f"当前摘要: {summary}")

    # 构造包含摘要的完整上下文
    context_messages = []
    if summary:
        context_messages.append(SystemMessage(content=f"之前的对话摘要: {summary}"))

    context_messages.extend(messages)
```

```
# 显示所有消息
for i, msg in enumerate(context_messages):
    print(f" 消息 {i+1}: {type(msg).__name__} - {msg.content[:50]}{'...' if len(msg.content) > 50 else ''}")

# 如果有模型则调用，否则使用模拟响应
if model:
    try:
        response = model.invoke(context_messages)
        print(f"生成的回复: {response.content}")
        return {"messages": [response]}
    except Exception as e:
        print(f"调用模型出错: {e}")
        # 出错时使用模拟响应
        pass

# 模拟模型调用
last_message = messages[-1].content if messages else ""

# 根据消息内容生成响应
if "名字" in last_message or "name" in last_message.lower():
    if "bob" in last_message.lower():
        response = "我记得你的名字是 bob。"
    else:
        response = "你还没有告诉我你的名字呢。"
elif "诗" in last_message or "poem" in last_message.lower():
    if "猫" in last_message or "cat" in last_message.lower():
        response = "这里是一首关于猫的短诗: \n 小猫咪咪叫, \n 尾巴摇啊摇, \n 捉鼠本领高, \n 主人乐陶陶。"
    elif "狗" in last_message or "dog" in last_message.lower():
        response = "这里是一首关于狗的短诗: \n 小狗汪汪叫, \n 忠诚又可靠, \n 看家护院好, \n 人类好朋友。"
    else:
        response = "我可以为你写一首关于猫或狗的诗。"
elif "你好" in last_message or "hi" in last_message.lower():
    response = "你好! 我是 AI 助手。"
else:
    response = "我理解你的问题, 让我来帮助你解答。"

print(f"生成的模拟回复: {response}")
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网


```
    return {"messages": [AIMessage(content=response)]}

def main():
    """主函数 - 演示对话总结功能"""
    print("=== LangGraph 对话总结演示 ===\n")

    # 创建检查点保存器
    checkpointer = InMemorySaver()

    # 构建图
    builder = StateGraph(SummaryState)
    builder.add_node("summarize", summarize_node)
    builder.add_node("call_model", call_model)

    # 添加边
    builder.add_edge(START, "summarize")
    builder.add_edge("summarize", "call_model")

    # 编译图
    graph = builder.compile(checkpointer=checkpointer)

    # 配置线程 ID
    config = {"configurable": {"thread_id": "1"}}

    # 第一次调用 - 问候
    print("1. 第一次调用 - 问候:")
    result1 = graph.invoke({
        "messages": [HumanMessage(content="hi, my name is bob")],
        "summary": ""
    }, config)
    print(f"回复: {result1['messages'][-1].content}")
    print(f"当前摘要: {result1.get('summary', '')}")

    print("\n" + "="*50 + "\n")

    # 第二次调用 - 请求写诗（关于猫）
    print("2. 第二次调用 - 请求写诗（关于猫）:")
    result2 = graph.invoke({
        "messages": [HumanMessage(content="write a short poem about
cats")],
```

```
        "summary": result1.get("summary", "")
    }, config)
    print(f"回复: {result2['messages'][-1].content}")
    print(f"当前摘要: {result2.get('summary', '')}")

    print("\n" + "="*50 + "\n")

    # 第三次调用 - 请求写诗（关于狗）
    print("3. 第三次调用 - 请求写诗（关于狗）:")
    result3 = graph.invoke({
        "messages": [HumanMessage(content="now do the same but for dogs")],
        "summary": result2.get("summary", "")
    }, config)
    print(f"回复: {result3['messages'][-1].content}")
    print(f"当前摘要: {result3.get('summary', '')}")

    print("\n" + "="*50 + "\n")

    # 第四次调用 - 询问名字
    print("4. 第四次调用 - 询问名字:")
    result4 = graph.invoke({
        "messages": [HumanMessage(content="what's my name?")],
        "summary": result3.get("summary", "")
    }, config)
    print(f"回复: {result4['messages'][-1].content}")
    print(f"当前摘要: {result4.get('summary', '')}")

    print("\n" + "="*50 + "\n")

    # 第五次调用 - 添加更多对话以触发总结
    print("5. 第五次调用 - 添加更多对话以触发总结:")
    conversation_history = [
        HumanMessage(content="让我们聊聊天气"),
        AIMessage(content="好的，你想聊什么地区的天气？"),
        HumanMessage(content="北京的天气怎么样？"),
        AIMessage(content="我无法获取实时天气信息，但北京属于温带大陆性季风气候。"),
        HumanMessage(content="what's my name?") # 再次询问名字
    ]

    result5 = graph.invoke({
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
        "messages": conversation_history,
        "summary": result4.get("summary", "")
    }, config)
    print(f"回复: {result5['messages'][-1].content}")
    print(f"当前摘要: {result5.get('summary', '')}")

    print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

4.7 子图 (Subgraphs)

子图是一个用作另一个图中节点的图。子图可用于：

- 构建多智能体系统。
- 在多个图中重用一组节点。
- 分布式开发：当你希望不同的团队独立处理图的不同部分时，你可以将每个部分定义为子图，而且只要遵循子图接口（输入和输出模式），父图就可以在不了解子图任何细节的情况下构建出来。

添加子图时，需要定义父图和子图如何通信：

- 从节点调用图 — 子图从父图的节点内部被调用。
- 将图添加为节点——子图直接作为节点添加到父图中，并与父图共享状态。

4.7.1 使用方式

1) 从节点调用图

实现子图的一种简单方法是从另一个图的节点内部调用一个图。在这种情况下，子图可以与父图具有完全不同的模式（没有共享键）。例如，你可能希望为多智能体系统中的每个智能体保存私人消息历史。如果你的应用程序是这种情况，你需要定义一个节点函数来调用子图。该函数需要在调用子图之前将输入（父级）状态转换为子图状态，并在从节点返回状态更新之前将结果转换回父级状态。

2) 将图形添加为节点

当父图和子图能够通过模式中的共享状态键（通道）进行通信时，你可以将一个图作为节点添加到另一个图中。例如，在多智能体系统中，智能体通常通过共享的消息键进行通信。

3) 添加持久性

更多 [Java](#) - 大数据 - 前端 - python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

在编译父图时，你只需提供检查点工具。LangGraph 会自动将检查点工具传播到子图中。

4) 查看子图状态

启用持久性后，您可以通过相应方法检查图状态（检查点）。要查看子图状态，可使用子图选项。

5) 流式输出子图结果

要在流式输出中包含子图的输出，你可以在父图的 `stream` 方法中设置 `subgraphs` 选项。这样会同时流式输出父图和所有子图的内容。

4.7.2 使用案例

```
"""
LangGraph 子图功能演示

该演示展示了如何在 LangGraph 中使用子图，包括：
1. 从节点调用图（不同的状态模式）
2. 将图添加为节点（共享状态模式）
3. 查看子图状态
4. 流式输出子图结果
"""

from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.state import StateGraph, START
from langgraph.checkpoint.memory import MemorySaver

# 定义子图状态（不同的状态模式）
class SubgraphState(TypedDict):
    bar: str
    baz: str

# 定义父图状态（不同的状态模式）
class ParentState(TypedDict):
    foo: str

# 定义共享状态的子图
```

```
class SharedSubgraphState(TypedDict):
    foo: str # 共享状态键
    bar: str # 子图私有状态键

# 定义用于中断演示的状态
class InterruptState(TypedDict):
    foo: str

def subgraph_node_1(state: SubgraphState):
    """子图节点 1"""
    print("执行子图节点 1")
    return {"baz": "baz"}

def subgraph_node_2(state: SubgraphState):
    """子图节点 2"""
    print("执行子图节点 2")
    return {"bar": state["bar"] + state["baz"]}

def shared_subgraph_node_1(state: SharedSubgraphState):
    """共享状态子图节点 1"""
    print("执行共享状态子图节点 1")
    return {"bar": "bar"}

def shared_subgraph_node_2(state: SharedSubgraphState):
    """共享状态子图节点 2"""
    print("执行共享状态子图节点 2")
    return {"foo": state["foo"] + state["bar"]}

def interrupt_subgraph_node(state: InterruptState):
    """用于中断演示的子图节点"""
    print("执行中断子图节点")
    # 模拟中断，实际应用中会使用 interrupt() 函数
    user_input = input("请输入值（模拟中断）：")
    return {"foo": state["foo"] + user_input}

def create_subgraph_different_schemas():
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
"""创建具有不同状态模式的子图"""
print("\n=== 创建具有不同状态模式的子图 ===")
subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node("subgraph_node_1", subgraph_node_1)
subgraph_builder.add_node("subgraph_node_2", subgraph_node_2)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph_builder.add_edge("subgraph_node_1", "subgraph_node_2")
return subgraph_builder.compile()

def node_1(state: ParentState):
    """父图节点 1"""
    print("执行父图节点 1")
    return {"foo": "hi! " + state["foo"]}

def node_2(subgraph):
    """父图节点 2 - 调用子图"""
    def _call_subgraph(state: ParentState):
        print("执行父图节点 2（调用子图）")
        # 转换状态到子图格式
        subgraph_input = {"bar": state["foo"], "baz": ""}
        response = subgraph.invoke(subgraph_input)
        # 转换响应回父图格式
        return {"foo": response["bar"]}
    return _call_subgraph

def create_parent_graph_with_subgraph_call(subgraph):
    """创建通过节点调用子图的父图"""
    print("\n=== 创建通过节点调用子图的父图 ===")
    builder = StateGraph(ParentState)
    builder.add_node("node_1", node_1)
    builder.add_node("node_2", node_2(subgraph))
    builder.add_edge(START, "node_1")
    builder.add_edge("node_1", "node_2")
    return builder.compile()

def create_shared_subgraph():
    """创建具有共享状态的子图"""
    print("\n=== 创建具有共享状态的子图 ===")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
    subgraph_builder = StateGraph(SharedSubgraphState)
    subgraph_builder.add_node("shared_subgraph_node_1",
shared_subgraph_node_1)
    subgraph_builder.add_node("shared_subgraph_node_2",
shared_subgraph_node_2)
    subgraph_builder.add_edge(START, "shared_subgraph_node_1")
    subgraph_builder.add_edge("shared_subgraph_node_1",
"shared_subgraph_node_2")
    return subgraph_builder.compile()

def create_parent_graph_with_node_subgraph(subgraph):
    """创建将子图作为节点添加的父图"""
    print("\n=== 创建将子图作为节点添加的父图 ===")
    builder = StateGraph(ParentState)
    builder.add_node("node_1", node_1)
    builder.add_node("node_2", subgraph) # 直接将子图作为节点添加
    builder.add_edge(START, "node_1")
    builder.add_edge("node_1", "node_2")
    return builder.compile()

def create_interrupt_subgraph():
    """创建用于中断演示的子图"""
    print("\n=== 创建用于中断演示的子图 ===")
    subgraph_builder = StateGraph(InterruptState)
    subgraph_builder.add_node("interrupt_subgraph_node",
interrupt_subgraph_node)
    subgraph_builder.add_edge(START, "interrupt_subgraph_node")
    return subgraph_builder.compile()

def create_parent_graph_with_interrupt_subgraph(subgraph):
    """创建包含中断子图的父图"""
    print("\n=== 创建包含中断子图的父图 ===")
    builder = StateGraph(InterruptState)
    builder.add_node("node_1", subgraph)
    builder.add_edge(START, "node_1")
    return builder.compile()

def demo_subgraph_call():
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
"""演示从节点调用图"""
print("\n=== 演示从节点调用图 ===")
subgraph = create_subgraph_different_schemas()
parent_graph = create_parent_graph_with_subgraph_call(subgraph)

print("开始执行图:")
for chunk in parent_graph.stream({"foo": "foo"}, subgraphs=True):
    print(f"流式输出: {chunk}")

def demo_add_graph_as_node():
    """演示将图添加为节点"""
    print("\n=== 演示将图添加为节点 ===")
    subgraph = create_shared_subgraph()
    parent_graph = create_parent_graph_with_node_subgraph(subgraph)

    print("开始执行图:")
    for chunk in parent_graph.stream({"foo": "foo"}):
        print(f"流式输出: {chunk}")

def demo_subgraph_streaming():
    """演示流式输出子图结果"""
    print("\n=== 演示流式输出子图结果 ===")
    subgraph = create_shared_subgraph()
    parent_graph = create_parent_graph_with_node_subgraph(subgraph)

    print("开始流式执行图:")
    for chunk in parent_graph.stream(
        {"foo": "foo"},
        stream_mode="updates",
        subgraphs=True,
    ):
        print(f"流式输出: {chunk}")

def main():
    """主函数"""
    print("=== LangGraph 子图功能演示 ===")

    # 演示从节点调用图
```



```
demo_subgraph_call()

print("\n" + "="*50 + "\n")

# 演示将图添加为节点
demo_add_graph_as_node()

print("\n" + "="*50 + "\n")

# 演示流式输出子图结果
demo_subgraph_streaming()

print("\n=== 演示完成 ===")

if __name__ == "__main__":
    main()
```

第 5 章 Functional API

5.1 Functional API 简介

5.1.1 核心概念

LangGraph 函数式 API 能在对现有代码改动极小的情况下，为应用添加持久性、内存、人机协作和流式传输等关键特性。它无需将代码重构为显式管道或 DAG，可利用 if 语句、for 循环等标准语言原语定义工作流，提供极简抽象用于构建含状态管理和流处理的工作流。

其与 Graph API（图 API）的关键区别如下：

对比维度	函数式 API	Graph API
控制流	用标准 Python 结构定义，无需考虑图结构，代码量更少	需以图范式定义工作流
短期内存	无需显式状态管理，状态作用于函数内，不跨函数共享	需声明 State，可能需定义 reducers 管理状态更新
检查点	任务执行结果保存到现有检查点（与入口点关联）	每超步生成新检查点

可视化	运行时动态生成图，不支持可视化	易将工作流可视化为图，便于调试、理解和分享
-----	-----------------	-----------------------

两者共享相同底层运行时，可在同一应用中一起使用。

5.1.2 关键构建块

5.1.2.1 @entrypoint（入口点）

1) 定义

用@entrypoint 装饰器标记函数作为工作流起点，该函数需接受单个位置参数（若需传递多份数据，可将第一个参数设为字典类型）。装饰后生成 Pregel 实例，用于管理工作流执行（如处理流式传输、恢复和检查点）。通常需向装饰器传递检查点（checkpoint）以实现持久性和人机协作等功能，示例如下：

```
checkpointer = InMemorySaver()
@entrypoint(checkpointer = checkpointer)
def workflow(topic:str)->dict:
    pass
```

2) 可注入参数

声明入口点时，可请求自动注入运行时的额外参数，需指定正确名称和类型注解，参数说明如下：

参数	描述
previous	访问与给定线程的上一个检查点相关的状态，用于短期记忆
store	BaseStore 实例，用于长期内存
writer	在 Async Python < 3.11 中使用，用于访问 StreamWriter
config	用于访问运行时配置 RunnableConfig

示例：

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore
from langchain_core.runnables import RunnableConfig
from langgraph.types import StreamWriter
from langgraph.store.base import BaseStore
from typing import Any
from langgraph.func import entrypoint,task
import time
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
@task
def write_essay(topic:str)->str:
    time.sleep(2)
    return f'An essay about topic:{topic}'

checkpointer = InMemorySaver()
store = InMemoryStore()
@entrypoint(checkpointer=InMemorySaver(),store=store)
def
workflow(topic:str,*,previous:Any,store:BaseStore,writer:StreamWriter,c
onfig:RunnableConfig)->dict:
    print('previous 打印结果为: ',previous)
    print('store 打印结果为: ',store)
    print('writer 打印结果为: ',writer)
    print('config 打印结果为: ',config)
    essay = write_essay("cat")
    print('当前 essay 的类型为: ',type(essay))
    print("获取结果")
    essay = essay.result()
    return {
        'essay':essay,
    }
```

3) 执行与恢复

entrypoint 生成的 Pregel 对象可通过 invoke（同步等待结果）、ainvoke（异步等待结果）、stream（同步流式传输）、astream（异步流式传输）方法执行。

在 entrypoint 装饰的函数当中，如果存在 interrupt 调用，那么和 graphAPI 一样，程序执行会在此处中断暂停，等待通过 Command 进行恢复。

代码示例如下：

```
import uuid
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore
from langchain_core.runnables import RunnableConfig
from langgraph.types import StreamWriter,interrupt
from langgraph.store.base import BaseStore
from typing import Any
from langgraph.func import entrypoint,task
import time
@task
def write_essay(topic:str)->str:
    time.sleep(2)
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
        return f'An essay about topic:{topic}'

checkpointer = InMemorySaver()
store = InMemoryStore()
@entrypoint(checkpointer=InMemorySaver(),store=store)
def workflow(topic:str)->dict:
    essay = write_essay(topic).result()

    is_approved = interrupt(
        {
            'essay':essay,
            'action':"Please approve/reject the essay"
        }
    )
    return {
        'essay':essay,
        'is_approved':is_approved
    }

import uuid
thread_id = str(uuid.uuid4())
config = {"configurable":{"thread_id":thread_id}}
res = workflow.invoke("cat",config=config)
print('当前 res 为: ',res)

from langgraph.types import Command
human_review = True
resumed_result =
workflow.invoke(Command(resume=human_review),config=config)
print('恢复后结果为: ',resumed_result)
```

4) 短期记忆与 entrypoint.final

短期记忆: entrypoint 在定义时用检查点时，会在同一线程 ID 的连续调用间存储信息，可通过 previous 参数访问上一次调用的状态，默认 previous 参数为上一次调用的返回值，示例：

```
from langgraph.checkpoint.memory import InMemorySaver
from typing import Any
checkpointer = InMemorySaver()
@entrypoint(checkpointer=checkpointer)
def my_workflow(number: int, *, previous: Any = None) -> int:
    previous = previous or 0
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
    return number + previous

config = {
    "configurable": {
        "thread_id": "some_thread_id"
    }
}

print(my_workflow.invoke(1, config)) # 1 首次调用时, previous 为空, 因此此处结果为空
print(my_workflow.invoke(2, config)) # 3 第二次调用时, previous 参数会被赋值为上一次调用的结果 1, 因此此处返回的结果是 3
```

entrypoint.final: 从入口点返回的特殊原语, 可将检查点中保存的值与入口点返回值解耦, 类型注解为 `entrypoint.final[return_type, save_type]`, 第一个值是入口点返回值, 第二个值是要保存在检查点中的值, 示例:

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint
from typing import Any
checkpointer = InMemorySaver()
@entrypoint(checkpointer=checkpointer)
def my_workflow(number: int, *, previous: Any = None) ->
entrypoint.final[int, int]:
    previous = previous or 0
    # 将上次调用结果返回给调用方, 将 2*number 存储到检查点中, 并且在下次调用时, 用作 previous 的值
    return entrypoint.final(value=previous, save=2 * number)

config = {
    "configurable": {
        "thread_id": "1"
    }
}

print(my_workflow.invoke(3, config)) # 0 首次调用时, previous 为空
print(my_workflow.invoke(1, config)) # 6 第二次调用, previous 为 2*3=6, 因此本次返回 6
```

5.1.2.2 @task (任务)

1) 定义

用 `@task` 装饰器包装普通 Python 函数, 定义独立的工作单元 (如 API 调用、数据处理步骤), 示例:

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
@task
def slow_computation(params:list):
    import time
    print('开始执行 slow_computation')
    time.sleep(5) # 此处模拟费时操作
    print("slow_computation 执行结束")
    return sum(params)
```

2) 执行

`task` 装饰的函数任务只能从 `entrypoint` 内部、其他任务或状态图节点中调用，不能从主应用代码直接调用。调用 `task` 时，不会阻塞 `task` 内部代码继续往下执行，而是会立即返回 `future` 对象（`future`，用于后续获取结果），可通过 `result()`（同步等待）或 `await`（异步等待）获取结果。

完整示例如下：

```
from langgraph.func import entrypoint,task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt,Command
checkpointer = InMemorySaver()

@task
def slow_computation(params:list):
    import time
    print('开始执行 slow_computation')
    time.sleep(5) # 此处模拟费时操作
    print("slow_computation 执行结束")
    return sum(params)

@entrypoint(checkpointer=checkpointer)
def my_workflow(params:list):
    print("开始执行 my_workflow")
    computation_result = slow_computation(params).result()
    user_input = interrupt(
        "中断，等待用户输入"
    )
    print(f"用户输入: {user_input}")
    print("my_workflow 执行结束")
    return computation_result+user_input
config = {"configurable":{"thread_id":1}}
my_workflow.invoke([1,2,3],config=config)
my_workflow.invoke(Command(resume=4),config=config)
```

3) 适用场景

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

检查点：需保存长时间运行操作结果到检查点，避免恢复 workflow 时重新计算。

人机协作：构建需人工干预的工作流时，必须用任务封装随机性（如 API 调用），确保工作流能正确恢复。

并行执行：对于 I/O 绑定任务，任务支持并行执行，可同时运行多个操作（如调用多个 API），不阻塞流程。

可观测性：将操作包装在任务中，可通过 LangSmith 跟踪 workflow 进度，监控单个操作执行情况。

可重试工作：需重试处理失败或不一致情况时，任务可封装和管理重试逻辑。

5.1.3 重要特性

5.1.3.1 序列化

为支持检查点和 workflow 恢复，入口点的输入输出以及任务的输出必须是 JSON 可序列化的，建议使用字典、列表、字符串、数字、布尔值等 Python 原语。若提供非序列化的输入输出，配置检查点的工作流会出现运行时错误。序列化能确保 workflow 状态（如任务结果、中间值）可靠保存和恢复，对实现人机协作、容错和并行执行至关重要。

5.1.3.2 确定性

为实现人机协作等功能，需将随机性封装在任务内。这样即使任务结果非确定性，workflow 暂停（如为人机协作）后恢复，仍会遵循相同步骤序列。LangGraph 会在任务和子图执行时持久化其结果，设计良好的 workflow 能确保恢复时获取之前计算的结果，无需重新执行，避免重复工作。不同 workflow 运行可能产生不同结果，但特定运行的恢复应遵循相同记录步骤。

5.1.3.3 幂等性

幂等性确保多次运行同一操作产生相同结果，可防止因失败重新执行步骤时出现重复 API 调用和冗余处理。需将 API 调用放在任务函数中（便于检查点），并设计为幂等的。若任务启动后未成功完成，恢复 workflow 时任务会重新运行，可使用幂等键或验证现有结果避免重复。

5.1.4 示例：写文章并请求人工审核 workflow

"""

基于 LangGraph 1.0 函数式 API 的示例程序，展示了以下特性：

1. 使用 `@task` 装饰器定义任务节点
 - `write_essay`：模拟生成文章的任务

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

2. 使用`@entrypoint` 装饰器定义工作流入口
 - 包含中断机制，可以暂停工作流等待外部输入
3. 中断与恢复机制
 - 使用 `interrupt()`函数暂停工作流并传递数据给外部
 - 使用 `Command(resume=value)`恢复工作流执行
4. 内存状态保存
 - 使用 `InMemorySaver()`保存工作流状态

示例功能流程：

1. 启动生成文章任务
2. 文章生成完成后中断工作流，等待人工审核
3. 接收人工审核结果
4. 根据审核结果继续执行并返回最终结果

"""

```
import time
```

```
import uuid
```

```
from langgraph.func import entrypoint, task
```

```
from langgraph.types import interrupt, Command
```

```
from langgraph.checkpoint.memory import InMemorySaver
```

```
# 模拟写文章的任务节点
```

```
@task
```

```
def write_essay(topic: str) -> str:
```

```
    """根据给定主题写一篇文章"""
```

```
    print(f"正在生成关于'{topic}'的文章...")
```

```
    time.sleep(1) # 模拟耗时操作
```

```
    essay = f"这是一篇关于'{topic}'的文章内容。文章包括引言、正文和结论等部分。
```

```
    """
```

```
    print(f"文章生成完成: {essay}")
```

```
    return essay
```

```
# 定义工作流入口点
```

```
@entrypoint(checkpointer=InMemorySaver())
```

```
def workflow(topic: str) -> dict:
```

```
    """一个简单的工作流，用于生成文章并请求审核"""
```

```
    # 执行写文章任务
```

```
    essay = write_essay(topic).result()
```



```
# 中断工作流，等待人工审核
is_approved = interrupt(
    {
        # 提供给中断的 JSON 序列化数据
        # 当从工作流中流式传输数据时，这些信息会在客户端显示为中断
        "essay": essay, # 待审核的文章
        # 可以添加任何我们需要的额外信息
        # 例如，添加一个名为"action"的键并附上说明指令
        "action": "请审核这篇文章，通过输入 True 或 False 来表示是否批准",
    }
)

# 返回最终结果
return {
    "essay": essay, # 生成的文章
    "is_approved": is_approved, # 来自人工审核的响应
}

def main():
    """主函数，演示如何运行带有人工中断的工作流"""
    print("=== LangGraph 1.0 函数式 API 示例 ===")

    # 生成唯一的线程 ID 用于状态保存
    thread_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": thread_id}}
    print(f"工作流线程 ID: {thread_id}")

    print("\n--- 第一阶段：开始生成文章 ---")
    # 启动工作流并流式输出结果
    stream_iter = workflow.stream("猫", config)

    try:
        for item in stream_iter:
            print(f"流式输出项: {item}")

            # 检查是否有中断信号
            if "__interrupt__" in item:
                print("\n--- 工作流中断，等待人工审核 ---")
                interrupt_data = item["__interrupt__"][0].value
                print(f"待审核文章: {interrupt_data['essay']}")
```

```
print(f"操作提示: {interrupt_data['action']}")

# 模拟人工审核过程
print("\n--- 模拟人工审核 ---")
# 在实际情况下, 这里可能会有一个 UI 界面让用户进行审核
# 在此示例中, 我们直接使用布尔值作为审核结果
human_review = True # 模拟用户批准

print(f"审核结果: {'批准' if human_review else '拒绝'}")

print("\n--- 第二阶段: 继续 workflow 执行 ---")
# 使用审核结果恢复 workflow 执行
for resume_item in
workflow.stream(Command(resume=human_review), config):
    print(f"恢复后的流式输出项: {resume_item}")

except Exception as e:
    print(f"执行过程中出现错误: {e}")

if __name__ == "__main__":
    main()
```

5.2 并行执行

通过并发调用任务并等待结果, 可以并行执行这些任务。这对于提高 IO 密集型任务(例如, 调用大语言模型的 API) 的性能很有用。

示例中会用到 langchain 的 api, 所以安装 langchain 依赖:

```
pip install langchain
```

案例:

"""
基于 LangGraph 1.0 函数式 API 的并发任务执行示例

本示例演示了如何使用 LangGraph 的 Functional API 并行执行多个任务,
特别适用于 IO 密集型任务(如调用大语言模型 API) 以提高性能。

主要特性:

1. 使用@task 装饰器定义可并行执行的任务
2. 使用@entrypoint 装饰器定义工作流入口点
3. 并发执行多个任务并等待所有结果
4. 使用 InMemorySaver 进行状态持久化

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
"""

import time
import uuid
from typing import List

# 导入 LangGraph Functional API 相关组件
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

# 模拟耗时的 IO 操作（如调用 LLM API）
def simulate_llm_call(topic: str, delay: float = 1.0) -> str:
    """
    模拟调用大语言模型 API 生成段落

    Args:
        topic: 主题
        delay: 模拟延迟时间（秒）

    Returns:
        生成的段落内容
    """
    print(f"开始生成关于 '{topic}' 的段落...")
    time.sleep(delay) # 模拟网络 IO 延迟
    paragraph = f"这是关于 '{topic}' 的一段内容。在这个段落中，我们会讨论这个主题的各种方面和细节。"
    print(f"完成 '{topic}' 段落生成")
    return paragraph

# 定义任务函数 - 生成指定主题的段落
@task
def generate_paragraph(topic: str) -> str:
    """
    生成关于给定主题的段落

    Args:
        topic: 段落主题

    Returns:
```

```
        生成的段落内容
        """
        return simulate_llm_call(topic, 1.0)

# 定义工作流入口点
@entrypoint(checkpointer=InMemorySaver())
def workflow(topics: List[str]) -> str:
    """
    并行生成多个主题的段落并组合成完整文本

    Args:
        topics: 主题列表

    Returns:
        组合后的完整文本
    """
    # 并行启动所有任务
    futures = [generate_paragraph(topic) for topic in topics]

    # 等待所有任务完成并获取结果
    paragraphs = [f.result() for f in futures]

    # 用双换行符合并所有段落
    return "\n\n".join(paragraphs)

def main():
    """主函数，演示并发执行任务"""
    print("=== LangGraph 1.0 函数式 API 并发执行示例 ===")

    # 定义要处理的主题列表
    topics = ["量子计算", "气候变化", "航空史"]
    print(f"待处理的主题: {topics}")

    # 生成唯一线程 ID 用于状态保存
    thread_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": thread_id}}
    print(f"工作流线程 ID: {thread_id}")

    # 记录开始时间
```

```
start_time = time.time()

# 执行工作流
print("\n--- 开始并行执行任务 ---")
result = workflow.invoke(topics, config=config)

# 计算执行时间
elapsed_time = time.time() - start_time

# 输出结果
print("\n--- 生成结果 ---")
print(result)

print(f"\n--- 执行统计 ---")
print(f"处理主题数量: {len(topics)} 个")
print(f"总耗时: {elapsed_time:.2f} 秒")
print(f"平均每个主题耗时: {elapsed_time/len(topics):.2f} 秒")

# 对比串行执行
print("\n--- 串行执行对比 ---")
serial_start = time.time()
serial_results = []
for topic in topics:
    serial_results.append(simulate_llm_call(topic, 1.0))
serial_elapsed = time.time() - serial_start
print(f"串行执行总耗时: {serial_elapsed:.2f} 秒")

print(f"\n--- 性能提升 ---")
improvement = serial_elapsed / elapsed_time
print(f"并行执行相较于串行执行提升了 {improvement:.1f} 倍性能")

if __name__ == "__main__":
    main()
```

5.3 调用 Graph

函数式 API 和图 API 可以在同一个应用程序中一起使用，因为它们共享相同的底层运行时。

```
"""
基于 LangGraph 1.0 混合 API 的示例程序
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

本示例演示了如何在同一应用程序中同时使用函数式 API 和图 API，展示了两种 API 共享相同底层运行时特点。

主要特性：

1. 使用图 API 定义状态图和节点
2. 使用函数式 API 定义工作流入口点
3. 在函数式 API 中调用图 API 构建的状态图
4. 使用 InMemorySaver 进行状态持久化

"""

```
import uuid
from typing import TypedDict
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import StateGraph, START, END
```

定义共享状态类型

```
class State(TypedDict):
    """状态定义，包含一个整数字段 foo"""
    foo: int
```

定义简单的转换节点

```
def double(state: State) -> State:
    """
    将状态中的 foo 值翻倍

    Args:
        state: 当前状态

    Returns:
        更新后的状态
    """
    print(f"执行 double 节点: {state['foo']} * 2 = {state['foo'] * 2}")
    return {"foo": state["foo"] * 2}
```

```
def add_five(state: State) -> State:
    """
```

将状态中的 `foo` 值加 5

Args:

`state`: 当前状态

Returns:

更新后的状态

"""

```
print(f"执行 add_five 节点: {state['foo']} + 5 = {state['foo'] + 5}")
```

```
return {"foo": state["foo"] + 5}
```

使用图 API 构建状态图

```
def build_graph():
```

"""

构建一个简单的状态图

Returns:

编译后的状态图

"""

```
builder = StateGraph(State)
```

```
builder.add_node("double", double)
```

```
builder.add_node("add_five", add_five)
```

```
builder.add_edge(START, "double")
```

```
builder.add_edge("double", "add_five")
```

```
builder.add_edge("add_five", END)
```

```
return builder.compile()
```

创建图实例

```
graph = build_graph()
```

定义函数式 API 任务 - 处理单个数字

```
@task
```

```
def process_number(x: int) -> int:
```

"""

使用图 API 处理单个数字

Args:

`x`: 输入数字

```
Returns:
    处理后的结果
    """
    result = graph.invoke({"foo": x})
    return result["foo"]

# 定义函数式 API 工作流入口点
@entrypoint(checkpointer=InMemorySaver())
def workflow(numbers: list[int]) -> dict:
    """
    处理数字列表的工作流

    Args:
        numbers: 数字列表

    Returns:
        处理结果字典
        """
    print(f"开始处理数字列表: {numbers}")

    # 并行处理所有数字
    futures = [process_number(num) for num in numbers]
    results = [f.result() for f in futures]

    return {
        "input_numbers": numbers,
        "output_numbers": results,
        "total": sum(results)
    }

def main():
    """主函数，演示混合 API 的使用"""
    print("=== LangGraph 1.0 混合 API 使用示例 ===")

    # 定义输入数据
    numbers = [1, 2, 3, 4, 5]
    print(f"输入数字列表: {numbers}")
```



```
# 生成唯一线程 ID 用于状态保存
thread_id = str(uuid.uuid4())
config = {"configurable": {"thread_id": thread_id}}
print(f"工作流线程 ID: {thread_id}")

# 执行工作流
print("\n--- 开始执行混合 API 工作流 ---")
result = workflow.invoke(numbers, config=config)

# 输出结果
print("\n--- 处理结果 ---")
print(f"输入数字: {result['input_numbers']}")
print(f"输出数字: {result['output_numbers']}")
print(f"总和: {result['total']}")

# 单独演示图 API 的使用
print("\n--- 单独演示图 API ---")
graph_result = graph.invoke({"foo": 10})
print(f"图 API 处理结果: {graph_result}") # 应该输出: {'foo': 25}
(10*2+5)

# 为了更好地理解两种 API 的关系，我们再创建一个更直接的混合示例
@entrypoint(checkpointer=InMemorySaver())
def simple_workflow(x: int) -> dict:
    """
    简单的工作流示例，直接在函数式 API 中调用图 API

    Args:
        x: 输入数字

    Returns:
        处理结果
    """
    # 直接调用图 API 构建的状态图
    result = graph.invoke({"foo": x})
    return {"bar": result["foo"]}

def simple_demo():
    """简单混合 API 演示"""
```

```
print("\n=== 简单混合 API 演示 ===")

# 生成唯一线程 ID 用于状态保存
thread_id = str(uuid.uuid4())
config = {"configurable": {"thread_id": thread_id}}
print(f"工作流线程 ID: {thread_id}")

# 执行简单工作流
result = simple_workflow.invoke(5, config=config)
print(f"简单工作流结果: {result}") # 应该输出: {'bar': 25}

# 参考资料中的直接示例
@entrypoint(checkpointer=InMemorySaver())
def reference_workflow(x: int) -> dict:
    """
    参考资料中的示例工作流

    Args:
        x: 输入数字

    Returns:
        处理结果
    """
    result = graph.invoke({"foo": x})
    return {"bar": result["foo"]}

def reference_demo():
    """参考资料示例演示"""
    print("\n=== 参考资料示例演示 ===")

    # 生成唯一线程 ID 用于状态保存
    thread_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": thread_id}}
    print(f"工作流线程 ID: {thread_id}")

    # 执行参考资料中的工作流
    result = reference_workflow.invoke(5, config=config)
    print(f"参考资料工作流结果: {result}") # 应该输出: {'bar': 25}
```

```
if __name__ == "__main__":
    main()
    simple_demo()
    reference_demo()
```

5.4 调用其他入口点

可以在一个入口点或一个任务中调用其他入口点。

```
"""
基于 LangGraph 1.0 嵌套工作流的示例程序

本示例演示了如何在一个入口点或任务中调用其他入口点，
展示了 LangGraph 函数式 API 的嵌套工作流功能。

主要特性：
1. 使用@entrypoint 装饰器定义可复用的子工作流
2. 在主工作流中调用子工作流
3. 使用 InMemorySaver 进行状态持久化
"""

import uuid
from langgraph.func import entrypoint
from langgraph.checkpoint.memory import InMemorySaver

# 初始化检查点保存器
checkpointer = InMemorySaver()

# 可复用的子工作流 - 执行乘法运算
@entrypoint()
def multiply(inputs: dict) -> int:
    """
    执行乘法运算的子工作流

    Args:
        inputs: 包含操作数的字典，必须包含"a"和"b"键

    Returns:
        两个数的乘积
    """
    a = inputs["a"]
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
b = inputs["b"]
result = a * b
print(f"执行乘法运算: {a} × {b} = {result}")
return result

# 可复用的子工作流 - 执行加法运算
@entrypoint()
def add(inputs: dict) -> int:
    """
    执行加法运算的子工作流

    Args:
        inputs: 包含操作数的字典，必须包含"a"和"b"键

    Returns:
        两个数的和
    """
    a = inputs["a"]
    b = inputs["b"]
    result = a + b
    print(f"执行加法运算: {a} + {b} = {result}")
    return result

# 可复用的子工作流 - 执行幂运算
@entrypoint()
def power(inputs: dict) -> int:
    """
    执行幂运算的子工作流

    Args:
        inputs: 包含底数和指数的字典，必须包含"base"和"exp"键

    Returns:
        底数的指数次幂
    """
    base = inputs["base"]
    exp = inputs["exp"]
    result = base ** exp
    print(f"执行幂运算: {base} ^ {exp} = {result}")
```

```
    return result

# 主工作流 - 调用多个子工作流
@entrypoint(checkpointer=checkpointer)
def main(inputs: dict) -> dict:
    """
    主工作流，调用多个子工作流执行复杂计算

    Args:
        inputs: 包含计算参数的字典

    Returns:
        计算结果字典
    """
    x = inputs["x"]
    y = inputs["y"]
    z = inputs["z"]

    print(f"开始执行主工作流，输入参数: x={x}, y={y}, z={z}")

    # 调用乘法子工作流
    product = multiply.invoke({"a": x, "b": y})

    # 调用加法子工作流
    sum_result = add.invoke({"a": product, "b": z})

    # 调用幂运算子工作流
    power_result = power.invoke({"base": sum_result, "exp": 2})

    return {
        "product": product,          # x * y
        "sum": sum_result,           # (x * y) + z
        "power": power_result,       # ((x * y) + z) ^ 2
        "final_result": power_result # 最终结果
    }

# 更复杂的嵌套示例 - 递归调用工作流
@entrypoint(checkpointer=checkpointer)
def factorial(inputs: dict) -> int:
```

```
"""
    计算阶乘的递归工作流

    Args:
        inputs: 包含数字 n 的字典

    Returns:
        n 的阶乘
    """
n = inputs["n"]

# 基础情况
if n <= 1:
    print(f"阶乘基础情况: {n}! = 1")
    return 1

# 递归情况
print(f"计算阶乘: {n}! = {n} × {n-1}!")
sub_result = factorial.invoke({"n": n - 1})
result = n * sub_result
print(f"阶乘结果: {n}! = {result}")
return result

# 组合工作流 - 调用阶乘工作流
@entrypoint(checkpointer=checkpointer)
def combination_workflow(inputs: dict) -> dict:
    """
        组合工作流，计算多个数的阶乘

        Args:
            inputs: 包含数字列表的字典

        Returns:
            阶乘计算结果字典
    """
    numbers = inputs["numbers"]
    print(f"开始计算数字列表 {numbers} 的阶乘")

    # 并行计算每个数的阶乘
    factorial_results = []
```

```
for num in numbers:
    fact = factorial.invoke({"n": num})
    factorial_results.append(fact)

return {
    "input_numbers": numbers,
    "factorials": factorial_results,
    "results_dict": dict(zip(numbers, factorial_results))
}

def main_demo():
    """主 workflow 演示"""
    print("=== LangGraph 1.0 嵌套 workflow 演示 ===")

    # 生成唯一线程 ID 用于状态保存
    thread_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": thread_id}}
    print(f"主 workflow 线程 ID: {thread_id}")

    # 执行主 workflow
    print("\n--- 执行主 workflow ---")
    inputs = {"x": 6, "y": 7, "z": 5}
    result = main.invoke(inputs, config=config)

    # 输出结果
    print("\n--- 计算结果 ---")
    print(f"输入: x={inputs['x']}, y={inputs['y']}, z={inputs['z']}")
    print(f"乘法结果 (x × y): {result['product']}")
    print(f"加法结果 ((x × y) + z): {result['sum']}")
    print(f"幂运算结果 (((x × y) + z) ^ 2): {result['power']}")
    print(f"最终结果: {result['final_result']}")

def factorial_demo():
    """阶乘 workflow 演示"""
    print("\n\n=== 阶乘 workflow 演示 ===")

    # 生成唯一线程 ID 用于状态保存
    thread_id = str(uuid.uuid4())
    config = {"configurable": {"thread_id": thread_id}}
```

```
print(f"阶乘工作流线程 ID: {thread_id}")

# 执行组合工作流
print("\n--- 执行组合工作流 ---")
inputs = {"numbers": [3, 4, 5]}
result = combination_workflow.invoke(inputs, config=config)

# 输出结果
print("\n--- 阶乘计算结果 ---")
print(f"输入数字: {result['input_numbers']}")
print(f"阶乘结果: {result['factorials']}")
print("详细结果:")
for num, fact in result['results_dict'].items():
    print(f" {num}! = {fact}")

if __name__ == "__main__":
    main_demo() #在主工作流中调用多个子工作流执行复杂计算
    factorial_demo() #递归调用工作流的功能
```

5.5 流式传输

Functional API 采用与 Graph API 相同的流机制。

```
"""
基于 LangGraph 1.0 函数式 API 的流式处理示例

本示例演示了如何使用 LangGraph 函数式 API 的流式处理机制，
展示了与 Graph API 相同的流机制。

主要特性：
1. 使用@entrypoint 装饰器定义支持流式处理的工作流
2. 使用 get_stream_writer 获取流写入器
3. 通过流写入器发送自定义流数据
4. 使用 stream_mode 参数控制流式输出模式
"""

from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.config import get_stream_writer
from langgraph.types import StreamWriter

# 初始化检查点保存器
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
checkpointner = InMemorySaver()

# 定义一个任务函数 - 模拟耗时操作
@task
def process_data(data: str) -> str:
    """
    处理数据的任务函数

    Args:
        data: 输入数据

    Returns:
        处理后的数据
    """
    # 模拟数据处理
    processed = f"[已处理] {data}"
    return processed

# 定义支持流式处理的主工作流
@entrypoint(checkpointer=checkpointner)
def main(inputs: dict, writer: StreamWriter) -> dict:
    """
    支持流式处理的主工作流

    Args:
        inputs: 输入数据字典

    Returns:
        处理结果字典
    """

    # 发送开始处理消息
    writer("开始处理数据")

    # 获取输入数据
    x = inputs["x"]

    # 发送处理进度消息
    writer(f"正在处理数据: {x}")

    # 执行处理操作
    result = x * 2
```

更多 [Java - 大数据](#) [- 前端](#) [- python 人工智能](#) 资料下载，可百度访问：[尚硅谷官网](#)

```
# 发送中间结果消息
writer(f"中间结果: {result}")

# 使用任务函数处理数据
processed_data = process_data(f"数据_{x}").result()

# 发送最终结果消息
writer(f"处理完成, 最终结果: {result}")

return {
    "input": x,
    "output": result,
    "processed_data": processed_data
}

# 更复杂的流式处理示例
@entrypoint(checkpointer=checkpointer)
def complex_workflow(inputs: dict) -> dict:
    """
    复杂的流式处理工作流

    Args:
        inputs: 输入数据字典

    Returns:
        处理结果字典
    """
    # 获取流写入器
    writer = get_stream_writer()

    # 发送开始消息
    writer("启动复杂工作流")

    # 获取输入数据
    numbers = inputs["numbers"]
    writer(f"待处理数字列表: {numbers}")

    # 逐步处理每个数字
    results = []
    for i, num in enumerate(numbers):
        writer(f"正在处理第 {i + 1} 个数字: {num}")
```

```
        processed = num ** 2 # 平方运算
        results.append(processed)
        writer(f"第 {i + 1} 个数字处理完成，结果: {processed}")

# 计算总和
total = sum(results)
writer(f"所有数字处理完成，总和: {total}")

return {
    "input_numbers": numbers,
    "squared_numbers": results,
    "total": total
}

def main_demo():
    """主工作流演示"""
    print("=== LangGraph 1.0 函数式 API 流式处理演示 ===")

    # 配置
    config = {"configurable": {"thread_id": "main-demo-123"}}

    print("\n--- 执行主工作流并流式输出 ---")
    # 流式执行工作流
    for mode, chunk in main.stream(
        {"x": 5},
        stream_mode=["custom", "values"],
        config=config
    ):
        print(f"[{mode}]: {chunk}")

# 并行任务流式处理示例
@task
def square_task(x: int) -> int:
    """
    计算平方的任务

    Args:
        x: 输入数字

    Returns:
        平方结果
    """
    return x ** 2
```

```
@task
def cube_task(x: int) -> int:
    """
    计算立方的任务

    Args:
        x: 输入数字

    Returns:
        立方结果
    """
    return x ** 3


@entrypoint(checkpointer=checkpointer)
def parallel_workflow(inputs: dict) -> dict:
    """
    并行任务流式处理工作流

    Args:
        inputs: 输入数据字典

    Returns:
        处理结果字典
    """
    # 获取流写入器
    writer = get_stream_writer()
    writer("启动并行任务工作流")

    # 获取输入数据
    num = inputs["number"]
    writer(f"待处理数字: {num}")

    # 并行启动任务
    writer("开始并行执行平方和立方计算任务")
    square_future = square_task(num)
    cube_future = cube_task(num)

    # 等待任务完成
    square_result = square_future.result()
    cube_result = cube_future.result()
```

```
writer(f"平方计算结果: {square_result}")
writer(f"立方计算结果: {cube_result}")

return {
    "input": num,
    "square": square_result,
    "cube": cube_result
}

def parallel_demo():
    """并行任务工作流演示"""
    print("\n\n=== 并行任务流式处理演示 ===")

    config = {"configurable": {"thread_id": "parallel-demo-789"}}

    print("\n--- 执行并行任务工作流并流式输出 ---")
    for mode, chunk in parallel_workflow.stream(
        {"number": 3},
        stream_mode=["custom", "updates"],
        config=config
    ):
        print(f"[{mode}]: {chunk}")

if __name__ == "__main__":
    # 执行所有演示
    main_demo() # 基本的流式处理功能
    parallel_demo() # 并行任务流式处理功能
```

5.6 重试策略

"""
基于 LangGraph 1.0 函数式 API 的重试策略示例

本示例演示了如何在 LangGraph 函数式 API 中配置和使用重试策略，展示了节点执行失败时的自动重试机制。

主要特性：

1. 使用 `RetryPolicy` 配置重试策略
2. 使用 `@task` 装饰器定义带重试策略的任务函数
3. 模拟网络故障并演示重试机制
4. 使用 `InMemorySaver` 进行状态持久化

"""

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint, task
from langgraph.types import RetryPolicy
import time
import random

# 全局变量仅用于演示目的，模拟网络故障
# 实际代码中不会使用这样的变量
attempts = 0
api_call_attempts = 0
database_attempts = 0

# 配置重试策略以重试 ValueError 异常
# 默认的 RetryPolicy 针对特定的网络错误进行了优化
retry_policy = RetryPolicy(retry_on=ValueError)

@task(retry_policy=retry_policy)
def get_info():
    """
    模拟可能失败的信息获取任务

    Returns:
        成功时返回 "OK"

    Raises:
        ValueError: 模拟网络故障
    """
    global attempts
    attempts += 1
    print(f"尝试获取信息，第 {attempts} 次尝试")

    # 模拟第一次调用失败
    if attempts < 2:
        print("发生网络故障，抛出 ValueError 异常")
        raise ValueError('网络连接失败')

    print("信息获取成功")
```

```
    return "OK"

# 更实际的重试策略示例 - 模拟 API 调用
api_retry_policy = RetryPolicy(
    retry_on=(ConnectionError, TimeoutError) # 指定重试的异常类型
    # 注意: LangGraph 1.0 的 RetryPolicy 可能不支持 interval、max_attempts 等
    参数
)

@task(retry_policy=api_retry_policy)
def call_external_api(data: str) -> dict:
    """
    模拟调用外部 API 的任务

    Args:
        data: 请求数据

    Returns:
        API 响应结果

    Raises:
        ConnectionError: 模拟网络连接错误
        TimeoutError: 模拟超时错误
    """
    global api_call_attempts
    api_call_attempts += 1
    print(f"调用外部 API, 第 {api_call_attempts} 次尝试")

    # 模拟前两次调用都失败
    if api_call_attempts < 3:
        # 随机抛出不同类型的异常
        error_type = random.choice([ConnectionError, TimeoutError])
        if error_type == ConnectionError:
            print("API 调用失败: 连接错误")
            raise ConnectionError("无法连接到 API 服务器")
        else:
            print("API 调用失败: 请求超时")
            raise TimeoutError("API 请求超时")
```

```
print("API 调用成功")
return {
    "status": "success",
    "data": f"处理后的数据: {data}",
    "attempts": api_call_attempts
}

# 数据库操作重试策略示例
db_retry_policy = RetryPolicy(
    retry_on=(Exception,) # 重试所有异常
    # 注意: LangGraph 1.0 的 RetryPolicy 可能不支持 interval、max_attempts 等
    参数
)

@task(retry_policy=db_retry_policy)
def database_operation(query: str) -> str:
    """
    模拟数据库操作任务

    Args:
        query: 查询语句

    Returns:
        查询结果

    Raises:
        Exception: 模拟数据库异常
    """
    global database_attempts
    database_attempts += 1
    print(f"执行数据库操作, 第 {database_attempts} 次尝试")

    # 模拟前几次调用都失败
    if database_attempts < 4:
        print("数据库操作失败: 连接超时")
        raise Exception("数据库连接超时")

    print("数据库操作成功")
    return f"查询结果: {query} -> 成功"
```



```
# 检查点保存器
checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def main(inputs: dict) -> dict:
    """
    主工作流

    Args:
        inputs: 输入数据字典

    Returns:
        处理结果字典
    """
    print("启动主工作流")

    # 执行带重试策略的任务
    info_result = get_info().result()

    # 执行外部 API 调用任务
    api_result = call_external_api(inputs.get("data", "默认数据
    ")).result()

    # 执行数据库操作任务
    db_result = database_operation(inputs.get("query", "SELECT * FROM
    users")).result()

    return {
        "info": info_result,
        "api_result": api_result,
        "database_result": db_result
    }

# 参考资料中的直接示例
attempts_ref = 0

retry_policy_ref = RetryPolicy(retry_on=ValueError)
```

```
@task(retry_policy=retry_policy_ref)
def get_info_ref():
    global attempts_ref
    attempts_ref += 1

    if attempts_ref < 2:
        raise ValueError('Failure')
    return "OK"

def main_demo():
    """主演示函数"""
    print("=== LangGraph 1.0 函数式 API 重试策略演示 ===")

    config = {
        "configurable": {
            "thread_id": "retry-demo-1"
        }
    }

    try:
        print("\n--- 执行主工作流 ---")
        result = main.invoke({
            "data": "用户信息",
            "query": "SELECT * FROM users WHERE id=1"
        }, config=config)

        print("\n--- 执行结果 ---")
        print(f"信息获取结果: {result['info']}")
        print(f"API 调用结果: {result['api_result']}")
        print(f"数据库操作结果: {result['database_result']}")

    except Exception as e:
        print(f"工作流执行失败: {e}")

# 自定义重试条件示例
def should_retry(exception):
    """
    自定义重试条件函数
    """
```

```
Args:
    exception: 捕获的异常

Returns:
    bool: 是否应该重试
"""
# 只有当异常包含特定文本时才重试
return "temporary" in str(exception).lower()

custom_retry_policy = RetryPolicy(
    retry_on=should_retry  # 使用自定义重试条件
)

@task(retry_policy=custom_retry_policy)
def custom_retry_task(input_data: str) -> str:
    """
    使用自定义重试条件的任务

    Args:
        input_data: 输入数据

    Returns:
        处理结果

    Raises:
        Exception: 模拟异常
    """
    print(f"执行自定义重试任务，输入: {input_data}")

    # 模拟临时性错误和永久性错误
    if "temporary" in input_data:
        raise Exception("Temporary network issue")
    elif "permanent" in input_data:
        raise Exception("Permanent failure - should not retry")
    else:
        return "任务执行成功"
```

```
@entrypoint(checkpointer=checkpointer)
def custom_retry_workflow(inputs: dict) -> str:
    """
    自定义重试策略工作流

    Args:
        inputs: 输入数据

    Returns:
        处理结果
    """
    print("启动自定义重试策略工作流")
    result = custom_retry_task(inputs["data"]).result()
    return result

def custom_retry_demo():
    """自定义重试策略演示"""
    print("\n\n=== 自定义重试策略演示 ===")

    config = {
        "configurable": {
            "thread_id": "custom-retry-demo"
        }
    }

    # 测试临时性错误（应该重试）
    print("\n--- 测试临时性错误（应该重试） ---")
    try:
        result = custom_retry_workflow.invoke({"data": "temporary error"},
config=config)
        print(f"临时性错误处理结果: {result}")
    except Exception as e:
        print(f"临时性错误处理失败: {e}")

    # 重置全局状态
    global attempts, api_call_attempts, database_attempts, attempts_ref
    attempts = 0
    api_call_attempts = 0
    database_attempts = 0
    attempts_ref = 0
```

```
# 测试永久性错误（不应该重试）
print("\n--- 测试永久性错误（不应该重试） ---")
try:
    result = custom_retry_workflow.invoke({"data": "permanent error"},
config=config)
    print(f"永久性错误处理结果: {result}")
except Exception as e:
    print(f"永久性错误处理失败（符合预期）: {e}")

if __name__ == "__main__":
    main_demo()
    custom_retry_demo()
```

5.7 缓存任务

"""

基于 LangGraph 1.0 函数式 API 的缓存任务示例

本示例演示了如何在 LangGraph 函数式 API 中配置和使用缓存策略，展示了任务结果的缓存机制以提高性能。

主要特性：

1. 使用 CachePolicy 配置任务缓存策略
2. 使用 InMemoryCache 作为缓存存储
3. 使用@task 装饰器定义带缓存策略的任务函数
4. 使用@entrypoint 装饰器定义带缓存的工作流入口点

"""

```
import time
from langgraph.cache.memory import InMemoryCache
from langgraph.func import entrypoint, task
from langgraph.types import CachePolicy

# 定义带缓存策略的任务函数
@task(cache_policy=CachePolicy(ttl=120)) # 设置 120 秒的缓存时间
def slow_add(x: int) -> int:
    """
    模拟耗时的加法运算任务
    """
```

```
Args:
    x: 输入数字

Returns:
    输入数字的两倍
"""
print(f"执行耗时运算: {x} * 2")
time.sleep(1) # 模拟耗时操作
result = x * 2
print(f"运算完成, 结果: {result}")
return result

# 定义带缓存的工作流入口点
@entrypoint(cache=InMemoryCache())
def main(inputs: dict) -> dict[str, int]:
    """
    主工作流, 演示缓存机制

    Args:
        inputs: 输入数据字典

    Returns:
        包含两次运算结果的字典
    """
    print("启动主工作流")

    # 第一次调用 slow_add 任务
    print("第一次调用 slow_add 任务")
    result1 = slow_add(inputs["x"]).result()

    # 第二次调用 slow_add 任务 (应该从缓存中获取结果)
    print("第二次调用 slow_add 任务")
    result2 = slow_add(inputs["x"]).result()

    return {"result1": result1, "result2": result2}

# 更复杂的缓存示例
@task(cache_policy=CachePolicy(ttl=60)) # 设置 60 秒的缓存时间
def complex_calculation(data: dict) -> dict:
```

```
"""
复杂计算任务

Args:
    data: 输入数据字典

Returns:
    计算结果字典
"""

print(f"执行复杂计算: {data}")
time.sleep(2) # 模拟复杂计算

# 执行一些计算
result = {
    "sum": sum(data.values()),
    "count": len(data),
    "average": sum(data.values()) / len(data) if data else 0
}

print(f"复杂计算完成, 结果: {result}")
return result

@entrypoint(cache=InMemoryCache())
def complex_workflow(inputs: dict) -> dict:
    """
    复杂工作流, 演示更复杂的缓存场景

    Args:
        inputs: 输入数据字典

    Returns:
        处理结果字典
    """

    print("启动复杂工作流")

    # 第一次调用复杂计算任务
    print("第一次调用复杂计算任务")
    result1 = complex_calculation(inputs["data"]).result()

    # 第二次调用复杂计算任务 (应该从缓存中获取结果)
```

```
print("第二次调用复杂计算任务")
result2 = complex_calculation(inputs["data"]).result()

# 修改输入数据后再次调用
modified_data = {k: v + 1 for k, v in inputs["data"].items()}
print(f"使用修改后的数据调用复杂计算任务: {modified_data}")
result3 = complex_calculation(modified_data).result()

return {
    "first_call": result1,
    "second_call": result2,
    "third_call": result3
}

def main_demo():
    """主演示函数"""
    print("=== LangGraph 1.0 函数式 API 缓存任务演示 ===")

    print("\n--- 执行主工作流 ---")
    # 流式执行主工作流
    for chunk in main.stream({"x": 5}, stream_mode="updates"):
        print(f"流式输出: {chunk}")

def complex_demo():
    """复杂缓存演示"""
    print("\n\n=== 复杂缓存演示 ===")

    print("\n--- 执行复杂工作流 ---")
    # 流式执行复杂工作流
    for chunk in complex_workflow.stream(
        {"data": {"a": 10, "b": 20, "c": 30}},
        stream_mode="updates"
    ):
        print(f"流式输出: {chunk}")

# 带有过期时间的缓存示例
@task(cache_policy=CachePolicy(ttl=3)) # 设置 3 秒的缓存时间
```



```
def short_ttl_task(x: int) -> int:
    """
    短缓存时间任务

    Args:
        x: 输入数字

    Returns:
        输入数字的平方
    """
    print(f"执行短缓存时间任务: {x}^2")
    time.sleep(0.5) # 模拟较短的耗时操作
    result = x ** 2
    print(f"短缓存时间任务完成, 结果: {result}")
    return result

@entrypoint(cache=InMemoryCache())
def ttl_workflow(inputs: dict) -> dict:
    """
    演示缓存过期的工作流

    Args:
        inputs: 输入数据字典

    Returns:
        处理结果字典
    """
    print("启动缓存过期演示工作流")

    # 第一次调用
    print("第一次调用短缓存时间任务")
    result1 = short_ttl_task(inputs["x"]).result()

    # 等待缓存过期
    print("等待缓存过期 (4 秒) ...")
    time.sleep(4)

    # 第二次调用 (缓存已过期, 需要重新计算)
    print("第二次调用短缓存时间任务 (缓存已过期)")
    result2 = short_ttl_task(inputs["x"]).result()
```

```
    return {
        "first_result": result1,
        "second_result": result2
    }

def ttl_demo():
    """缓存过期演示"""
    print("\n\n=== 缓存过期演示 ===")

    print("\n--- 执行缓存过期工作流 ---")
    for chunk in ttl_workflow.stream({"x": 3}, stream_mode="updates"):
        print(f"流式输出: {chunk}")

if __name__ == "__main__":
    # 执行所有演示
    main_demo()
    complex_demo()
    ttl_demo()
```

5.8 人机协同

"""
基于 LangGraph 1.0 函数式 API 的人机协同工作流示例

本示例演示了如何在 LangGraph 函数式 API 中实现人机协同工作流，展示了使用 `interrupt` 函数和 `Command` 原语实现人机交互的机制。

主要特性：

1. 使用 `interrupt` 函数实现人机交互中断
2. 使用 `Command` 原语恢复工作流执行
3. 使用 `InMemorySaver` 进行状态持久化
4. 实现任务间的顺序执行和人机协同

"""

```
from langgraph.func import entrypoint, task
from langgraph.types import Command, interrupt
from langgraph.checkpoint.memory import InMemorySaver
```

@task

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
def step_1(input_query):
    """
    第一步任务：追加"bar"

    参数：
        input_query: 输入查询字符串

    返回：
        追加"bar"后的字符串
    """
    print(f"执行第一步任务，输入：{input_query}")
    result = f"{input_query} bar"
    print(f"第一步任务完成，结果：{result}")
    return result

@task
def human_feedback(input_query):
    """
    人工反馈任务：暂停以等待人工输入，恢复时附加人工输入

    参数：
        input_query: 输入查询字符串

    返回：
        追加人工反馈后的字符串
    """
    print(f"执行人工反馈任务，输入：{input_query}")
    # 中断执行，等待人工输入
    feedback = interrupt(f"请提供反馈：{input_query}")
    result = f"{input_query} {feedback}"
    print(f"人工反馈任务完成，结果：{result}")
    return result

@task
def step_3(input_query):
    """
    第三步任务：追加"qux"

    参数：
```

```
        input_query: 输入查询字符串

    返回:
        追加"qux"后的字符串
    """
    print(f"执行第三步任务，输入: {input_query}")
    result = f"{input_query} qux"
    print(f"第三步任务完成，结果: {result}")
    return result

# 初始化检查点保存器
checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def graph(input_query):
    """
    主工作流：组合三个任务的执行

    参数:
        input_query: 输入查询字符串

    返回:
        最终处理结果
    """
    print("启动主工作流")

    # 顺序执行三个任务
    result_1 = step_1(input_query).result()
    result_2 = human_feedback(result_1).result()
    result_3 = step_3(result_2).result()

    return result_3

def main_demo():
    """主演示函数"""
    print("=== LangGraph 1.0 函数式 API 人机协同工作流演示 ===")

    # 配置工作流执行参数
```

```
config = {"configurable": {"thread_id": "human-collaboration-demo-1"}}

print("\n--- 启动 workflows 执行 ---")
# 启动 workflows 执行
for event in graph.stream("foo", config):
    print(f"workflows 事件: {event}")
    print()

print("--- workflows 在 human_feedback 任务处中断, 等待人工输入 ---")

print("\n--- 恢复 workflows 执行, 提供人工输入 'baz' ---")
# 继续执行 workflows, 提供人工输入
for event in graph.stream(Command(resume="baz"), config):
    print(f"workflows 事件: {event}")
    print()

print("--- workflows 执行完成 ---")

# 更复杂的人机协同示例
@task
def data_processing(input_data):
    """
    数据处理任务

    参数:
        input_data: 输入数据

    返回:
        处理后的数据
    """
    print(f"执行数据处理任务, 输入: {input_data}")
    processed_data = f"已处理数据: {input_data}"
    print(f"数据处理完成: {processed_data}")
    return processed_data

@task
def approval_process(input_data):
    """
    审批流程任务: 等待人工审批
```

```
    参数:
        input_data: 待审批的数据

    返回:
        审批结果
    """
    print(f"执行审批流程任务，待审批数据: {input_data}")
    # 等待人工审批
    approval_result = interrupt(f"请审批以下数据:\n{input_data}\n 输入
'approved' 批准或'rejected' 拒绝:")

    if approval_result.lower() == "approved":
        result = f"{input_data} [已批准]"
        print(f"审批通过: {result}")
    elif approval_result.lower() == "rejected":
        result = f"{input_data} [已拒绝]"
        print(f"审批拒绝: {result}")
    else:
        result = f"{input_data} [审批结果未知: {approval_result}]"
        print(f"未知审批结果: {result}")

    return result

@task
def finalization(input_data):
    """
    最终化任务

    参数:
        input_data: 输入数据

    返回:
        最终结果
    """
    print(f"执行最终化任务，输入: {input_data}")
    result = f"最终结果: {input_data}"
    print(f"最终化完成: {result}")
    return result
```

```
complex_checkpointer = InMemorySaver()

@entrypoint(checkpointer=complex_checkpointer)
def complex_workflow(initial_data):
    """
    复杂人机协同工作流

    参数:
        initial_data: 初始数据

    返回:
        最终处理结果
    """
    print("启动复杂人机协同工作流")

    # 顺序执行任务
    processed_data = data_processing(initial_data).result()
    approved_data = approval_process(processed_data).result()
    final_result = finalization(approved_data).result()

    return final_result

def complex_demo():
    """复杂人机协同演示"""
    print("\n\n=== 复杂人机协同演示 ===")

    config = {"configurable": {"thread_id":
"complex-human-collaboration-demo"}}

    print("\n--- 启动复杂工作流执行 ---")
    # 启动工作流执行
    for event in complex_workflow.stream("原始数据", config):
        print(f"工作流事件: {event}")
        print()

    print("--- 工作流在审批流程处中断, 等待人工审批 ---")

    print("\n--- 恢复工作流执行, 提供审批输入 'approved' ---")
    # 继续执行工作流, 提供审批结果
```

```
    for event in complex_workflow.stream(Command(resume="approved"),
config):
        print(f" workflow事件: {event}")
        print()

    print(" --- 复杂 workflow 执行完成 ---")

# 多轮人机交互示例
@task
def initial_task(input_data):
    """初始任务"""
    print(f"执行初始任务: {input_data}")
    return f"初始处理: {input_data}"

@task
def first_interrupt(input_data):
    """第一次人工交互"""
    print(f"第一次人工交互任务: {input_data}")
    feedback = interrupt(f"第一次交互, 请提供反馈 [{input_data}]:")
    return f"{input_data} + 反馈 1[{feedback}]"

@task
def middle_task(input_data):
    """中间任务"""
    print(f"执行中间任务: {input_data}")
    return f"中间处理: {input_data}"

@task
def second_interrupt(input_data):
    """第二次人工交互"""
    print(f"第二次人工交互任务: {input_data}")
    feedback = interrupt(f"第二次交互, 请提供反馈 [{input_data}]:")
    return f"{input_data} + 反馈 2[{feedback}]"

@task
def final_task(input_data):
    """最终任务"""
```



```
print(f"执行最终任务: {input_data}")
return f"最终结果: {input_data}"

multi_checkpointer = InMemorySaver()

@entrypoint(checkpointer=multi_checkpointer)
def multi_interrupt_workflow(initial_data):
    """
    多轮人机交互 workflow

    参数:
        initial_data: 初始数据

    返回:
        最终处理结果
    """
    print("启动多轮人机交互 workflow")

    # 顺序执行任务
    result1 = initial_task(initial_data).result()
    result2 = first_interrupt(result1).result()
    result3 = middle_task(result2).result()
    result4 = second_interrupt(result3).result()
    result5 = final_task(result4).result()

    return result5

def multi_interrupt_demo():
    """多轮人机交互演示"""
    print("\n\n=== 多轮人机交互演示 ===")

    config = {"configurable": {"thread_id": "multi-interrupt-demo"}}

    print("\n--- 启动多轮交互 workflow 执行 ---")
    # 启动 workflow 执行
    for event in multi_interrupt_workflow.stream("开始数据", config):
        print(f"workflow 事件: {event}")
        print()
```

```
print("---  workflow在第一次交互处中断 ---")

print("\n---  恢复workflow执行，提供第一次反馈 ---")
# 第一次恢复执行
for event in multi_interrupt_workflow.stream(Command(resume="第一次反馈内容"), config):
    print(f"workflow事件: {event}")
    print()

print("---  workflow在第二次交互处中断 ---")

print("\n---  恢复workflow执行，提供第二次反馈 ---")
# 第二次恢复执行
for event in multi_interrupt_workflow.stream(Command(resume="第二次反馈内容"), config):
    print(f"workflow事件: {event}")
    print()

print("---  多轮交互workflow执行完成 ---")

if __name__ == "__main__":
    # 执行所有演示
    main_demo()
    complex_demo()
    multi_interrupt_demo()
```

5.9 短期记忆

短期记忆允许在同一**线程 ID** 的不同**调用**之间存储信息。

"""
基于 LangGraph 1.0 函数式 API 的短期记忆示例

本示例演示了如何在 LangGraph 函数式 API 中使用短期记忆功能，展示了检查点机制和状态管理。

主要特性：

1. 使用 InMemorySaver 实现检查点存储
2. 使用@entrypoint 装饰器定义带检查点的工作流入口点
3. 展示状态查看和历史记录功能
4. 演示线程状态的持久化和恢复

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
"""

from typing import List, Optional, Dict, Any
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import add_messages
import time

# 简单的累加器示例
@entrypoint(checkpointer=InMemorySaver())
def accumulate(n: int, *, previous: Optional[int] = None) ->
    entrypoint.final[int, int]:
    """
        累加器示例，展示如何解耦返回值和保存值

        参数:
            n: 当前输入数字
            previous: 之前的累加值

        返回:
            entrypoint.final 对象，包含返回值和保存值
    """
    print(f"执行累加任务，输入: {n}")

    # 获取之前的值，如果没有则为 0
    previous = previous or 0
    print(f"之前的累加值: {previous}")

    # 计算新的总和
    total = previous + n
    print(f"新的累加值: {total}")

    # 返回之前的值给调用者，但保存新的总和到检查点
    return entrypoint.final(value=previous, save=total)

def accumulate_demo():
    """累加器演示"""
    print("=== LangGraph 1.0 函数式 API 累加器演示 ===")
```

```
# 配置 workflow 执行参数
config = {"configurable": {"thread_id": "accumulate-thread"}}

print("\n--- 第一次调用累加器 ---")
result1 = accumulate.invoke(1, config=config)
print(f"第一次调用结果: {result1}")

print("\n--- 第二次调用累加器 ---")
result2 = accumulate.invoke(2, config=config)
print(f"第二次调用结果: {result2}")

print("\n--- 第三次调用累加器 ---")
result3 = accumulate.invoke(3, config=config)
print(f"第三次调用结果: {result3}")

print("\n--- 查看累加器线程状态 ---")
# 查看当前线程状态
current_state = accumulate.get_state(config)
print(f"当前状态值: {current_state.values}")
print(f"元数据: {current_state.metadata}")

print("\n--- 查看累加器历史记录 ---")
# 查看线程历史记录
history = list(accumulate.get_state_history(config))
print(f"历史记录条数: {len(history)}")

for i, state in enumerate(history):
    print(f"\n 历史状态 {i+1}:")
    print(f"    值: {state.values}")
    print(f"    创建时间: {state.created_at}")
    print(f"    步骤: {state.metadata.get('step', 'N/A')}")

# 状态管理示例
@entrypoint(checkpointer=InMemorySaver())
def state_manager(data: Dict[str, Any], *, previous: Optional[Dict[str, Any]] = None) -> entrypoint.final[Dict[str, Any], Dict[str, Any]]:
    """
    状态管理示例

    参数:
```

```
        data: 当前数据
        previous: 之前的状态

    返回:
        entrypoint.final 对象, 包含返回值和保存值
    """
    print(f"处理数据: {data}")

    # 如果有之前的状态, 则合并
    if previous:
        print(f"之前状态: {previous}")
        # 合并状态
        new_state = {**previous, **data}
    else:
        new_state = data

    print(f"新状态: {new_state}")

    # 返回新状态给调用者, 同时保存到检查点
    return entrypoint.final(value=new_state, save=new_state)

def state_management_demo():
    """状态管理演示"""
    print("\n\n=== 状态管理演示 ===")

    config = {"configurable": {"thread_id": "state-management-thread"}}

    # 第一次调用
    print("\n--- 第一次调用状态管理器 ---")
    data1 = {"name": "张三", "age": 25}
    result1 = state_manager.invoke(data1, config=config)
    print(f"第一次调用结果: {result1}")

    # 第二次调用
    print("\n--- 第二次调用状态管理器 ---")
    data2 = {"city": "北京", "occupation": "工程师"}
    result2 = state_manager.invoke(data2, config=config)
    print(f"第二次调用结果: {result2}")

    # 第三次调用
```

```
print("\n--- 第三次调用状态管理器 ---")
data3 = {"age": 26, "hobby": "编程"}
result3 = state_manager.invoke(data3, config=config)
print(f"第三次调用结果: {result3}")

print("\n--- 查看状态管理线程状态 ---")
current_state = state_manager.get_state(config)
print(f"当前状态值: {current_state.values}")

print("\n--- 查看状态管理历史记录 ---")
history = list(state_manager.get_state_history(config))
print(f"历史记录条数: {len(history)}")

for i, state in enumerate(history):
    print(f"\n 历史状态 {i+1}:")
    print(f"    值: {state.values}")
    print(f"    创建时间: {state.created_at}")
    print(f"    步骤: {state.metadata.get('step', 'N/A')}")

# 对话历史管理示例
@entrypoint(checkpointer=InMemorySaver())
def conversation_history(message: str, *, previous: Optional[List[str]] =
None) -> entrypoint.final[str, List[str]]:
    """
    对话历史管理示例

    参数:
        message: 当前消息
        previous: 对话历史

    返回:
        entrypoint.final 对象, 包含返回值和保存值
    """
    print(f"收到消息: {message}")

    # 初始化历史记录
    history = previous or []

    print(f"当前历史记录: {history}")
```

```
# 添加当前消息到历史记录
updated_history = history + [message]

# 生成回复（简单模拟）
if len(updated_history) == 1:
    response = f"你好！我收到了你的消息：'{message}'"
else:
    response = f"你之前说过：{'，'.join(history[-2:])}。现在你说：'{message}'"

print(f"生成回复：{response}")

# 返回回复给调用者，保存更新后的历史记录
return entrypoint.final(value=response, save=updated_history)

def conversation_demo():
    """对话历史演示"""
    print("\n\n=== 对话历史管理演示 ===")

    config = {"configurable": {"thread_id": "conversation-thread"}}

    # 第一轮对话
    print("\n--- 第一轮对话 ---")
    message1 = "你好，我是李四"
    response1 = conversation_history.invoke(message1, config=config)
    print(f"用户：{message1}")
    print(f"AI：{response1}")

    # 第二轮对话
    print("\n--- 第二轮对话 ---")
    message2 = "我想了解一下人工智能"
    response2 = conversation_history.invoke(message2, config=config)
    print(f"用户：{message2}")
    print(f"AI：{response2}")

    # 第三轮对话
    print("\n--- 第三轮对话 ---")
    message3 = "能推荐一些学习资源吗？"
    response3 = conversation_history.invoke(message3, config=config)
    print(f"用户：{message3}")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
print(f"AI: {response3}")

print("\n--- 查看对话历史线程状态 ---")
current_state = conversation_history.get_state(config)
print(f"当前状态值: {current_state.values}")

print("\n--- 查看对话历史记录 ---")
history = list(conversation_history.get_state_history(config))
print(f"历史记录条数: {len(history)}")

for i, state in enumerate(history):
    print(f"\n 历史状态 {i+1}:")
    print(f"    值: {state.values}")
    print(f"    创建时间: {state.created_at}")
    print(f"    步骤: {state.metadata.get('step', 'N/A')}")

# 复杂状态管理示例
@entrypoint(checkpointer=InMemorySaver())
def complex_state_manager(updates: Dict[str, Any], *, previous:
Optional[Dict[str, Any]] = None) -> entrypoint.final[Dict[str, Any],
Dict[str, Any]]:
    """
    复杂状态管理示例

    参数:
        updates: 状态更新
        previous: 当前状态

    返回:
        entrypoint.final 对象, 包含返回值和保存值
    """
    print(f"收到状态更新: {updates}")

# 初始化状态
if previous is None:
    previous = {
        "user_info": {},
        "preferences": {},
        "interactions": 0,
        "last_updated": None
    }
```



```
    }

    print(f"当前状态: {previous}")

    # 更新用户信息
    if "user_info" in updates:
        previous["user_info"].update(updates["user_info"])

    # 更新偏好设置
    if "preferences" in updates:
        previous["preferences"].update(updates["preferences"])

    # 增加交互次数
    previous["interactions"] += 1

    # 更新时间戳
    previous["last_updated"] = time.time()

    # 生成状态报告
    report = {
        "message": "状态更新成功",
        "interactions": previous["interactions"],
        "user_name": previous["user_info"].get("name", "未知用户")
    }

    print(f"更新后状态: {previous}")
    print(f"生成报告: {report}")

    # 返回报告给调用者，保存完整状态
    return entrypoint.final(value=report, save=previous)

def complex_state_demo():
    """复杂状态管理演示"""
    print("\n\n=== 复杂状态管理演示 ===")

    config = {"configurable": {"thread_id": "complex-state-thread"}}

    # 第一次状态更新
    print("\n--- 第一次状态更新 ---")
    update1 = {
```

```
"user_info": {"name": "王五", "email": "wangwu@example.com"},
"preferences": {"theme": "dark", "language": "zh-CN"}
}
result1 = complex_state_manager.invoke(update1, config=config)
print(f"更新内容: {update1}")
print(f"返回结果: {result1}")

# 第二次状态更新
print("\n--- 第二次状态更新 ---")
update2 = {
    "preferences": {"notifications": True},
    "user_info": {"age": 30}
}
result2 = complex_state_manager.invoke(update2, config=config)
print(f"更新内容: {update2}")
print(f"返回结果: {result2}")

# 第三次状态更新
print("\n--- 第三次状态更新 ---")
update3 = {
    "user_info": {"location": "上海"},
    "preferences": {"theme": "light"}
}
result3 = complex_state_manager.invoke(update3, config=config)
print(f"更新内容: {update3}")
print(f"返回结果: {result3}")

print("\n--- 查看复杂状态线程状态 ---")
current_state = complex_state_manager.get_state(config)
print(f"当前状态值: {current_state.values}")

print("\n--- 查看复杂状态历史记录 ---")
history = list(complex_state_manager.get_state_history(config))
print(f"历史记录条数: {len(history)}")

for i, state in enumerate(history):
    print(f"\n 历史状态 {i+1}:")
    print(f"    值: {state.values}")
    print(f"    创建时间: {state.created_at}")
    print(f"    步骤: {state.metadata.get('step', 'N/A')}")
```

```
if __name__ == "__main__":
    # 执行所有演示
    accumulate_demo()
    state_management_demo()
    conversation_demo()
    complex_state_demo()
```

5.10 长期记忆

长期记忆允许跨不同的线程 ID 存储信息。这对于在一次对话中了解特定用户的信息并在另一次对话中使用该信息可能很有用。

```
"""
```

基于 LangGraph 1.0 函数式 API 的长期记忆示例

本示例演示了如何在 LangGraph 函数式 API 中实现长期记忆功能，展示了跨不同线程 ID 存储信息的能力。

主要特性：

1. 使用共享存储实现长期记忆
2. 跨会话保持用户信息
3. 使用@entrypoint 装饰器定义工作流入口点
4. 演示多线程 ID 间的信息共享

```
"""
```

```
from typing import Dict, List, Optional
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
import time
import json
```

模拟长期记忆存储（在实际应用中可能是数据库或外部存储）

```
class LongTermMemoryStore:
```

```
    """长期记忆存储类"""
```

```
    def __init__(self):
```

```
        self.storage: Dict[str, Dict] = {}
```

```
    def save_user_info(self, user_id: str, info: Dict) -> None:
```

```
        """
```

```
        保存用户信息到长期记忆
```

```
    参数:
        user_id: 用户 ID
        info: 用户信息字典
    """
    print(f"保存用户 {user_id} 的信息到长期记忆: {info}")
    if user_id not in self.storage:
        self.storage[user_id] = {}
    self.storage[user_id].update(info)

def get_user_info(self, user_id: str) -> Dict:
    """
    从长期记忆获取用户信息

    参数:
        user_id: 用户 ID

    返回:
        用户信息字典
    """
    info = self.storage.get(user_id, {})
    print(f"从长期记忆获取用户 {user_id} 的信息: {info}")
    return info

def list_users(self) -> List[str]:
    """
    列出所有用户

    返回:
        用户 ID 列表
    """
    return list(self.storage.keys())

# 全局长期记忆存储实例
long_term_memory = LongTermMemoryStore()

@task
def extract_user_info(message: str) -> Dict:
    """
    从消息中提取用户信息的任务
```

参数:

`message`: 用户消息

返回:

提取的用户信息字典

"""

```
print(f"从消息中提取用户信息: {message}")
```

```
user_info = {}
```

简单的信息提取逻辑

```
if "我叫" in message or "我是" in message:
```

```
    if "我叫" in message:
```

```
        name_start = message.find("我叫") + 2
```

```
    else:
```

```
        name_start = message.find("我是") + 2
```

提取到下一个标点符号或字符串结尾

```
name_end = len(message)
```

```
for i in range(name_start, len(message)):
```

```
    if message[i] in [",", " ", ".", "。", "!", "!", "?", "? "]:
```

```
        name_end = i
```

```
        break
```

```
name = message[name_start:name_end].strip()
```

```
if name: # 只有非空名称才保存
```

```
    user_info["name"] = name
```

```
if "岁" in message:
```

查找年龄数字

```
age_pos = message.find("岁")
```

向前查找数字

```
age_str = ""
```

```
for i in range(age_pos - 1, -1, -1):
```

```
    if message[i].isdigit():
```

```
        age_str = message[i] + age_str
```

```
    else:
```

```
        break
```

```
if age_str and age_str.isdigit():
```

```
    user_info["age"] = int(age_str)
```

```
if "来自" in message:
    location_start = message.find("来自") + 2
    # 提取到下一个标点符号或字符串结尾
    location_end = len(message)
    for i in range(location_start, len(message)):
        if message[i] in [",", " ", ".", "!", "?", " "]:
            location_end = i
            break
    location = message[location_start:location_end].strip()
    if location: # 只有非空位置才保存
        user_info["location"] = location

print(f"提取到的用户信息: {user_info}")
return user_info
```

@task

```
def generate_response(user_id: str, message: str, user_info: Dict) -> str:
    """
```

生成回复的任务

参数:

user_id: 用户 ID

message: 用户消息

user_info: 用户信息

返回:

生成的回复

"""

```
    print(f"为用户 {user_id} 生成回复, 消息: {message}, 用户信息: {user_info}")
```

根据用户信息生成个性化回复

```
if "你好" in message or "hello" in message.lower():
```

```
    if user_info.get("name"):
```

```
        response = f"你好, {user_info['name']}! 很高兴再次见到你。"
```

```
    else:
```

```
        response = "你好! 我是 AI 助手。能告诉我你的名字吗?"
```

```
elif "我叫" in message or "我是" in message:
```

```
    name = user_info.get("name", "朋友")
```

```
    response = f"很高兴认识你, {name}! 有什么我可以帮助你的吗?"
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
elif "再见" in message or "bye" in message.lower():
    name = user_info.get("name", "朋友")
    response = f"再见, {name}! 期待下次与你交流。"
else:
    # 基于用户信息的个性化回复
    info_parts = []
    if user_info.get("name"):
        info_parts.append(f"名字是{user_info['name']}")
    if user_info.get("age"):
        info_parts.append(f"年龄是{user_info['age']}岁")
    if user_info.get("location"):
        info_parts.append(f"来自{user_info['location']}")

    if info_parts:
        info_summary = ", 而且我知道你" + ", ".join(info_parts)
        response = f"我理解你的问题。{info_summary}。让我来帮助你解答。"
    else:
        response = "我理解你的问题。让我来帮助你解答。"

print(f"生成的回复: {response}")
return response

# 检查点保存器
checkpointer = InMemorySaver()

@entrypoint(checkpointer=checkpointer)
def long_term_memory_chat(inputs: Dict, *, previous_context: Optional[Dict]
= None) -> Dict:
    """
    具有长期记忆的聊天工作流

    参数:
        inputs: 包含 user_id 和 message 的字典
        previous_context: 之前的上下文信息（来自检查点）

    返回:
        包含回复和更新后用户信息的字典
    """
    user_id = inputs["user_id"]
```

```
message = inputs["message"]

print(f"\n=== 启动长期记忆聊天 workflow ===")
print(f"用户 ID: {user_id}")
print(f"消息: {message}")
print(f"之前的上下文: {previous_context}")

# 从长期记忆获取用户信息
stored_user_info = long_term_memory.get_user_info(user_id)

# 合并存储的用户信息和之前的上下文
current_user_info = {}
if stored_user_info:
    current_user_info.update(stored_user_info)
if previous_context:
    current_user_info.update(previous_context)

print(f"当前用户信息: {current_user_info}")

# 提取用户信息
extracted_info = extract_user_info(message).result()

# 更新用户信息
if extracted_info:
    current_user_info.update(extracted_info)
    # 保存到长期记忆
    long_term_memory.save_user_info(user_id, current_user_info)

# 生成回复
response = generate_response(user_id, message,
current_user_info).result()

# 返回回复和更新后的用户信息
result = {
    "user_id": user_id,
    "message": message,
    "response": response,
    "user_info": current_user_info
}

print(f"workflow 结果: {result}")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)


```
# 保存用户信息到检查点（短期记忆）
return entrypoint.final(value=result, save=current_user_info)

def conversation_session_1():
    """第一次对话会话"""
    print("=== 第一次对话会话 ===")

    user_id = "user_001"
    config = {"configurable": {"thread_id": "session_1"}}

    # 第一条消息
    print("\n--- 第一条消息 ---")
    message1 = "你好，我叫张三，来自北京。"
    result1 = long_term_memory_chat.invoke({"user_id": user_id, "message":
message1}, config=config)
    print(f"用户: {message1}")
    print(f"AI: {result1['response']}")
    print(f"当前用户信息: {result1['user_info']}")

    # 第二条消息
    print("\n--- 第二条消息 ---")
    message2 = "我今年 25 岁了。"
    result2 = long_term_memory_chat.invoke({"user_id": user_id, "message":
message2}, config=config)
    print(f"用户: {message2}")
    print(f"AI: {result2['response']}")
    print(f"当前用户信息: {result2['user_info']}")

def conversation_session_2():
    """第二次对话会话（不同的线程 ID）"""
    print("\n\n=== 第二次对话会话（不同线程 ID） ===")

    user_id = "user_001" # 相同的用户 ID
    config = {"configurable": {"thread_id": "session_2"}} # 不同的线程 ID

    # 第一条消息
    print("\n--- 第一条消息 ---")
    message1 = "你好！"
```

```
    result1 = long_term_memory_chat.invoke({"user_id": user_id, "message":
message1}, config=config)
    print(f"用户: {message1}")
    print(f"AI: {result1['response']}")
    print(f"当前用户信息: {result1['user_info']}")

    # 第二条消息
    print("\n--- 第二条消息 ---")
    message2 = "再见！"
    result2 = long_term_memory_chat.invoke({"user_id": user_id, "message":
message2}, config=config)
    print(f"用户: {message2}")
    print(f"AI: {result2['response']}")
    print(f"当前用户信息: {result2['user_info']}")

def conversation_session_3():
    """第三次对话会话（新用户）"""
    print("\n\n=== 第三次对话会话（新用户） ===")

    user_id = "user_002" # 新的用户 ID
    config = {"configurable": {"thread_id": "session_3"}}

    # 第一条消息
    print("\n--- 第一条消息 ---")
    message1 = "你好，我是李四，来自上海。"
    result1 = long_term_memory_chat.invoke({"user_id": user_id, "message":
message1}, config=config)
    print(f"用户: {message1}")
    print(f"AI: {result1['response']}")
    print(f"当前用户信息: {result1['user_info']}")

def demonstrate_long_term_memory():
    """演示长期记忆功能"""
    print("=== 长期记忆功能演示 ===")

    print("\n当前长期记忆中的用户:")
    users = long_term_memory.list_users()
    for user_id in users:
        user_info = long_term_memory.get_user_info(user_id)
```

```
print(f" {user_id}: {user_info}")

def multi_user_demo():
    """多用户演示"""
    print("\n\n=== 多用户演示 ===")

    # 用户 1
    print("\n--- 用户 1 对话 ---")
    user1_id = "user_003"
    config1 = {"configurable": {"thread_id": "user1_session"}}
    message1 = "你好，我是王五，28 岁。"
    result1 = long_term_memory_chat.invoke({"user_id": user1_id, "message":
message1}, config=config1)
    print(f"用户 1: {message1}")
    print(f"AI: {result1['response']}")

    # 用户 2
    print("\n--- 用户 2 对话 ---")
    user2_id = "user_004"
    config2 = {"configurable": {"thread_id": "user2_session"}}
    message2 = "你好，我是赵六，来自深圳。"
    result2 = long_term_memory_chat.invoke({"user_id": user2_id, "message":
message2}, config=config2)
    print(f"用户 2: {message2}")
    print(f"AI: {result2['response']}")

    # 用户 1 再次对话
    print("\n--- 用户 1 再次对话 ---")
    message3 = "再见！"
    result3 = long_term_memory_chat.invoke({"user_id": user1_id, "message":
message3}, config=config1)
    print(f"用户 1: {message3}")
    print(f"AI: {result3['response']}")

    # 用户 2 再次对话
    print("\n--- 用户 2 再次对话 ---")
    message4 = "我今年 30 岁了。"
    result4 = long_term_memory_chat.invoke({"user_id": user2_id, "message":
message4}, config=config2)
    print(f"用户 2: {message4}")
```

```
print(f"AI: {result4['response']}")

def check_memory_status():
    """检查内存状态"""
    print("\n\n=== 最终内存状态 ===")

    print("\n长期记忆存储内容:")
    users = long_term_memory.list_users()
    if users:
        for user_id in users:
            user_info = long_term_memory.get_user_info(user_id)
            print(f" {user_id}: {user_info}")
    else:
        print(" 没有存储的用户信息")

    print("\n检查点状态:")
    # 检查各个会话的状态
    sessions = ["session_1", "session_2", "session_3", "user1_session",
"user2_session"]
    for session in sessions:
        try:
            config = {"configurable": {"thread_id": session}}
            state = long_term_memory_chat.get_state(config)
            if state.values:
                print(f" {session}: {state.values}")
            else:
                print(f" {session}: 无状态")
        except Exception as e:
            print(f" {session}: 无法获取状态 ({e})")

if __name__ == "__main__":
    # 执行演示
    conversation_session_1()
    conversation_session_2()
    conversation_session_3()
    demonstrate_long_term_memory()
    multi_user_demo()
    check_memory_status()
```

5.11 常见陷阱及解决方法

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

5.11.1 处理副作用

在 LangGraph 的工作流中，当工作流被中断并恢复时，如果副作用直接写在工作流中而不是封装在任务里，这些副作用会被重复执行，可能导致：

- 文件被重复写入或覆盖
- 邮件被重复发送
- 数据库记录被重复创建

正确处理方式：

将副作用封装在 `@task` 装饰的函数中，这样即使工作流被中断并恢复，任务也不会被重复执行。

```
"""
基于 LangGraph 1.0 函数式 API 的副作用处理简单示例

这个示例更展示了如何正确封装副作用。
"""

from langgraph.func import entrypoint, task
from langgraph.types import Command, interrupt
from langgraph.checkpoint.memory import InMemorySaver

# 正确的方式：将副作用封装在 task 中
@task
def write_to_file():
    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("Side effect executed")
    return "文件写入完成"

@entrypoint(checkpointer=InMemorySaver())
def my_workflow(inputs: dict) -> dict:
    # 正确的方式：副作用被封装在任务中
    result = write_to_file().result()
    value = interrupt("question")
    return {
        "file_result": result,
        "user_response": value
    }
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
# 错误的方式：直接在工作流中执行副作用
@entrypoint(checkpointer=InMemorySaver())
def my_incorrect_workflow(inputs: dict) -> dict:
    # 错误的方式：副作用直接包含在工作流中
    # 当恢复工作流时，会再次执行这个副作用
    with open("output_incorrect.txt", "w", encoding="utf-8") as f:
        f.write("Side effect executed")
    value = interrupt("question")
    return {
        "warning": "副作用会重复执行",
        "user_response": value
    }

def demo():
    """演示正确和不正确的副作用处理方式"""
    print("=== 副作用处理演示 ===\n")

    # 正确方式演示
    print("1. 正确的副作用处理方式:")
    config = {"configurable": {"thread_id": "correct-demo"}}

    for event in my_workflow.stream({"input": "test"}, config):
        print(f"    事件: {event}")

    # 恢复执行
    for event in my_workflow.stream(Command(resume="answer"), config):
        print(f"    恢复事件: {event}")

    print("\n2. 不正确的副作用处理方式:")
    config2 = {"configurable": {"thread_id": "incorrect-demo"}}

    for event in my_incorrect_workflow.stream({"input": "test"}, config2):
        print(f"    事件: {event}")

    # 恢复执行（副作用会重复执行）
    for event in my_incorrect_workflow.stream(Command(resume="answer"),
config2):
        print(f"    恢复事件: {event}")
```

```
print("    注意：文件会被重复写入！")

if __name__ == "__main__":
    demo()
```

5.11.2 非确定性控制流

每次运行结果可能不同的操作（如获取当前时间、随机数）需封装在任务中，确保恢复时返回相同结果。若不封装，恢复时可能产生不同结果，影响 workflow 正确性。例如，workflow 用当前时间决定执行哪个任务，不封装在任务中会导致不同时间运行结果不同。

```
"""
基于 LangGraph 1.0 函数式 API 的非确定性控制流简单示例

这个示例说明如何正确处理非确定性操作。
"""

import time
import random
from langgraph.func import entrypoint, task
from langgraph.types import Command, interrupt
from langgraph.checkpoint.memory import InMemorySaver

# 模拟耗时任务
@task
def slow_task(task_id: int) -> str:
    """模拟耗时任务"""
    return f"任务 {task_id} 完成"

# 正确的方式：将非确定性操作封装在 task 中
@task
def get_time() -> float:
    """获取当前时间的任务"""
    return time.time()

# 不正确的非确定性处理方式
@entrypoint(checkpointer=InMemorySaver())
def my_incorrect_workflow(inputs: dict) -> dict:
    """不正确处理时间的工作流"""
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
t0 = inputs["t0"]
# 错误：直接在工作流中获取时间
t1 = time.time() # 恢复时会重新获取时间，可能导致不同结果

delta_t = t1 - t0

if delta_t > 1:
    result = slow_task(1).result()
    value = interrupt("question")
else:
    result = slow_task(2).result()
    value = interrupt("question")

return {
    "result": result,
    "value": value
}
```

正确的非确定性处理方式

```
@entrypoint(checkpointer=InMemorySaver())
def my_correct_workflow(inputs: dict) -> dict:
    """正确处理时间的工作流"""
    t0 = inputs["t0"]
    # 正确：将获取时间的操作封装在任务中
    t1 = get_time().result() # 恢复时会返回相同的时间值

    delta_t = t1 - t0

    if delta_t > 1:
        result = slow_task(1).result()
        value = interrupt("question")
    else:
        result = slow_task(2).result()
        value = interrupt("question")

    return {
        "result": result,
        "value": value
    }
```



```
def demo():
    """演示正确和不正确的非确定性处理方式"""
    print("=== 非确定性控制流处理演示 ===\n")

    # 正确方式演示
    print("1. 正确的非确定性处理方式:")
    config = {"configurable": {"thread_id": "correct-demo"}}
    inputs = {"t0": time.time()}

    for event in my_correct_workflow.stream(inputs, config):
        print(f"    事件: {event}")

    # 恢复执行
    for event in my_correct_workflow.stream(Command(resume="answer"),
config):
        print(f"    恢复事件: {event}")

    print("\n2. 不正确的非确定性处理方式:")
    config2 = {"configurable": {"thread_id": "incorrect-demo"}}
    inputs2 = {"t0": time.time()}

    for event in my_incorrect_workflow.stream(inputs2, config2):
        print(f"    事件: {event}")

    # 恢复执行（非确定性操作会重新执行）
    for event in my_incorrect_workflow.stream(Command(resume="answer"),
config2):
        print(f"    恢复事件: {event}")
        print("    注意: 时间会被重新获取, 可能导致不同分支执行!")

if __name__ == "__main__":
    demo()
```